www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

DESIGN AND ANALYSIS OF ALGORITHMS.

SYLLABUS.

Unit-1:

Introduction: Algorithm, Pseudo code for expressing algorithms, Performance analysis - Space complexity, Time complexity, Asymptotic notation - Big oh notation, Omega notation, Theta notation and Little oh notation, Probabilistic analysis, Amortized complexity.

Divide and conquer: General method, applications - Binary Search, quick sort, merge sort, strassen's matrix multiplication

Unit-2:

Searching and traversal technique: Efficient non-recursive binary tree traversal algorithms, disjoint set operations, union and find algorithms, Spanning trees, Graph traversals - Breadth first search and Depth first search, AND/OR graphs, game trees, Connected components, Bi-connected components.

Unit-3:

Greedy method: General method, applications - Job Sequencing with deadlines, 0/1 knapsack problem, Minimum cost Spanning trees, Single source Shortest path problem.

Dynamic programming: General method, applications - Multistage graphs, Optimal binary search trees, 0/1 knapsack problem, All pairs shortest path problem, travelling sales person problem, Reliability methodor design.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Unit-4:

Backtracking: General method, applications: n-queen proble sum of subsets problem, graph coloring, Hamiltonian cycles.

Brands and Bound: General method, applications-Traveling sales person problem, 0/1 knapsack problem-LC Branch and bound solution, FIFO branch and bound solution

Unit-5:

NP-Hard and NP- Complete problems: Basic concepts, Non-deterministic algorithms, NP-Hard and NP-complet classes, NP-Hard problems, Cook's theorem.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

14/12/16

UNIT-I

ALGORITHM INTRODUCTION.

* Algorithm:

An algorithm is a finite set of instructions that performs a particular task. The word 'algorithm' comes from the name of a persian author Abu Ja'far Mohammed ibn mosa al khowarizmi (c. 825 A.D)

* Attributes or properties or characteristics of algorithm:

All algorithms must satisfy the following criteria

1. Input:

An algorithm may take zero or more number of inputs.

2. Output:

An algorithm must produce atleast one output.

3. Definiteness:

Each instruction must be defined clearly and unambiguous.

4. Finiteness:

An algorithm must be terminated successfully

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

after a finite number of steps.

5. Effectiveness:

Performing airthmetic operations on integers is an example of effective operation. But, performing arithemetic operations with real numbers is not effective.

Example for definiteness:

```
int a=5, b=10, c;
c=a+b; // defined clearly
add 6 or 7 to b // not defined clearly.
```

20/12/16.

* Areas to study an algorithm:

→ There are four areas to study an algorithm

1. How to devise algorithms.
2. How to validate algorithms.
3. How to analyse algorithms
4. How to test a program.

1. How to devise algorithms:

→ Devise an algorithm means designing an algorithm or writing or creating or developing an algorithm

→ There are different design techniques to

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

develop an algorithm.

1. Divide and conquer.
2. Greedy method.
3. Dynamic programming
4. Back tracking.
5. Branch and bound.

2. How to validate algorithm:

→ Once an algorithm is designed it is necessary to show that it computes the correct answer for all legal inputs. This process is known as algorithm validation.

→ Once the validity of a algorithm has been shown a program can be written and a second phase begins. This phase is called as program verification or program proving.

3. How to analyze algorithm:

→ This field of study is also known as analysis of algorithms or performance analysis.

Performance analysis:

→ performance analysis refers to a task of determining how much computing time and storage an algorithm requires.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

4. How to test a program:

Testing a program consists of two phases.

1. Debugging
2. Profiling/performance measurement.

1. Debugging:

→ Debugging is the process of executing process on sample data sets.

→ To determine whether faulty results occur and, if so to correct them.

2. Profiling/performance measurement:

→ Performance measurement is the process of executi a correct program on data sets and measuring the time and space, it takes to compute the results.

* pseudo code:

→ Pseudo code is one of the way to describe an algorithm.

→ Pseudo code uses both natural and programming language to describe an algorithm.

→ Pictorial representation of an algorithm is know

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

as flowchart.

→ Textual representation of an algorithm is known as pseudo code.

* Pseudo code specifications:

1. Comments begins with // and continue until the end of the line.
2. Blocks are indicated with matching braces { and }. A compound statement (collection of statements) can be represented as a block. The body or procedure or function also forms a block.
3. An identifier begins with a letter.
4. Assignment of values to variables is done using assignment statement.

   Assignment of

   (variable) := (expression) ;

5. There are two boolean values true and false. In order to produce these values the logical operators and, or, and not and relational operations <, >, ≤, ≥, ≠ are provides
6. Elements of multi-dimensional arrays are accessed using '[' and ']'

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

7. The following loop statements are used for, whi

do-while (repeat-until).

→ The while loop takes the following form

```
while (condition) do
 {
  statement 1;
     :
     :
  statement n;
 }
```

→ The for loop takes the following form

```
for var:= value1 to value2 step step do
 {
  statement 1;
     :
     :
  statement n;
 }
```

→ The repeat-until statement takes the following f

```
repeat
 {
  statement 1
     :
  statement 2
 } until (condition);
```

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

8. A conditional statement has the following forms

Ex: 1. if (condition) then ~~statement;~~
statement;

2. if (condition) then

Statement 1

else

Statement 2

9. Input and output are done using the instructions. read and write.

10. There is only one type of procedure Algorithm.

An algorithm consists of a heading and body. The heading takes the form

Algorithm Name (parameter list)

{

.....

}

22/12/16

* Recursive algorithms:

→ There are two types of recursive algorithms.

1. Direct recursive algorithms
2. Indirect recursive algorithms.

1. Direct recursive algorithms:

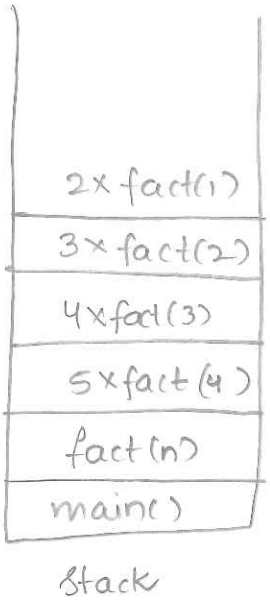
An algorithm called by itself known as direct

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

recursive algorithms

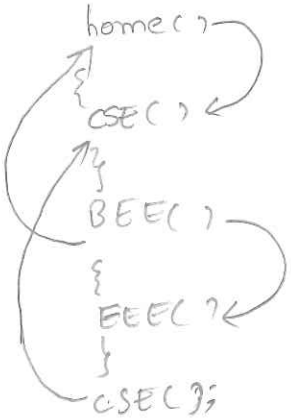
```
void main()
{
  .....
f = fact(n);        → actual arguments
  .....
}
int fact (int n)    → formal arguments
{
if(n==1)
return 1;
else
 return (n*fact(n-1));
}
```



Stack

2. Indirect recursive algorithms:

```
void main()
 {
  A();
 }
 A()
 {
  B();
 }
```



www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

```
B( )
{
 A( );
}
```

* Performance Analysis:

→ Determining time and space an algorithm requires is known as performance analysis.

→ Performance analysis contains two components.

1. Time complexity
2. Space complexity.

1. Time Complexity:

→ Time required for an algorithm to perform a given task is known as time complexity.

(or)

→ Number of steps required for an algorithm to ~~performance~~ a given task is known as time Complexity.

→ Time complexity = compile time + Run time

→ Compile time does not depends on ~~instance~~ characteristics. ~~statistics~~ (input values).

→ Compile time is constant or fixed

→ Execution time depends on instance characteristics

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

(input values).

→ Run time is not fixed that means it varies.

→ We can determine time complexity of an algorithm in two ways.

1. Table method [using frequency]
2. Count variable method

1. Table method (using frequency):

→ First, determine the number of steps for execution (s/e) of the statement.

→ Determine the total number of times each statement is executed.

→ Number of times a statement executes is known as frequency of that statement.

→ By combining this two quantities (i) Step for execution ((s/e), frequency) (ii) frequency, the total continuation of each statement is obtained.

→ By adding the contributions of all the statements, step count for entire algorithm is obtained.

Ex: 1. Find ~~table count~~ time complexity for the given algorithm using table method.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

L.No	Statement	s/e	frequency	total steps
1	Algorithm Sum(a,n)	0	-	0
2	{	0	-	0
3	s:=0.0;	1	1	1
4	for i:=1 to n do	1	n+1	n+1
5	s:=s+a[i];	1	n	n
6	return s;	1	1	1
7	}	0	-	0
				2n+3

→ Time complexity of this algorithm is 2n+3.

→ 2n+3 is number of steps required for this algorithm to perform the task.

→ If n=5 then 2n+3 = 13

if n=10 then 2n+3 = 23

where n is instance characteristics (input value)

→ when 'n' value increases, time complexity also increases (run time).

2) Determine the time complexity for the given algorithm addition of two matrices.

Sol:-

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

S.No	Statement	s/e	frequency	total steps
1.	Algorithm add(a,b,c,m,n)	0	-	0
2.	{	0	-	0
3.	for i=1 to m do	1	m+1	m+1
4.	for j=1 to n do	1	m(n+1)	mn+m
5.	c[i,j]=a[i,j]+ b[i,j];	1	mn	mn
6.	}	0	-	0
				2m+2mn+1

3) Write an algorithm to find n^{th} fibonacci number and find time complexity.

Sol:-

	Statement	s/e	frequency n≤1	frequency n>1	total steps n≤1
1	Algorithm Fibonacci(n)	0	-	-	-
2	{	0	-	-	-
3	if (n≤1) then	1	1	1	1
4	write(n);	1	1	0	1
5	else	0	-	-	0
6	{	0	-	-	0
7	f2:=0; f1:=1;	2	0	1	0
8	for i:=2 to n do	1	0	n	0
9	{	0	-	-	0
10	fn =f1+f2;	1	0	n-1	0
11	f2=f1; f1=fn;	2	0	n-1	0
12	}	0	-	-	0
13	write (fn);	1	0	1	0
14	}	0	-	-	0
15	}	0	-	-	0
					2

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Sl.No	Statement	s/e	frequency	total steps
1.	Algorithm add(a,b,c,m,n)	0	-	0
2.	{	0	-	0
3.	for i=1 to m do	1	m+1	m+1
4.	for j=1 to n do	1	m(n+1)	mn+m
5.	c[i,j]=a[i,j]+ b[i,j];	1	mn	mn
6.	}	0	-	0
				2m+2mn+1

3) Write an algorithm to find n^{th} fibonacci number and find time complexity.

Sol:-

	Statement	s/e	frequency n≤1	frequency n>1	total steps n≤1	total steps n>1
1	Algorithm Fibonacci(n)	0	-	-	-	-
2	{	0	-	-	-	-
3	if (n≤1) then	1	1	1	1	1
4	write(n);	1	1	0	1	0
5	else	0	-	-	0	0
6	{	0	-	-	0	0
7	f2:=0; f1:=1;	2	0	1	0	2
8	for i:=2 to n do	1	0	n	0	n
9	{	0	-	-	0	0
10	fn =f1+f2;	1	0	n-1	0	n
11	f2=f1; f1=fn;	2	0	n-1	0	2n
12	}	0	-	-	0	0
13	write (fn);	1	0	1	0	1
14	}	0	-	-	0	0
15	}	0	-	-	0	0
					2	4n

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ To analyise the time complexity of an algorithm we need to consider two cases $n \leq 1$ and $n > 1$

case i : $n \leq 1$

when $n \leq 1$, lines 3 and 4 get executed once each. Since each line has an steps for execution of one(1) so total step count for this case is 2 that means time complexity is 2 (constant).

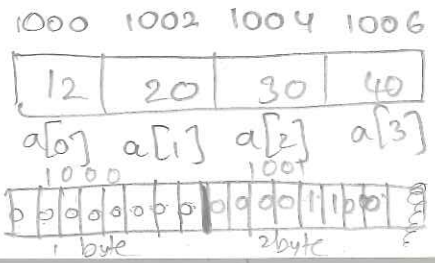
```
2  for(i=1; i<=n; i++)   | n+1 | n
                         |  n  | n-1
   Pf("%d", i);
```

case ii : $n > 1$

The total steps for this case is $4n+1$.

4. Write an algorithm to find sum of elements of an array using recursion and find time complexity?

S.No	Statement	s/e	frequency $n \leq 0$	frequency $n > 0$	total steps $n \leq 0$	total steps $n > 0$
1	Algorithm Rsum(a,n) (a: array of elements, n: size)	0	-	-	0	0
2	{	-	-	-	-	-
3	if (n ≤ 0) then	1	1	1	1	1
4	return 0;	1	1	0	1	0
5	else	-	-	-	-	-
6	return (Rsum(a,n-1)+a[n]);	1+x	0	1	0	1+x
7	}	0	-	-	0	0
					2	2+x



$x = t_{Rsum(n-1)}$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

To analyze the time complexity of the algorithm a need to consider 2 cases.

case (i): [$n \leq 0$]:

when $n \leq 0$ lines 3, 4 executed once each. Since each lin has s/e of 1. Total step = 2 i.e Time complexity = 2.

case (ii) $n > 0$

The total step count for this case is $x+2$

where $x = t_{Rsum}(n-1)$

27/12/16

2. Count variable method:

→ In this method, count variable is used in the program.

→ Count variable is a global variable initialized with a value 'zero.'

→ statements to increment count variable by the appropriate amount are introduced into the program. This

→ This is done so that each time a statement original program is executed. Value of count variable is incremented by the step count of that statement.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Ex:

```
Algorithm Sum(a,n)
{
s:=0.0;
count=count+1;  // count is global, it is intially zero
for i:=1 to n do
{
count:=count+1;  // for loop
s:=s+a[i];
count:=count+1;
}
Count:=count+1; // last time of for loop
count:=count+1; // for the return statement
return s;
}
```

Output:
n=5
s=13

* Space Complexity:

→ Space complexity is the amount of memory it means to run.

→ Space needed by an algorithm is the sum of the following components.

1. Fixed part
2. Variable part.

1. Fixed part:

→ A fixed part that consisting of the space needed by instructions, variables, constants etc

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

2. Variable part:

→ A variable part that consists of the space need by reference variables and the recursion stack space and instance characteristics.

→ The space requirement $S(P)$ of any algorithm 'P' may be written as $S(P) = C + S_P$(instance characteristics)

where C is constant.

→ When analysing the space complexity of an algorithm we concentrate on estimating instance characteristics (S_P).

→ For any given problem we need to determine which instance characteristics to use to measure the space requirements.

Ex: 1. Find space complexity for given algorithm.

Iterative function for sum of array of elements

```
Algorithm Sum(a,n)
{
 s:=0.0
 for i:=1 to n do
 s:=s+a[i];
 return s;
}
```

→ The given algorithm consisting of three variables (n, i, s) and one array 'a'.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ The space needed by each variable is 1 word since it is of integer type. So, 3 words of space required for three variables.

→ The space needed by an array 'a' is n words. Since it contains 'n' elements.

→ The space complexity of this algorithm is

$$S(P) = C + S_p$$

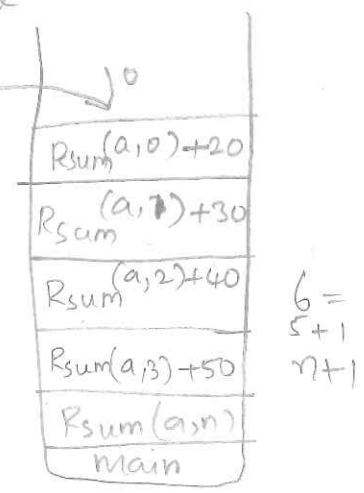
$$S_{Sum(n)} \geq 3 + n$$

2. Find the space complexity for given algorithm.
[Sum of array of elements using recursive function].

```
Algorithm Rsum (a,n)
{
 if (n ≤ 0) then
 return 0;
 else
 return (Rsum (a,n-1) +a[n]);
}
```

↳ formal parameter

a[0]



→ This algorithm consists of recursive function. So recursive function uses stack concept.

→ The recursion stack space includes space for the formal parameters, local variables and the ~~written~~ return address.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ Assume that the return address requires only one word of memory.

→ Each call to Rsum requires atleast three words [value of n, return address and pointer to array 'a'].

→ Since the depth of recursion is n+1. So recursion Stack space needed is $\geq 3(n+1)$

* → Performance evaluation can be divided into two major phases.

1. Priori estimates
2. Posteriori testing

1. Priori estimates:

→ These are also known as performance analysis. Estimating time and space required for an algorithm to perform a task is known as performance analysis. This can be performed before execution.

2. Posteriori testing:

→ It is also known as performance measurement. Estimating time and space required for a program to perform a task is known as performance measurement. This can be performed during execution of program.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

*Best case, worst case, Average case in time complexity:

Best case:

Minimum number of steps executed on a given parameter (n) is known as best case

Ex: 1. Time complexity of linear search in best case is O(1)

2. Time complexity of binary search in best case is O(1)

Average case:

Average number of steps executed on a given parameter is known as average case.

Ex: 1. Time complexity of linear search in average case is O(n)

2. Time complexity of binary search in average case is O(logn)

Worst Case:

Maximum number of steps executed on a given parameter is known as worst case.

Ex: 1. Time complexity of linear search in worst case

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

is O(n).

2. The time complexity of binary search in worst case is O(logn).

* Order of growth:

Measuring performance of an algorithm in relation with input size(n) is known as order of growth.

n	logn	n logn	n^2	2^n
1	0	0	1	2
2	1	2	4	4
4	2	8	16	16
8	3	24	64	256
16	4	64	256	65536
32	5	160	1024	4294967296.

Example for best case, worst case and average case

1. Here n=5 10 20 30 40 50

Linear Search:

→ Given input is 10. One comparison is required to search an element 10. So time complexity of linear search in best case is O(1).

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ Given input is 50. '5' comparisons is required to search the element 50. Here input size n=5. So time complexity of linear search is O(n).

→ Given input values are 10, 20, 30, 40, 50 .. 1, 2, 3, 4, 5 comparisons are required to search the elements 10, 20, 30, 40, 50 respectively.

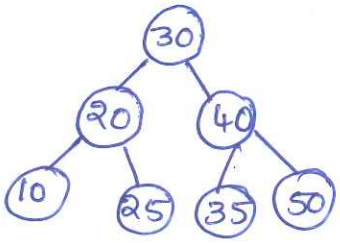
$1+2+3+4+5 = 15$

$Avg = 15/5 = 3$

$$\frac{n(n+1)}{2n} = \frac{n+1}{2} = \frac{n}{2} + \frac{1}{2} = O(n)$$

neglected neglected

2. Consider the elements 10, 20, 25, 30, 35, 50, 40.



Binary Search:

→ Given input is 30. '1' comparison is required to search this element. So time complexity of binary search is O(1).

→ Given input is 10. '3' comparisons required to search element 10. So time complexity of binary

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Search in worst case is O log n.

$\log_2 7 = 2.81$

$= 3$ (approximately)

→ 1, 2, 2, 3, 3, 3, 3 comparisons are required to search the elements 30, 20, 40, 10, 25, 35, 50 ~~10, 20, 25, 30, 35, 40, 50~~. respectively.

$1+2+2+3+3+3+3 = 17$ total comparisons

$Avg = \frac{17}{7} = 2.42 = 3$ (Approximately)

* Asymptotic Notation (O, Ω, Θ):

Time complexity can be expressed in different notations known as asymptotic notations. They are

1. Big "oh" (O)
2. Omega (Ω)
3. Theta (Θ)
4. Little "oh" (o)
5. Little Omega

1. Big "Oh" Notation:

→ Big "oh" notation, the function $f(n) = O(g(n))$ [read as $f(n)$ is big oh of $g(n)$] if and only if there exists positive constants c and n_0. Such that

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$f(n) \leq c * g(n)$ for all n, $n \geq n_0$.

Problem:

1. Convert the given function $f(n) = 2n+3$ into big O notation.

$f(n) = 2n+3$

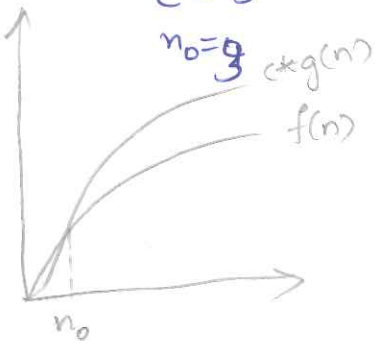
$f(n) = O(g(n))$

$2n+3 = O(g(n))$

$2n+3 = O(n)$

$C = 3$

$n_0 = 3$



$f(n) \leq C * g(n)$

$2n+3 \leq C \cdot n$

$2n+3 \leq 3n$

$5 \leq 3$ (F) $n=1$

$7 \leq 6$ (F) $n=2$

$9 \leq 9$ (T) $n=3$

$11 \leq 12$ (T) $n=4$.

→ This statement $f(n) = O(g(n))$ states only that $g(n)$ is an upper bound value of $f(n)$ for all $n \geq n_0$.

→ Value of 'c' must be least as possible.

→ The function $2n+3 = O(n)$ iff there exists positive constants $c=3$ and $n_0 = 3$ such that $2n+3 \leq 3n$ for all n, $n \geq 3$.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | universityupdates.in | https://telegram.me/jntuoh

2. Convert the function $f(n)=3n+2$ into big "oh" notation?

(4)

$f(n)=O(g(n))$ iff .....

$f(n)=3n+2$

$f(n)=O(g(n))$

$3n+2=O(n)$

$c=4$

$n_0=2$

$f(n) \le c*g(n)$

$3n+2 \le c \cdot n$

$3n+2 \le 4n$

$5 \le 4$ (F) $n=1$

$8 \le 8$ (T) $n=2$

$11 \le 12$ (T) $n=3$

→ The function $3n+2=O(n)$ iff there exists positive constants c and n_0 $f(n) \le c*g(n)$ for all n, $n \ge 2$.

3. Convert the function $f(n)=100n+6$ into big "oh" notation.

$f(n)=O(g(n))$

$f(n)=100n+6$

$f(n)=O(g(n))$

$100n+6=O(n)$

$c=101$

$n_0=6$ $[n \ge n_0, n \ge 6]$

$f(n) \le c*g(n)$

$100n+6 \le c \cdot n$

$100n+6 \le 101 \cdot n$

$n=6.$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

4. Convert the function $f(n) = 10n^2 + 4n + 2$ into big "oh" notation.

$f(n) = 10n^2 + 4n + 2$

$f(n) = O(g(n))$

$10n^2 + 4n + 2 = O(n^2)$

$c = 11$

$n_0 = 5$

$f(n) \leq c * g(n)$

$10n^2 + 4n + 2 \leq c \cdot n^2$

$10n^2 + 4n + 2 \leq 11n^2$

$16 \leq 11$ (F) $n = 1$

$50 \leq 44$ (F) $n = 2$

$n = 3$

$n = 4$

$272 \leq 275$ $n = 5$

5. $f(n) = 6 \cdot 2^n + n^2$ into big "oh" notation.

$f(n) = 6 2^n + n^2$

$f(n) = O(g(n))$

$6 \cdot 2^n + n^2 = O(2^n)$

$c = 7$

$n_0 = 4$

$f(n) \leq c * g(n)$

$6 \cdot 2^n + n^2 \leq c \cdot 2^n$

$6 \cdot 2^n + n^2 \leq 7 \cdot 2^n$

$13 \leq 14$ (T) $n = 1$

$28 \leq 28$ (T) $n = 2$

$57 \leq 56$ (F) $n = 3$

$112 \leq 112$ (T) $n = 4$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

6. $f(n) = 2n^2+n^3$ into 'O' notation. (8)

$f(n) = O(g(n))$

$2n^2+n^3 = O(n^3)$

$c=2$

$n_0=2$

$f(n) \leq c * g(n)$

$2n^2+n^3 \leq 2n^3$

$3 \leq 2(F)$ $n=1$

$16 \leq 16(T)$ $n=2$

$45 \leq 54(T)$ $n=3$

27
18

* Time complexities names:

Time Complexity	Names
$O(1)$	constant
$O(n)$	linear
$O(n^2)$	Quadratic
$O(n^3)$	cubic
$O(\log n)$	logn
$O(n \log n)$	nlogn
$O(2^n)$	Exponent

Order of time complexities:

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n)$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

2. Omega Notation (Ω):

A function $f(n) = \Omega g(n)$ iff there exists positive constants c and n_0 such that $f(n) \geq c * g(n)$ $\forall n, n \geq n_0$.

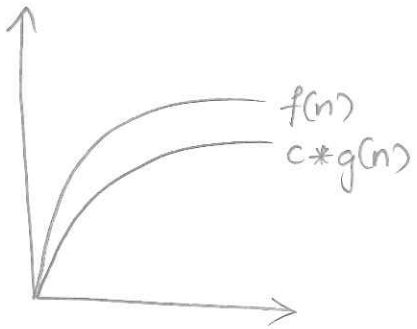
Problems:

1. Convert the given function $f(n) = 3n+2$ into Omega.

$f(n) = 3n+2$
$f(n) = \Omega(g(n))$
$3n+2 = \Omega(n)$
$c = 3$

$f(n) \geq c * g(n)$
$f(n) \geq c \cdot n$
$3n+2 \geq 3n$
$5 \geq 3$ $n=1$ T
$8 \geq 6$ $n=2$ T
$11 \geq 9$ $n=3$ T

$n_0 = 1$ $\begin{pmatrix} n \geq 1 \\ n \geq n_0 \end{pmatrix}$



→ The function $3n+2 = \Omega(n)$ iff there exist positive constants $c=3, n_0=1$ such that $3n+2 \geq 3n$ $\forall n, n \geq 1$.

→ The function $g(n)$ is only a lower bond on $f(n)$
↔ value of c must be maximum as possible.

2. Convert the function $2n+3$ into Ω.

$f(n) = 2n+3$
$f(n) = \Omega(g(n))$
$2n+3 = \Omega(n)$
$c = 2$
$n_0 = 1$ $\begin{pmatrix} n \geq 1 \\ n \geq n_0 \end{pmatrix}$

$f(n) \geq c * g(n)$
$2n+3 \geq 2n$ $[\because c=2]$
$2n+3 \geq 2n$
$5 \geq 2$ $n=1$ T
$8 \geq 4$ $n=2$ T
$9 \geq 6$ $n=3$ T

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

3. $10n^2+4n+2$.

$f(n) = 10n^2+4n+2$

$10n^2+4n+2 = \Omega(n^2)$

$C=10$, $n_0=1$ $\left(\begin{matrix} n \geq 1 \\ n \geq n_0 \end{matrix}\right)$

$f(n) \geq C*g(n)$

$10n^2+4n+2 \geq Cn^2$ $[\because C=10]$

$16 \geq 10$ $n=1$ T

$50 \geq 40$ $n=2$ T

$104 \geq 90$ $n=3$

4. 6.2^n+n^2

$f(n) = 6.2^n+n^2 = \Omega(g(n))$

$6.2^n+n^2 = \Omega(2^n)$

$f(n) \geq C*g(n)$

$6.2^n+n^2 \geq C.2^n$

$6.2^n+n^2 \geq 6.2^n$

$13 \geq 12$ $n=1$

$28 \geq 24$ $n=2$

$57 \geq 48$ $n=3$

$C=6, n_0=1$ $\left(\begin{matrix} n \geq 1 \\ n \geq n_0 \end{matrix}\right)$

3. Theta (Θ) Notation:

The function $f(n) = \Theta(g(n))$ iff there exist positive constants c_1, c_2, and n_0 such that

$$c_1*g(n) \leq f(n) \leq c_2*g(n) \quad \forall\, n_1, n \geq n_0$$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

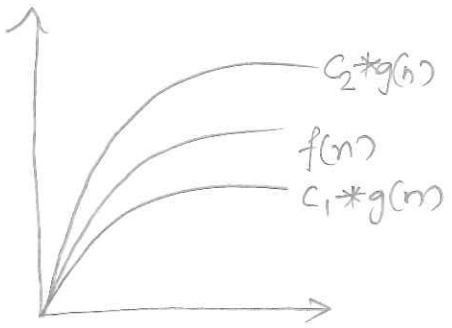
1. Convert the function $3n+2$ into 'Θ' notation.

Sol: Given

$f(n) = 3n+2$

$f(n) = \Theta(g(n))$

$3n+2 = \Theta(n)$



$c_1*g(n) \le f(n) \le c_2*g(n)$

$f(n) \le c_2*g(n)$

$3n+2 \le c_2*n$

$3n+2 \le 4n$

$5 \le 4$ F $n=1$

$8 = 8$ T $n=2$

$11 \le 12$ T $n=3$

$c_2 = 4, n_0 = 2.$

$\begin{bmatrix} n \ge 2 \\ n \ge n_0 \end{bmatrix}$

$c_1*g(n) \le f(n)$

$c_1*n \le 3n+2$

$3n \le 3n+2$

$3 \le 5$ $n=1$ T

$6 \le 8$ $n=2$ T

$c_1 = 3, n_0 = 1$

$\begin{bmatrix} n \ge 1 \\ n \ge n_0 \end{bmatrix}$

$c_1 = 3, c_2 = 4$

$n_0 = 1, 2, 3, \ldots\ldots, n_0 = 2, 3, 4 \ldots\ldots$

→ The function $3n+2 = \Theta(n)$ iff there exists positive constants $c_1 = 3$, $c_2 = 4$ & $n_0 = 2$ such that

$3n \le 3n+2 \le 4n \quad \forall n, n \ge 2$

→ Note: The 'Θ' notation is more precious than both O & 'Ω' notations.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ The $f(n)$ is $\Theta(g(n))$ iff $g(n)$ is both an upper and lower bound on $f(n)$.

4. Little 'oh'(o):

The function $f(n) = o(g(n))$ iff there exists

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$$

1. Convert $3n+2$ into little 'oh'.

$f(n) = 3n+2$

$f(n) = o(g(n))$

$3n+2 = o(n^2)$

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$$

$$\lim_{n\to\infty} \frac{3n+2}{n^2} = \lim_{n\to\infty} \frac{3n}{n^2} + \frac{2}{n^2}$$

$$= \lim_{n\to\infty} \frac{3\cdot n}{n\cdot n} + \frac{2}{n\cdot n}$$

$$= \lim_{n\to\infty} 3\times\frac{1}{n} + 2\times\frac{1}{n}\times\frac{1}{n}$$

$$= 0+0 = 0.$$

5. Little 'Omega' (ω):

The function $f(n) = \omega(g(n))$ iff $\lim_{n\to\infty} \frac{g(n)}{f(n)} = 0$.

Problem:

1. Write an algorithm for selection sort and find time complexity.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

S.No		s/e	frequency	total frequency
1.	Algorithm Selectionsort (a,n)	0	-	-
2	{	0	-	-
3	for i:=1 to n do	1	n+1	n+1
4	{	0	-	-
5	j:=i;	1	n	n
6	for k:=i+1 to n do	1	n×n	n^2
7	if (a[k]<a[j]) then	1	n(n-1)	n^2-n
8	j:=k;	1	n(n-1)	n^2-n
9	t:=a[i]; a[i]:=a[j]; a[j]:=t;	3	n	3n
10	}	0	-	-
11	}	0	-	-
				$3n^2+3n+1$

→ $3n^2+3n+1$ is number of steps required for Selection Sort algorithm to perform the given task.

→ Here $f(n) = 3n^2+3n+1$ convert this function into big 'oh' notation

$f(n) = O(g(n))$

$f(n) = O(n^2)$

$f(n) \leq c * g(n)$

$3n^2+3n+1 \leq c \cdot n^2$

$c = 4$

$n_0 = 4$

$7 \leq 4$	$n=1$
$19 \leq 16$	$n=2$
$37 \leq 36$	$n=3$
$61 \leq 64$	$n=4$

→ Time complexity of selection sort is big 'oh' notation.

$3n^2+3n+1 = O(n^2)$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* Towers of Hanoi problem is an example of recursive algorithm:

→ There was a diamond tower (labelled 'A') with 64 golden disks. The disks were of decreasing size and were stacked on the tower in decreasing order of size from bottom to top.

→ Besides this tower there were two other diamond towers [labelled 'B' & 'C'].

→ Since the time of creation Brahman priests have been attempting to move the disk from tower 'A' to tower 'B' using tower 'C' were intermediate stories.

→ As the disks were very heavy they can moved only one at a time.

→ In addition, at no time can a disk be on a top of smaller disc

```
Algorithm Towers of Hanoi (n, x, y, z)
{
if (n ≥ 1) then
{
 Towers of Hanoi (n-1, x, z, y);
 write ("move top disk from tower", x, " to top of
                    tower", y);
```

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

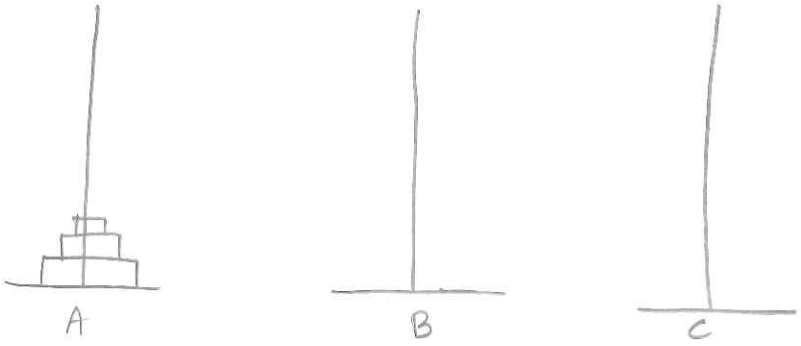
www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Towers of Hanoi(n-1, z, y, x);

}
}

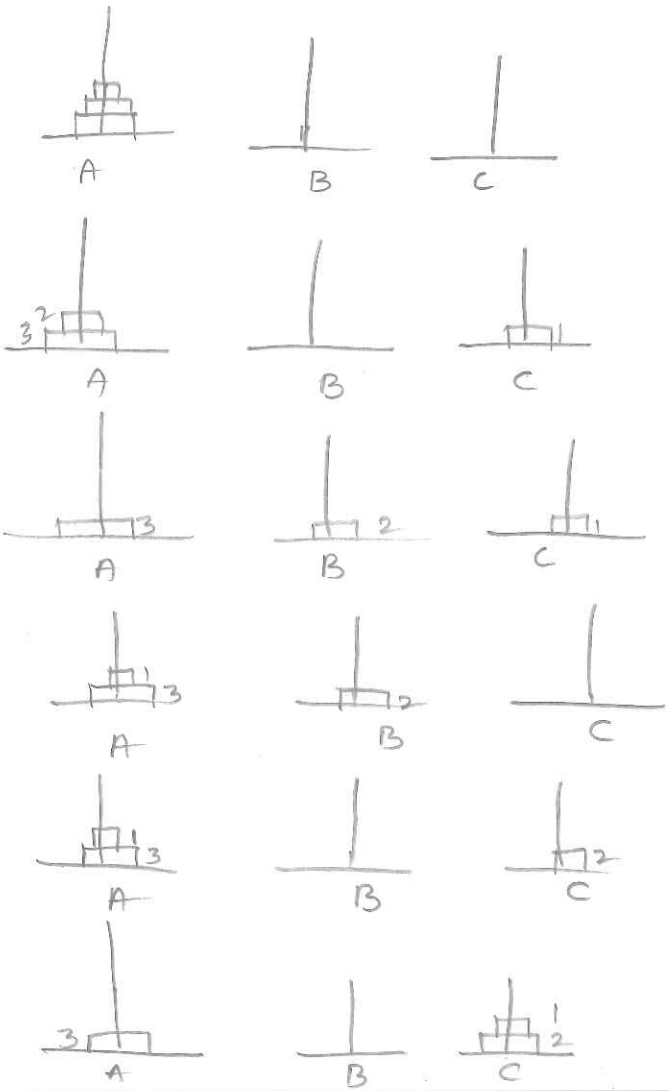
Recurrence Relation of towers of Hanoi is $2T(n-1)+1$

→ Assume that tower 'A' consists of 3 discs and move discs from 'A' to 'B' by using 'c' as intermediate

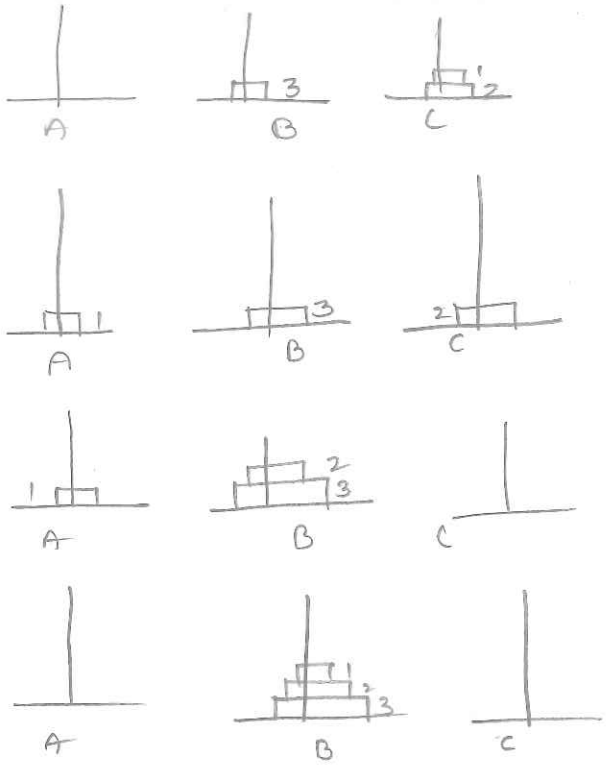


one-move
Heavy - Small

Sol:-



www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh



*Divide and conquer:

→ Divide and conquer is a method to develop an algorithm. In this method, if the problem is large, then it divides the problem into k-subproblems.

→ If the sub-problem is still large then divide the sub-problem into smaller sub problems.

→ Apply divide and conquer strategy ~~and~~ on sub-problems until sub problem becomes small enough to find the answer and find solutions of each and every sub-problem. Atlast combine solutions of the sub-problem to get the solution for given problem.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* Control abstraction or general method for divide and conquer:

```
Algorithm  D and C(P)
{
if (Small (P)) then
    return S(P);
else
{
divide P into smaller instances P1, P2, .... Pk, k ≥ 1;
Apply D and C to each of these subproblems.
return combine (D and C (P1), D and C (P2) ... D and C (Pk));
}
}
```

→ Control abstraction consists of three methods

1. Small (P)
2. S(P)
3. Combined function.

1. Small(P):

→ Small (P) is a boolean function that determines whether the problem is small or not. If the problem is small then boolean function returns true then the function S(P) is invoked to find the answer.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ If the problem is large, boolean function returns false. Now the problem is divided into smaller sub problems. These sub problems $P, P_2, \ldots$ $\ldots P_k$ are solved by recursive applications of D And C.

→ Combine is a function that determines the solution to the given problem 'p' by using solutions of k-sub problems.

→ If the size of the problem 'p' is n and the size of the k-sub problems are $n, n_2, \ldots n_k$ respectively. Then computing time of D And C is described by the recurrence relation

$$T(n) = \begin{cases} g(n) & n\text{-small} \\ T(n_1) + T(n_2) + \ldots\ldots + T(n_k) + f(n) & n \geq 1 \end{cases}$$

where $T(n)$ = Time complexity of divide and conquer algorithm on any input size of 'n'.

$g(n)$ = Time complexity of the algorithm for smaller inputs

$f(n)$ = The function $f(n)$ is the time for dividing the problem 'p' and

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

combining these solutions to sub problems.

→ The time complexity of divide and conquer algorithms is given by recurrence relation

$$T(n) = \begin{cases} T(1) & n=1 \\ aT(n/b) + f(n) & n>1 \end{cases}$$

where a, b are known constants and

$n = b^k$

→ One of the methods for solving such recurrence relation is called the substitution method.

1) Consider the case $a=1, b=2, f(n)=1$ and solve the recurrence relation.

$T(n) = 1 \cdot T(n/2) + 1$

$T(n) = T(n/2) + 1$

$T(n/2) = T((n/2/2) + 1) + 1$

$= (T(n/4) + 1) + 1 = T(n/4) + 2$

$T(n/4) = (((T(n/4/2) + 1) + 1) + 1)$

$= T(n/8) + 3$

$= T(n/2^3) + 3$

$\vdots$

$= T(n/2^k) + k$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$= T(n/n) + \log_2 n$

$= T(1) + \log_2 n$

$= 1 + \log_2 n$

$= O(\log n)$

$n = 2^k$

$k = \log_2 n$

2) Consider the case in which $a=2$ and $b=2$. let $T(1)=2$ and $f(n)=n$

Sol:- $T(n) = 2T(n/2) + n$

$T(n) = 2T(n/2) + n$

$= 2[2T(n/4) + n/2] + n$

$= 2[2[2T(n/8) + n/4] + n/2] + n$

$= 8T(n/8) + n + n + n$

$= 2^3 T(n/2^3) + 3n$

$\vdots$

$= 2^k T\left(\frac{n}{2^k}\right) + kn$

$= n T(n/n) + n \log_2 n$

$= nT(1) + n\log_2 n$

$= 2n + n\log_2 n.$

$n = b^k = 2^k$

$k = \log_2 n$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

3. Solve the recurrence relation for the following choices of a, b and f(n), is a constant.

a) a=1, b=2, f(n)=cn

b) a=5, b=4, f(n)=cn²

(a) a=1, b=2, f(n)=nc

T(1)=2

$T(n) = a \cdot T(n/b) + f(n)$

$= 1 \cdot T(n/2) + cn$

$T(n/2) = (1 \cdot T(n/4) + c\,n/2) + cn$

$= T(n/4) + c\,n/2 + nc$

$T(n/8) = [[1 \cdot T(n/8) + c\,n/4] + cn/2] + nc]$

$= T(n/8) + \frac{nc}{4} + \frac{nc}{2} + nc$

$= 1 \cdot T(n/8) + \frac{7n}{4}$

$= T(n/2^3) + \frac{7}{4}n$

$= T(1) + \frac{7}{4}n$

$= 1 + 7/4\,n$

$b = 2$

$n = b^k$

$n = 2^k$

$k = \log_2 n$

b) a=5, b=4, f(n)=cn²

$T(n) = a \cdot T(n/b) + f(n)$

$= 5 \cdot T(n/4) + cn^2$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$$T(n) = 5 \cdot T(n/4) + cn^2$$

$$T(n/4) = 5\left[5 \cdot T(n/4/4) + c(n/4)^2\right] + cn^2$$

$$T(n/16) = \left[5\left[5\left[5T(n/16/4) + c(n/16)^2\right] + c(n/4)^2\right] + cn^2\right]$$

$$= 5\left[5\left[5T(n/64) + c(n^2/256)\right] + \frac{n^2}{16}\right] + n^2\Big]$$

$$= 5^3 T(n/4^3) + \frac{n^2 + 16n^2 + 256n^2}{256}$$

$$=$$

* Divide and conquer Applications:

- This design technique is used in following applications

1. Binary search
2. Quick sort
3. Merge sort
4. Strassen's matrix multiplication

1. Binary search:

→ Binary search is a searching technique. It takes list of elements and searching element as inputs.

→ It compares an element to be searched with middle element. If searching element is less than

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

the middle element then searching process continues in the left part of the list.

→ If the searching element is greater than middle element searching process continues in the right part of the list.

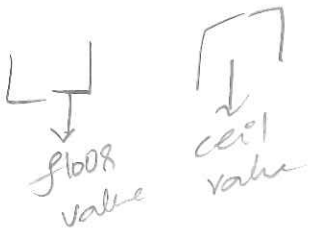
→ This process continues until the end of the list.

*Note:

List of elements divided into two parts based on middle element.

middle element = $\frac{low+high}{2}$

i) write an algorithm for binary search using Iterative method (Non-recursive method)




```
Algorithm Binsearch (a, n, x)
{
 low := 1;
 high := n;
while (low ≤ high) do
{
 mid : ⌊(low + high)/2⌋;
 if (x < a[mid]) then
    high = mid - 1;
 else if (x > a[mid]) then
  low = mid + 1;
 else
```

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

```
        return mid;
    }
}
```

Time complexity:

$T(n) = 1 \cdot T(n/2) + 1$

$= T(n/2) + 1$

$= T(n/4) + 1 + 1$

$= T(n/8) + 1 + 1 + 1$

$= T(n/8) + 3$

$= T(n/2^3) + 3$

$\vdots$

$= T(n/2^k) + k$

$= T(n/n) + \log_2 n$ $\left[\because n = b^k,\ n = 2^k,\ k = \log_2 n\right]$

$= T(1) + \log_2 n$

$= 1 + \log n$

$= O(\log n)$

Binary search using recursive function:

```
Algorithm Binsearch (a, i, l, x)
{
 if (l == i) then
 if (x == a[i]) then
   return i;
```

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

```
else
 return 0;
 }
else
{
mid= ⌊(i+l)/2⌋;
 if (x==a[mid]) then
   return mid;
 else if (x<a[mid]) then
   return Binsearch (a, i, mid-1, x);
 else
 return Binsearch (a, mid+1, l, x);
  }
 }
```

2) Consider the following elements -15, -6, 0, 7, 9, 23, 54, 82, 101, 112, 125, 131, 142, 151. Construct binary decision tree or binary search tree for given elements.

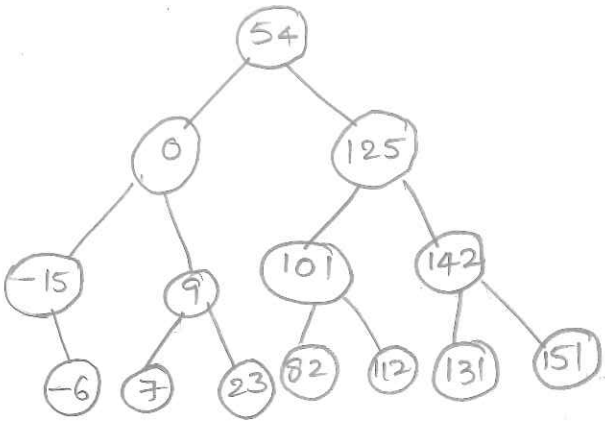
a-	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]
elements-	-15,	-6,	0,	7,	9,	23,	54,	82,	101,	112,	125,	131,	142,	151

$$mid = \left\lfloor \frac{l+h}{2} \right\rfloor = \left\lfloor \frac{1+14}{2} \right\rfloor = \left\lfloor \frac{15}{2} \right\rfloor = \lfloor 7.5 \rfloor = 7.$$

Comparisons-	3	4	2	4	3	4	1	4	3	4	2	4	3	4

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh



$n = 14$

$\log_2 14 = 3.21$

Best - O(1)

Worst - O(logn)

→ In the above binary search tree, minimum number of comparisons required to search an element 54 is 1. So time complexity of binary search in best case on successful search is O(1).

→ Maximum number of comparisons required to search the elements -6, 7, 23, 82, 112, 131, 151 is 4.

$n = 14$

$\log n = \log_2 14 = 3.8$ (Approximately $\cong 4$)

Time complexity of binary search in worst case on successful search is O(logn).

→ Average number of comparisons required to find all 14 elements is

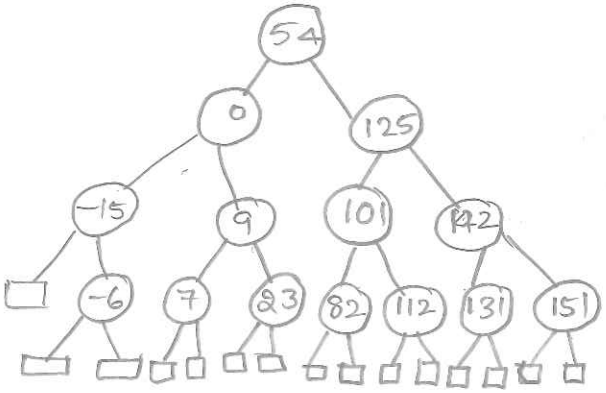
$$\frac{3+4+2+4+3+4+1+4+3+4+2+4+3+4}{14} = 3.21 \text{ (Approx} \cong 4)$$

Time complexity of binary search in average case on successful search is O(logn).

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

*Time complexity of binary search on unsuccessful search:



→ Circular nodes are called internal nodes.

→ Square nodes are refered as External nodes.

→ All successful searches end at a circular node.

→ All unsuccessful searches end at a square node.

→ There are 15 possible ways that an unsuccessful search may terminate depending on value of 'x'.

→ Average number of elements comparison for an unsuccessful search is $(3+14\times4)/15 = 59/15 = 3.93$

(Approxi ≅ 4)

→ Time complexity of binary search in best case on unsuccessful search is O(logn)

→ Time complexity of binary search in Avg case on unsuccessful search is O(logn)

→ Time complexity of binary search in worst case on unsuccessful search is O(logn).

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* Merge Sort:

→ In this method given sequence of a elements divided into two sets and each set individually sorted and the resulting sorted sequence are merged to produce a single sorted sequence of n elements.

Ex: Consider the array of 10 elements.

a[1:10] = (310, 285, 179, 652, 351, 423, 861, 254, 450, 520)

→ Algorithm merge sort begins by splitting a[1:10] into two subarrays each of size five (a[1:5] and a[6:10])

→ The elements in a[1:5] are then splitted into 2 subarrays of size 3 (a[1:3]) and two a[4:5]

→ Then elements in a[1:3] are splitted into subarrays of size two (a[1:2]) and one a[3:3].

→ The elements in a[1:2] are split into one element solution and now the meging begins.

→ Note that no moment of data has yet taken place

→ A record of the subarrays is implicity maintained by the reaursive mechanisms.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

led

,520)

nto

10])

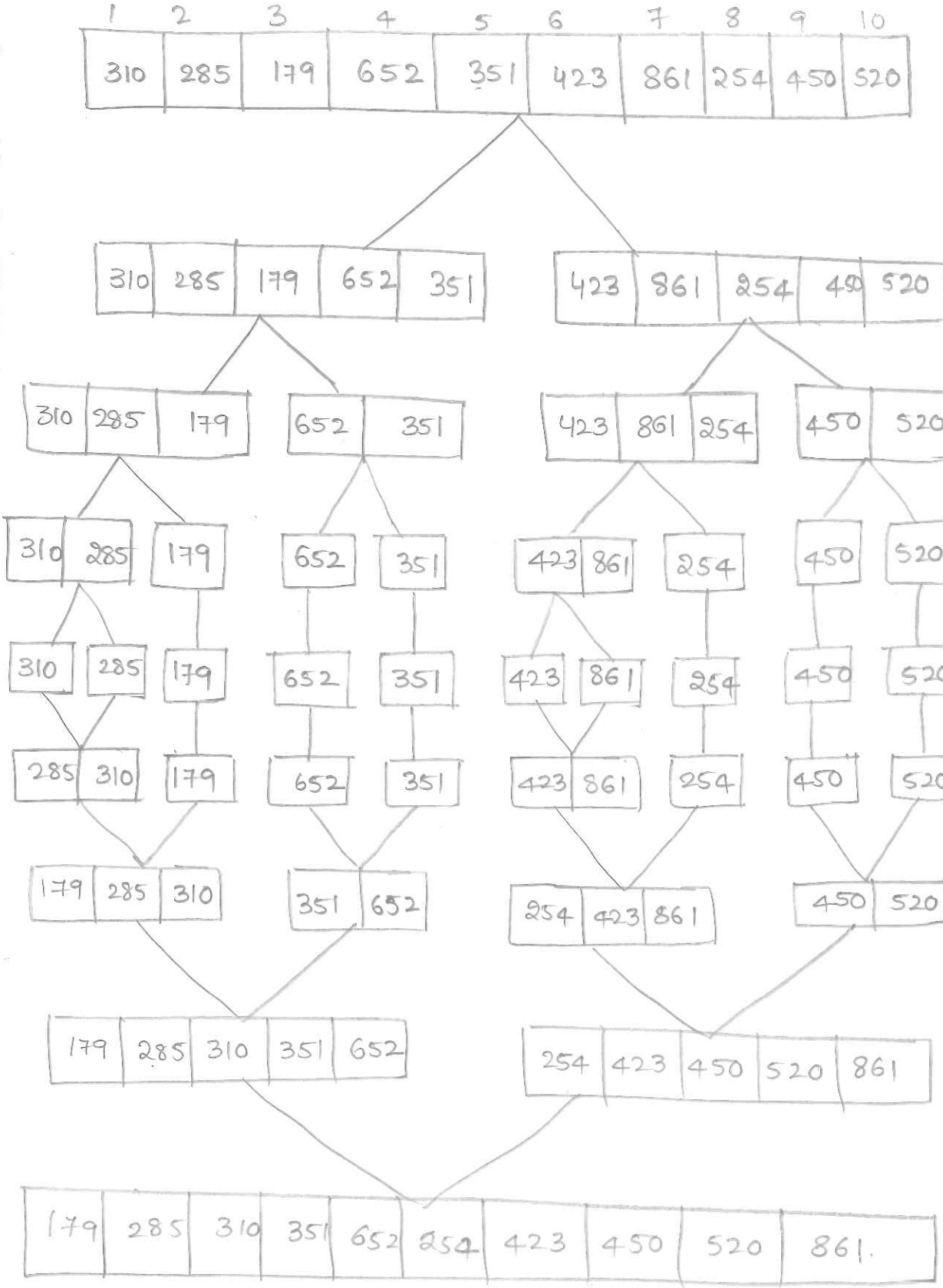
barays

s of

t

place

ined



www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

```
Algorithm Mergesort (low, high)
{
 if (low < high)
 {
   mid = (low + high)/2;
   merge sort (low, mid);
   merge sort (mid+1, high);
   merge sort (low, mid, high);
  }
 }
Algorithm merge (low, high, mid)
{
 i = low; j = mid+1; k = low;
 while ((i ≤ mid) and (j ≤ high)); do
 {
 if (a[i] ≤ a[j]) then
 {
  b[k] = a[i]; i = i+1;
 }
 else
 {
  b[k] = a[j]; j = j+1;
 }
 k = k+1;
 }
 if (i > mid) then
 for m = j to high do
 {
```

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

```
b[k]=a[m];
k=k+1;
}
else
for m=i to mid do
{
b[k]=a[m];
k=k+1;
}
}
```

Time complexity:

$T(n) = a \cdot T(n/b) + f(n)$

$= 2 \cdot T(n/2) + c \cdot n$

$= 2(2T(n/4) + n/2) + n$

$= 2(2(2T(n/8) + n/4) + n/2) + n$

$= 8T(n/8) + n + n + n$

$= 2^3 T(n/2^3) + 3n$

$= 2^5 T(n/2^3) + 3n$

$= 2^k T(n/n) + n \cdot k$

$= n \cdot T(1) + n \log n$

$= n + n \log n$

$= n(1 + \log n)$

$n = b^k$

$n = 2^k$

$k = \log_2 n$

www.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Time complexity of merge sort is O(nlogn)

Time complexity of bubble sort, selection sort, insert sort is $O(n^2)$

*Quick sort:

Quick sort is an efficient sorting method. Divide and conquer method used in quicksort

Description:

→ First element is considered as pivot element, partioning element.

→ place the pivot element in correct position in such a way that left side elements should be less than pivot element and right side elements should be greater than pivot element.

→ Repeat this process for left part and right part of the pivot element until elements are sorted.

→ Consider the following example

(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(i)	(P)
65	70	75	80	85	60	55	50	45	2	9
65	45	75	80	85	60	55	50	70	3	8
65	45	50	80	85	60	55	75	70	4	7
65	45	50	55 / 80	85	60	80	75	70	5	6

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

65 45 50 55 60 85 80 75 70 | 6 5

sort

60 45 50 55 65 85 80 75 70

→ First element 65 is considered as pivot element. place the pivot element 65 in the correct position in such a way that left side element should be less than the pivot element and the right side element should be greater than the pivot element. The list is divided into two parts left part contains elements 60, 45, 50, 55 and right part contains 85, 80, 75, 70

→ Each part is sorted separately.

Algorithm:

```
Algorithm Quicksort(p,q)
{
if (p<q) then
{
j:= partition (a,p,p+1);
Quicksort (p,j-1);
Quicksort (j+1,q);
}
}
```

```
Algorithm partition (a,i,n)
{
pivot = a[i]; lb= i ,ub=n+1;
```

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

```
repeat
{
repeat
 lb=lb+1;
until (a[lb] ≥ pivot);
repeat
 ub:=ub-1;
until (a[ub]≤ pivot);
if (lb<ub) then interchange (a, lb, ub);
}until (lb≥ub);
 a[i]=a[ub]; a[ub]=pivot ; return ub;
}
Algorithm Interchange (a,lb,ub)
{
 temp=a[lb];
 a[lb]=a[ub]; a[ub]=temp;
 }
```

17/1/17:

* Strassen's matrix multiplication:

Basic matrix multiplication: This requires 8 multiplications and 4 additions to find product of 2 matrices.

Consider the following ex:

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$C_{11} = (A_{11} \cdot B_{11}) + (A_{12} \cdot B_{21})$

$C_{12} = (A_{11} \cdot B_{12}) + (A_{12} \cdot B_{22})$

$C_{21} = (A_{21} \cdot B_{11}) + (A_{22} \cdot B_{21})$

$C_{22} = (A_{21} \cdot B_{12}) + (A_{22} \cdot B_{22})$.

→ Note that no. of elements $n=8$

→ Matrix A contains $n/2$ elements (4)

→ Matrix B contains $n/2$ elements (4)

→ To find product of 2 matrices A and B we need to perform 8 multiplications and 4 additions.

→ Overall computing time $T(n)$ of the resulting of divide and conquer is given by the reference.

$$T(n) = \begin{cases} b & n \leq 2 \\ 8 \cdot T(n/2) + cn^2 ; & n > 2 \end{cases}$$

→ Two matrices can be added in time cn^2 for some constant 'c'.

→ This recurrence relation can be solved using substitution method to obtain $T(n) = O(n^3)$.

→ Volkers strazen has discovered a way to compute the product of two matrices using only 7 multiples and 18 additions / substractions.

www.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$P = (A_{11} + A_{22})(B_{11} + B_{22})$

$Q = (A_{21} + A_{22}) B_{11}$

$R = A_{11} (B_{12} - B_{22})$

$S = A_{22} (B_{21} - B_{11})$

$T = (A_{11} + A_{12}) B_{22}$

$U = (A_{21} - A_{11}) \cdot (B_{11} + B_{12})$

$V = (A_{12} - A_{22}) (B_{21} + B_{22})$

$C_{11} = P + S - T + V$

$C_{12} = R + T$

$C_{21} = Q + S$

$C_{22} = P + R - Q + U$

→ The resulting recurrence relation for $T(n)$ is

$$T(n) = \begin{cases} b & n \leq 2 \\ 7T(n/2) + cn^2 & n > 2 \end{cases}$$

$T(n) = 7T(n/2) + n^2$

$= 7\left[7 \cdot T(n/4) + (n/2)^2\right] + n^2$

$= 7\left[7\left[7T(n/8) + (n/4)^2\right] + (n/2)^2\right] + n^2$

$= 7^3 T(n/8) + 7^2 \frac{n^2}{4^2} + 7 \frac{n^2}{2^2} + n^2$

$= 7^3 T(n/2^3) + n^2\left[\left(\frac{7}{4}\right)^2 + \frac{7}{4} + 1\right]$

$= 7^K T(n/2^K) + n^2\left[\left(\frac{7}{4}\right)^{K-1} + \ldots\ldots + \left(\frac{7}{4}\right)^2 + \left(\frac{7}{4}\right)^1\right]$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$= 7^{\log_2 n} + (n/n) + n^2 \left[(7/4)^{\log_2 n} \right]$ $\quad [a^{\log_c b} = b^{\log_c a}]$

$= n^{\log_2 7} + (1) + n^2 \dfrac{7^{\log_2 n}}{4^{\log_2 n}}$

$= n^{\log_2 7} \cdot 1 + n^2 \dfrac{n^{\log_2 7}}{n^{\log_2 4}}$

$= n^{\log_2 7} + n^2 \dfrac{n^{\log_2 7}}{n^2}$

$= n^{\log_2 7} + n^{\log_2 7}$

$= 2n^{\log_2 7}$

$= O(n^{\log_2 7})$

$= O(n^{2.81})$

www.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

18/1/17

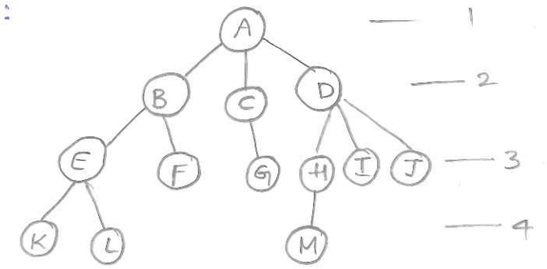
UNIT-II

SEARCHING AND TRAVERSAL TECHNIQUE.

*Tree:

A tree is a finite set of one or more nodes such that there is a special node called the root node and the remaining nodes are partitioned into $n \geq 0$ disjoint sets. $T_1, T_2, \ldots T_n$. The sets $T_1, T_2, \ldots T_n$ are called the sub-trees of the root node.

Ex:



* Terminology:

*Degree: The number of subtrees of a node is called its degree.

Ex: The degree of root node 'A' is 3.

The degree of node 'C' is 1.

*Degree of a tree:

A degree of tree is the maximum degree of nodes

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

of the tree.

→ The degree of above tree is '3'.

* Terminal nodes or Leaf nodes:

→ Nodes that have degree '0' are called leaf or terminal nodes. (or)

→ Nodes that doesn't have child nodes called leaf or terminal nodes

→ In above tree, leaf nodes are K, L, F, G, M, I, J

* Non-terminal nodes:

→ Nodes that have one or more child nodes are called non-terminal nodes.

B, C, D .... are example of non-terminal nodes.

* Siblings:

→ Children of the same parent are said to be siblings

Ex: H, I, J are siblings of D.

* Ancestors:

→ The ancestors of nodes are all the nodes along the path from the root node to that node.

Ex: The ancestors of M are A, D, H.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* Level:

The level of node is defined by, Initially the root node will be at level 1. If a node is at level P then its child nodes are at level P+1.

* Height or depth of a tree:

Height or depth of a tree is defined to be the maximum level of any node in the tree.

Ex: The height of given tree is 4.

* Forest:

A forest is set of $n \geq 0$ disjoint trees.

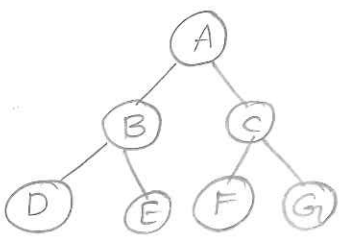
* Binary trees:

→ A Binary tree is a finite set of nodes that is either empty or contains root node and two disjoint binary trees called left and right subtrees.

(or)

→ Binary tree is a tree in which each node may have zero or one or two nodes exactly.

Ex:



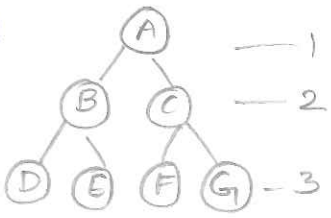
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* Properties:

→ The maximum no. of nodes on level i of a binary tree is 2^{i-1}

Ex:



At level i=3, max no. of nodes $2^{i-1} = 2^{3-1} = 2^2 = 4$.

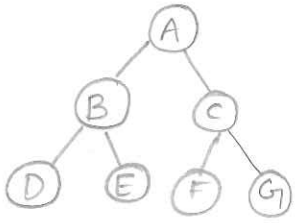
→ The maximum no. of nodes in a binary tree of depth k is $2^k - 1$

Ex: Maximum no. of nodes in a binary tree is k=3

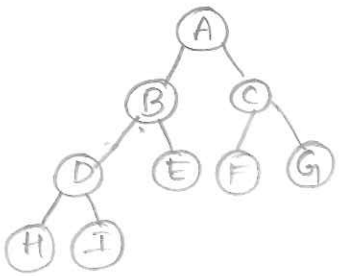
$2^k - 1 = 2^3 - 1 = 8 - 1 = 7$.

* Full binary tree:

→ The binary tree of depth 'k' that has exactly $2^k - 1$ nodes. is called full binary tree of depth 'k'.



It is a full binary tree.



It is not full binary tree because it doesn't have $2^k - 1$ $2^4 - 1 = 15$ nodes.

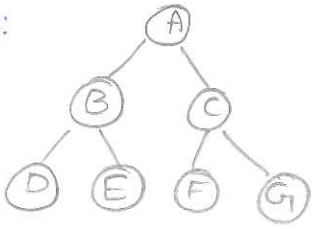
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

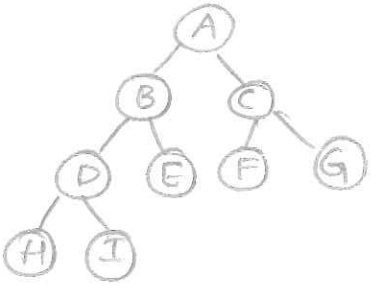
www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* Complete binary tree:

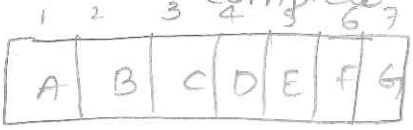
→ If a binary tree is represented using arrays and if it doesn't contain empty location then the tree is known as complete binary tree.

Ex:

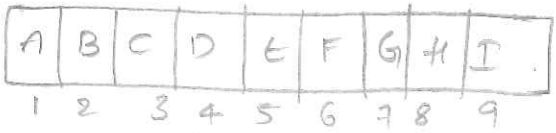




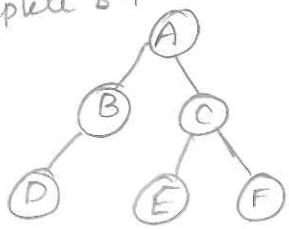
Complete binary tree



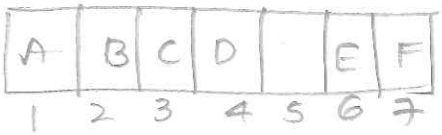
1. Full B.T
2. Complete B.T



1. Full B.T
2. Complete B.T



Not complete binary tree



1. Not full B.T
2. Not complete B.T

Note:

→ All full binary trees are complete binary trees but all complete binary trees are not full binary trees.

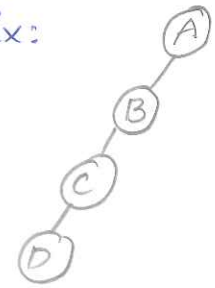
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* Left skewed binary tree:

→ If nodes of a tree placed in left side to its parent then such type of tree is called left skewed binary tree.

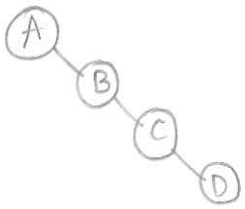
Ex:



* Right skewed binary tree:

→ If nodes of a tree placed in right side to its parent then such type of tree is called right skewed binary tree.

Ex:



* Tree traversal techniques (Binary tree):

→ Traversal is a process of visiting all the nodes of the tree.

→ There are three traversal techniques in binary tree.

1. In-order traversal
2. Pre-order traversal

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

3. Post order traversal.

1. In-order traversal:

→ In this traversal method, the following steps used to visit all the nodes of the binary tree.

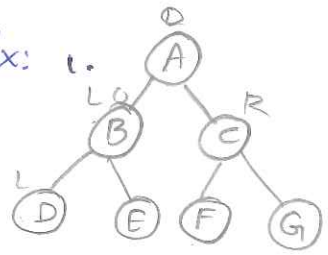
Algorithm:

Step 1: Traverse left subtree

Step 2: Visit root node

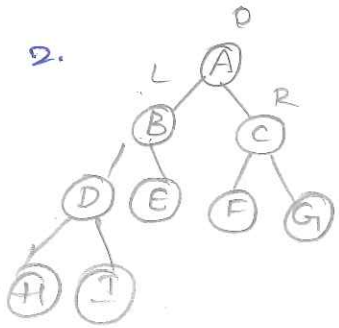
Step 3: Traverse Right Subtree.

Ex: 1.



D, B, E, A, F, C, G - Inorder.

2.



H, D, I, B, E, A, F, C, G

2. Pre-order traversal:

→ In this traversal method following steps are used to visit all the nodes of a binary tree.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

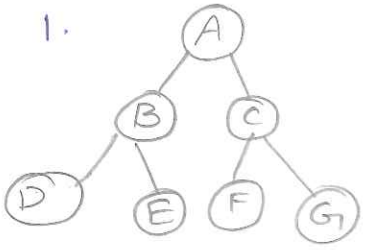
Algorithm:

Step 1: visit root node

Step 2: traverse left subtree

Step-3: traverse right subtree

Ex: 1.



A, B, D, E, C, F G.

3. Post-order traversal:

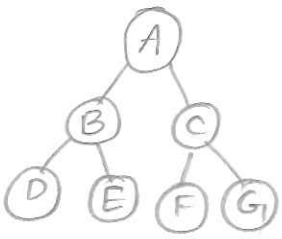
→ In this traversal method following steps are used to visit all the nodes of the binary tree.

Algorithm:

Step 1: traverse left subtree

Step 2: traverse right subtree

Step 3: visit root node.



D, E, B, F, G, C, A

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

*Disjoint sets:

→ If the sets doesn't contain common elements then such types of sets known as disjoint sets.

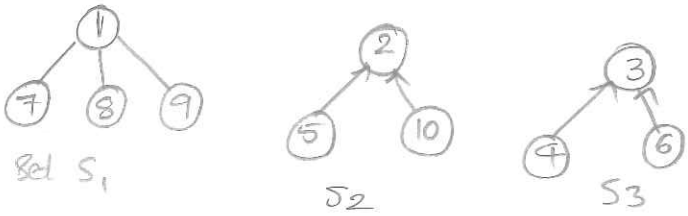
Ex: when n=10, the elements can be partitioned into 3 disjoint sets.

$S_1 = \{1, 7, 8, 9\}$

$S_2 = \{2, 5, 10\}$

$S_3 = \{3, 4, 6\}$

→ This sets can be represented in tree format.



These are the possible tree representation of sets is:

* Disjoint sets operations:

→ The following operations can be performed on disjoint set.

1. Disjoint set union
2. Find (i)

1. Disjoint set union:

To obtain the union of two sets, set the root node of one tree as child node of another tree.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

*Disjoint sets:

→ If the sets doesn't contain common elements then such type of sets known as disjoint sets.

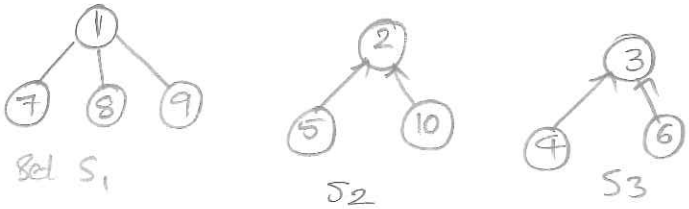
Ex: when $n = 10$, the elements can be partitioned into 3 disjoint sets.

$S_1 = \{1, 7, 8, 9\}$

$S_2 = \{2, 5, 10\}$

$S_3 = \{3, 4, 6\}$

→ This sets can be represented in tree format.



These are the possible tree representation of sets is:

* Disjoint sets operations:

→ The following operations can be performed on disjoint set.

1. Disjoint set union
2. Find (i)

1. Disjoint set union:

To obtain the union of two sets, set the root node of one tree as child node of another tree.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

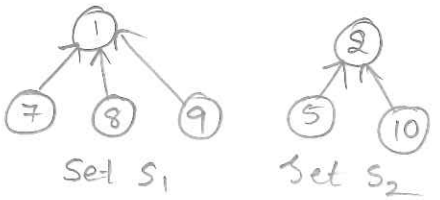
www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

(or)

If S_i and S_j are two disjoint sets then their union $S_i \cup S_j$ = all elements 'x' such that 'x' is in either S_i or S_j.

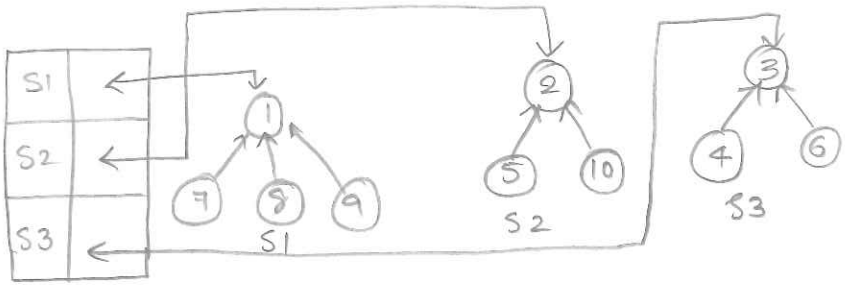
Ex: $S_1 = \{1, 7, 8, 9\}$

$S_2 = \{2, 5, 10\}$



Ans:

→ Data representation for S_1, S_2 and S_3

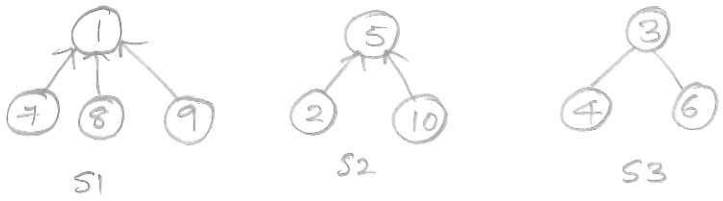


→ Each root node has a pointer to the set name. To determine which set an element is currently in, we follow parent links to the root node of its tree.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

*Array representation of S_1, S_2 and S_3 sets:



i	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]
P	-1	5	-1	3	-1	3	1	1	1	5

→ We represent the tree nodes using an array P[1.....n] where n is maximum number of elements. The ith element of this array represents the tree node that contains element 'i'. This array element ~~represents~~ gives the ~~tree node that~~ parent pointer of the corresponding tree node.

→ Notice that root nodes ~~are~~ have a parent of '-1'.

Ex: i = 2

P[i] = P[2] = 5.

* Simple union algorithm:

→ In this algorithm, root node of one tree becomes child node for another tree

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

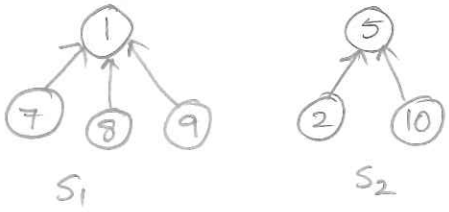
www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Simpleunion algorithm

```
Algorithm Simpleunion (i, j)
{
 P[i]=j;
}
```

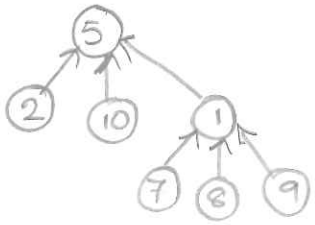
where i, j are root nodes of different trees.

Ex:



Here i=1, j=5.

when we perform Simple union (1, 5) on the trees S_1 and S_2. Root node of S_1 tree becomes child node for root node of S_2 tree.



i	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]
P	5	5			-1		5	5	5	5

→ Now Let us process the following sequence of union operations

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

union (1,2),

union (2,3), union (3,4) ..... union (n-1,n). This sequence of union operations produces the following tree called as degenerate tree

→ The n-1 union operations can be processed in time O(n)

* Find operation:

Find (i) determines the root node of the tree containing the element 'i'

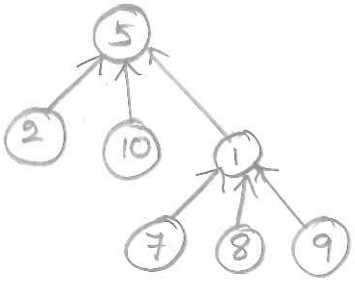
Simple find algorithm:

```
Algorithm SimpleFind(i)
{
  while (p[i]≥0) do
     i:= p[i];
  return i;
}
```

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Ex:



Find (7) = 5

Find (10) = 5

Find (5) = -1

→ The total amount of time required to process the 'n' finds is $O(n^2)$. For 'i find' operation at level i time complexity is $O(i)$.

*Note:

→ we can improve the performance of union and find algorithm by avoiding the creation of degenerate trees.

→ To perform this we make use of weighting rule for union (i,j)

*Weighting rule for union (i,j):

→ If the no. of the nodes in the tree with root 'i' is less than the no. of nodes in the tree with root 'j' then make 'j' the parent of 'i' otherwise make 'i' the parent of 'j'.

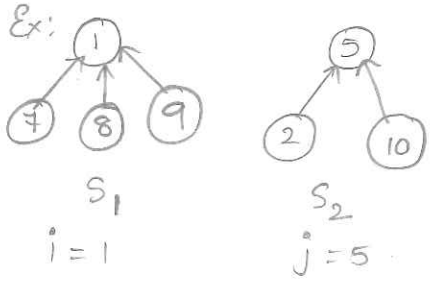
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Algorithm weighted union (i, j)

```
// P[i] := -count[i] and P[j] = -count[j]
{
temp := P[i] + P[j];
if (P[i] > P[j]) then
 {
  P[i] := j;
  P[j] := temp;
 }
else
 {
  P[j] := i;
  P[i] := temp;
 }
}
```

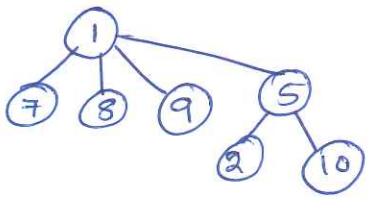
Ex:



P[i] = count[i], P[j] = - count[j]

↳ -count[i] means number of nodes which returns the tree 'i'

P[i] = -4 , P[j] = -3



www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ The number of nodes with root 'i' is 4

→ The no. of nodes in tree with root 'j' is 3 So make 'i' as a parent of 'j'.

* Collapsing rule:

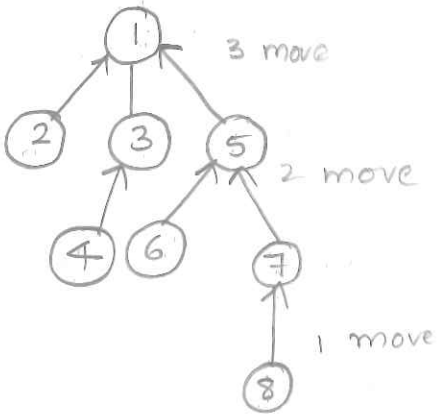
→ If 'j' is a node on the path from 'i' to its root and P[i] ≠ root [i] then set. P[j] to root[i].

* description:

→ Process the following 8 find operations.

Find(8), find(8). ... 8 times.

→ If simplefind algorithm is used each find(8) requires 3 moves in the following tree.



find(8) = 1.

→ 24 moves required to process all 8 find operations

→ when collapsing find algorithm is used the first find(8) requires going up 3 links and then

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

resetting 3 links.

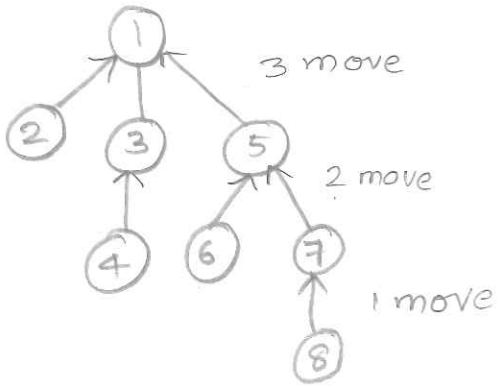
→ Each of the remaining 7 find operations requires going up only 1 link field.

→ The total cost is $6+1+1+1+1+1+1+1 = 13$

Algorithm:

```
Algorithm collapsingfind (i)
{
 r := i;
while (p[r] > 0) do
  r = P[r];
while (i ≠ r)
{
 s := P[i];
 P[i] := r;
  i := s;
}
return r;
}
```

Ex:



www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Before execution

i	1	2	3	4	5	6	7	8
P	-1	1	1	3	1	5	5	7

After execution

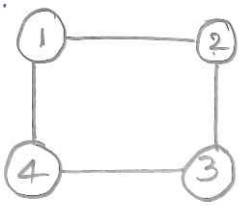
i	1	2	3	4	5	6	7	8
P	-1	1	1	3	1	5	1	1

total cost = 6 + 1 + 1 + 1 + 1 + 1 + 1 + 1 = 13

* Spanning tree:

Spanning tree is subgraph of given graph $G = \{V, E\}$ and contains all vertices of graph 'G' and no cycles

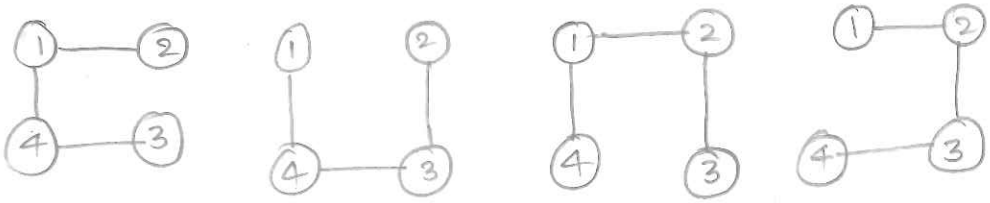
Ex: 1.



$G = \{V, E\}$

$V = \{1, 2, 3, 4\}$

$E = \{(1,2), (2,3), (3,4), (4,1)\}$



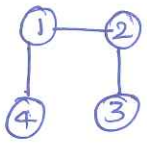
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Note:

→ If a spanning tree contains n vertices then it contains n-1 edges.

Ex:-



n = 4 vertices

n-1 = 3 edges

* Graph traversal techniques:

→ Graph traversal is a process of visiting all vertices of the graph.

→ There are two graph traversal techniques. They are

1. Breadth first search (BFS)
2. Depth first search (DFS).

1. Breadth first search (BFS):

→ In BFS we start at vertex 'V' and mark it as visited vertex.

→ The vertex 'V' is at this time is said to be unexplored.

→ A vertex is said to have been explored by an BFS algorithm when algorithm has visited all vertices adjacent from it.

www.android.previousquestionpapers.com | www.universityupdates.com | https://telegram.me/jntuh

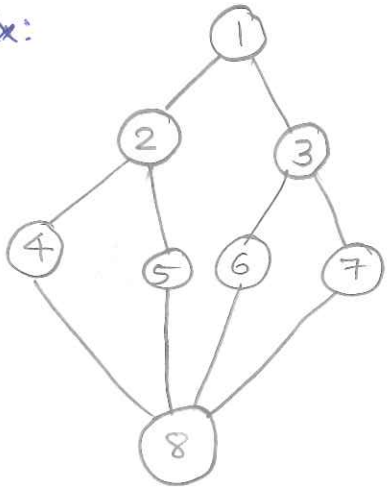
www.androiduniversityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ All unvisited vertices adjacent from 'v' are visited next. These are new unexplored vertices.

→ Vertex 'v' has been explored. The newly visited vertices have not been explored and are put into the queue.

→ The first vertex in the queue is the next vertex to be explored. Exploration continues until no unexplored ~~conditi~~ vertex is left.

Ex:

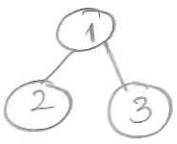


1. Visit the first vertex v=1. Mark this vertex as visited vertex.

(1)

2. This vertex is said to be unexplored.

2. Explore this vertex '1' by finding adjacent vertices (2 & 3) and visit.



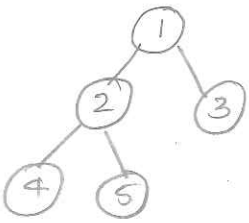
2	3

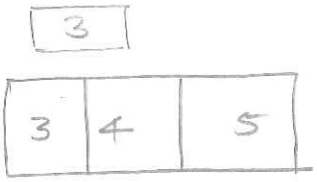
put adjacent vertices into the queue.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

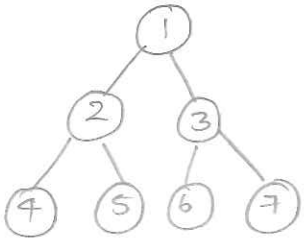
3. First vertex in the queue is 2 which is to be explored now explore this vertex by finding adjacent vertices (1,4,5) and visit. vertex '1' is



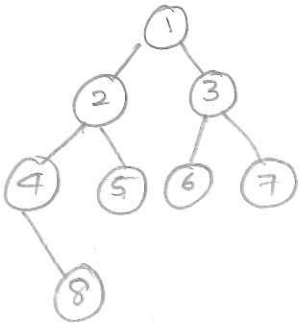


already visited. So insert vertices 4 & 5 into the queue.

4. Next first vertex in the queue is 3 which is to explored. Now, explore this vertex by finding adjacent vertices of 3 is 6 and 7.



5. Next first vertex in the queue is 4 which is to explored. Now find adjacent vertices of 4 is 8.



www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

6. Next vertex in the queue is 5 to be explored. But, all adjacent vertices of vertex '5' has already processed. So no exploration is required.

6	7	8

7. Next vertex in the queue is 6 to be explored. But all adjacent vertices of vertex '6' has already processed. So no exploration is required.

7	8

8. Next vertex in the queue is 7 to be explored. But all adjacent vertices of vertex '7' has already processed. So no exploration is required

8

9. Next vertex in the queue is 8 to be explored But all adjacent vertices of vertex '8' has already processed. So no exploration is required.

Connected graph:

→ A graph 'G' is said to be connected graph if their exists a path between every pair of vertices.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

An undirected graph is said to be connected if and only if for every pair distinct vertices.(u,v) in graph G.

~~There if if is~~

*BFS algorithm:

```
Algorithm BFS(v)
// A breadth first search of G is carried out beginning
// At vertex V, For any node i, visited [i] =1 if i has
// already been visited. The graph G and array visited[ ]
// are global ; visited[ ] is initialized to zero.
{
  u:=v; // q is a queue of unexplored vertices
  visited [v]:=1;
  repeat
  {
   for all vertices w adjacent from u do
    {
    if (visited [w] =0) then
    {
     Add w to q; // w is unexplored
      visited [w]:=1;
    }
   }
  if q is empty then return; // no unexplored vertex.
  delete u from q; // Get first unexplored vertex.
 }until (false);
}
```

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://t.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

2. Depth first search (DFS): Time complexity is $\theta(|V|+|E|)$

→ DFS is a graph traversal technique. DFS is an algorithm for traversing all vertices of the given graph or searching spanning tree one starts at the root (selecting some arbitary node as its root in the case of graph) and explores as far as possible along each branch before backtracking.

→ It employees the following rules

rule 1: Visit the adjacent unvisited vertex. Mark it as visited. display it. Push it in a stack.

rule 2: If no adjacent vertex is found, pop up a vertex from a stack. (It will pop up all the vertices from the stack, which do not have adjacent vertices)

rule 3: Repeat rule 1 and rule 2 until the stack is empty.

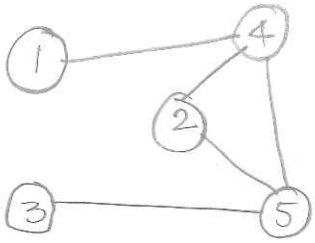
Algorithm for DFS:

```
Algorithm DFS(w)
{
  visited[v] := 1;
  for each vertex w adjacent from v do
  {
   if (visited[w]=0) then
    DFS(w);
  }
}
```

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Ex: Traverse a graph shown below using DFS start from a vertex with number 1.



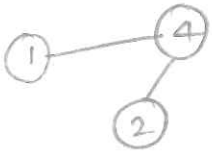
$2T(n-1)+1$

1. Mark a vertex 1 as visited

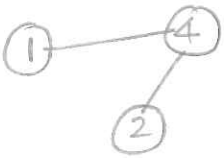
2. There is an edge (1,4) and vertex 4 is unvisited go there.

3. Mark the vertex 4 as visited

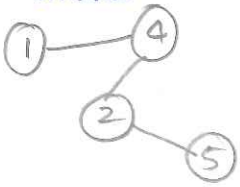
4. There is an edge (4,2) and vertex 2 is unvisited go there



5. Mark the vertex 2 as visited.



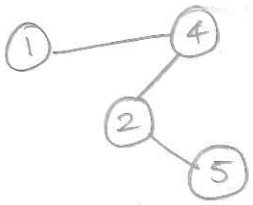
6. There is an edge (2,5) and vertex 5 is unvisited go there



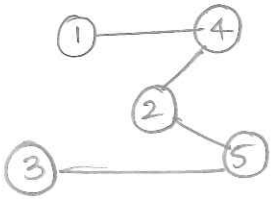
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

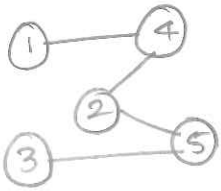
7. Mark the vertex 5 as visited



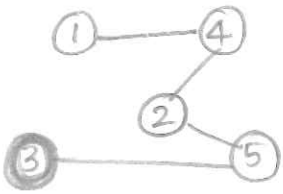
8. There is an edge (5,3) and a vertex 3 is unvisited go there



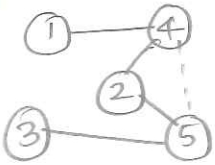
9. Mark the vertex 3 as visited.



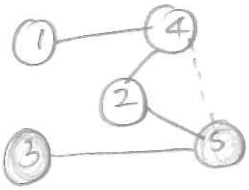
10. There are no ways to go from vertex 3. Mark it as black and backtrack to vertex 5.



11. There is an edge (5,4) but the vertex 4 is already visited



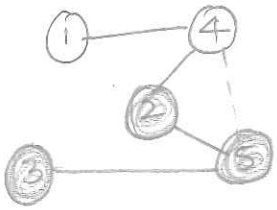
12. There are no ways to go from the vertex 5. Mark it as black and backtrack to vertex 2



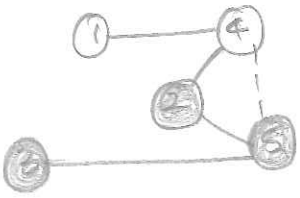
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

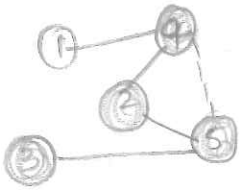
13. There are no more edges, adjacent to vertex 2. Mark it as black and backtrack to the vertex 4.



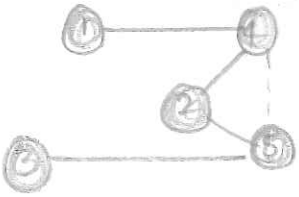
14. There is an edge (4,5) but the vertex 5 is black



15. There are no more edges, adjacent to the vertex 4. Mark it as black and backtrack to vertex '1'.



16. There are no more edges, adjacent to the vertex 1. Mark it as black. DFS is over.



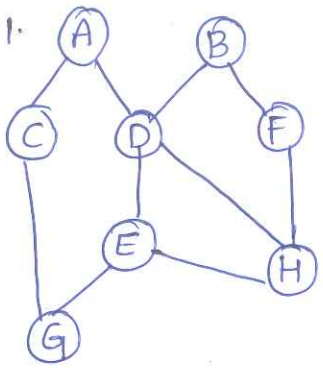
* Connected graph:

A graph is said to be connected iff there exists a simple path b/w every pair of vertices of given graph

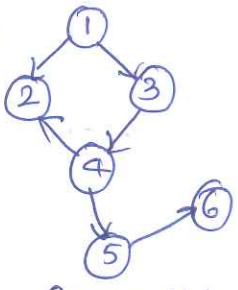
www.android.previousquestionpapers.com | www.universityupdates.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

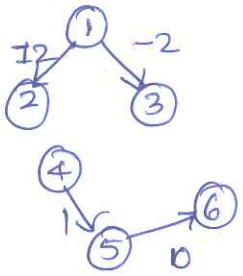
Ex: 1.



2.



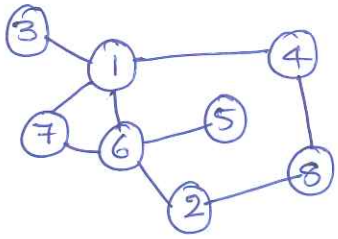
Connected

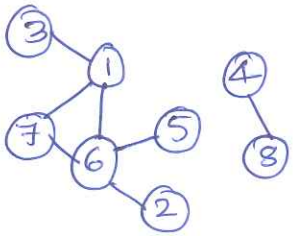


disconnected.

*Connected component:

Connected component of an undirected graph is a maximal connected subgraph.





A graph that is connected and a graph that consists of two connected components.

* Strongly connected:

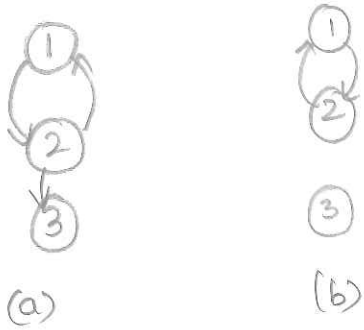
→ A directed graph G is said to be strongly connected iff for every pair of distinct vertices u and v in G there is a directed path from u to v and also from u to v.

→ A strongly connected component is a maximal subgraph that is strongly connected. The below figure (a) is not

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

strongly connected as there is no path from vertex from 3 to 2.



(a) (b)

A graph and its strongly connected components.

* Biconnected graph:

A graph is said to be biconnected if

1. It is connected i.e it is possible to reach every vertex from every other vertex, by a simple path.
2. Even after removing any vertex the graph remains connected.

→ A graph with no articulation point is called biconnected. In other words, a graph is biconnected if and only if any vertex is deleted, the graph remains connected.

* Articulation point:

→ An articulation point in a connected graph is a vertex that, if delete, would break the graph into two or more pieces (connected component).

→ An articulation point is a vertex of G such that the deletion of v, together with all edges incident on v, produces a graph G_1, that has atleast two

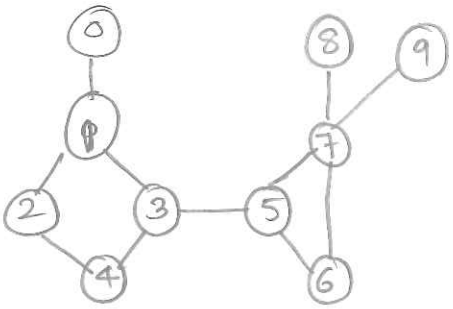
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

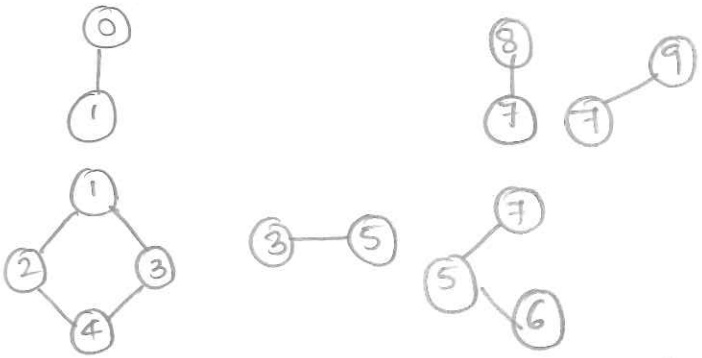
connected components. for ex, the connected graph of below figure has four articulation points, vertices 1,3,5,7.

*Biconnected component:

A biconnected component of graph is maximal biconnected subgraph.



(a) Connected graph



(b) Biconnected components.

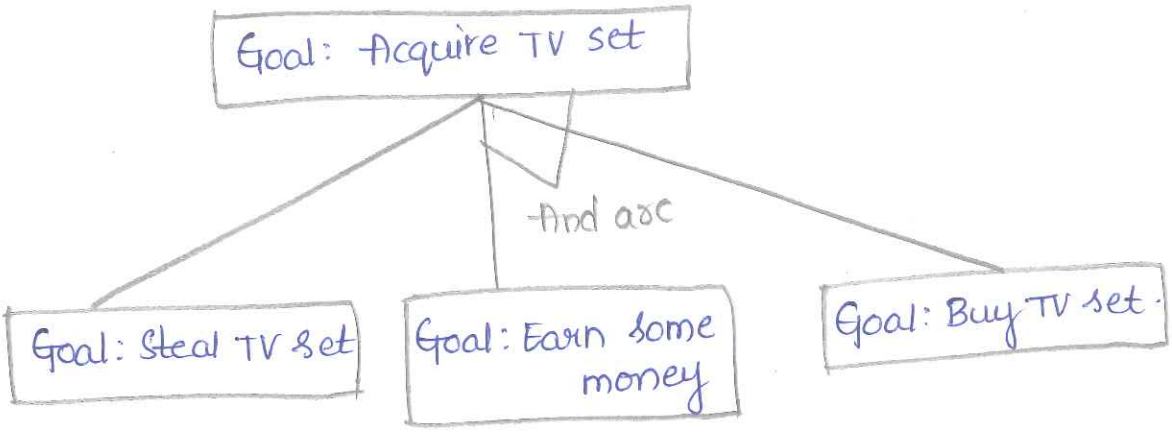
*AND-OR Graph:

→ when a problem can be divided into a set of sub problems, where each sub problem can be solved separately and a combination of these will be a solution, AND-OR graphs or AND-OR trees are used for representing the solution. The decomposition of the problem or problem reduction generates AND arcs. One AND are may point to any no of successor nodes. All these must be solved so that the arc will rise to many arcs, indicating several possible solutions

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Hence the graph is known as AND-OR instead of AND. figure shows an AND-OR graph.



Ex: AND-OR graph

www.android.previousquestionpapers.com | www.universityupdates.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

25/1/17

UNIT-III

GREEDY METHOD

*Greedy method:

*Comparison of divide and conquer and greedy method:

→ Divide and conquer is applicable for the problems which can be decomposible into sub problems. where as greedy method is applicable for the problems which can't be decomposible.

→ All of these problems have 'n' inputs and require is to obtain a subset that satisfies some constraints. Any subset that satisfies these constraints is called as a feasible solution.

→ We need to find a feasible solution that either maximize and minimize objective function.

→ A feasible condition or solution that does this called an optimal solution

Ex: Function $Z = 4x+3y$

Subject to the constraints $-1 \le x \le 1$

$-1 \le y \le 1$

www.android.universityupdates.com | www.universityquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Find maximum value of z (objective function)

input set (x, y)	$-1 \le x \le 1$	$-1 \le y \le 1$
	$-1, 0, 1$	$-1, 0, 1$

$(-1, -1)$ $z = 4x-1 + 3x-1 = -7$

$(-1, 0)$ $z = 4x-1 + 3x0 = -4$

$(-1, 1)$ $z = 4x-1 + 3x1 = -1$

$(0, -1)$ $z = 4x0 + 3x-1 = -3$

$(0, 0)$ $z = 4x0 + 3x0 = 0$

$(0, 1)$ $z = 4x0 + 3x1 = 3$

$(1, -1)$ $z = 4x1 + 3x-1 = 1$

$(1, 0)$ $z = 4x1 + 3x0 = 4$

$(1, 1)$ $z = 4x1 + 3x1 = 7$

Feasible solution

Optimal solution

→ Optimal solution is $z = 7$ for the input values $(x, y) = (1, 1)$.

→ In greedy method, an algorithm works in stages, considering one input at a time.

→ At each stage, a decision is made regarding whether a particular ~~line~~ input is ~~in~~ an optimal solution.

→ This can be done by considering inputs in an order determined by some selection procedure.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

*Control abstraction / General method of greedy method:

```
Algorithm Greedy(a,n)
// a[1...n] contains n inputs
{
 Solution = 0;
 for := 1 to n do
  x := Select(a);
  if Feasible(Solution, x) then
     Solution := Union(Solution, x);
 }
 return Solution;
 }
```

→ In this control abstraction, the function select() selects an input from array a[] and removes it. The selected input value is assigned to x.

→ Feasible is a boolean valued function that determines whether input value x satisfying the given constraint or not. If the input 'x' is not satisfying the given constraint then eliminate from the input set. If the input 'x' is satisfying the given constraint then find feasible solution.

→ The function union combines 'x' with the solution and updates the objective function.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* Applications:

* 0/1 - Knapsack problem:

Ex: $n = 3$

$m = 20$

$(P_1, P_2, P_3) = (25, 24, 15)$

$(w_1, w_2, w_3) = (18, 15, 10)$

→ In knapsack problem we have 'n' items and a knapsack (empty bag). Each item has a weight (w_i) and profit (P_i) and knapsack has a capacity of 'm' units.

→ The objective of the problem is to obtain a filling of the knapsack that maximize the total profit.

→ ~~Beside~~ Decide which item are to be placed in the bag so that we get maximum profit.

→ Generally, the problem can be stated as

maximize profit $\sum_{1 \le i \le n} P_i x_i$

Subject to constraint $\sum_{1 \le i \le n} w_i x_i$

and $0 \le x_i \le 1$, $1 \le i \le n$.

→ Consider the following instance of knapsack

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

problem. $n=3, m=20, (P_1, P_2, P_3) = (25, 24, 15)$,

$(w_1, w_2, w_3) = (18, 15, 10)$. Find optimal solution.

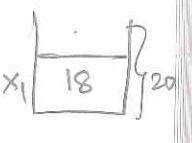
Sol:- Case(i): Maximum profit

Select the item which has maximum profit and place it in the bag.

$x_1 = 18/18 = 1$

$x_2 = 2/15 = 2/15$

$x_3 = 0 = 0$



$$\sum_{1 \le i \le n} P_i x_i = P_1 x_1 + P_2 x_2 + P_3 x_3$$

$= 25 \times 1 + 24 \times 2/15 + 15 \times 0$

$= 25 + 3.2 + 0$

profit = 28.2.

→ Now a bag consists of two units of space and so second item can be placed partially.

$x_2 = 2/15 = 2/15$ $[(x_1, x_2, x_3) = (1, 2/15, 0)]$

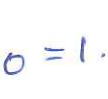
→ Now the bag is full so x_3 item cannot be placed in the bag

→ Profit = 28.2

Case(ii): Minimum ~~profit~~ weight

→ Place the items which has minimum weight into the bag. First place item x_3 into the bag completely.

$x_3 = 10/10 = 1$.

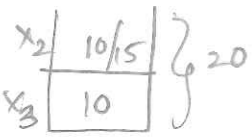


→ Now bag consists of 10 units of space so

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

item x_2 is placed partially



$x_2 = 10/15 = 2/3$

→ Now bag is full, item x_1 cannot be placed in bag.

x_1 = Not placed = 0 $\quad (x_1, x_2, x_3) = (0, 2/3, 1)$

$$\sum P_i x_i = P_1x_1 + P_2x_2 + P_3x_3$$

$$= 25 \times 0 + 24 \times 2/3 + 15 \times 1$$

profit = 0 + 16 + 15 = 31

<u>case(iii)</u>: Maximum profit per unit weight.

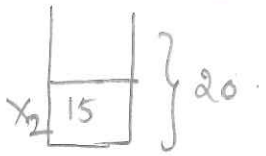
$$x_1 = \frac{P_1}{w_1} = \frac{25}{18} = 1.4$$

$$x_2 = \frac{P_2}{w_2} = \frac{24}{15} = 1.6$$

$$x_3 = \frac{P_3}{w_3} = \frac{15}{10} = 1.5$$

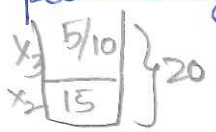
→ place item x_2 in the bag because it has maximum profit per unit weight.

$$x_2 = \frac{15}{15} = 1$$



→ Now bag consists of 5 units of space so item x_3 is placed partially

$$x_3 = 5/10 = 1/2$$



www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ Now bag is full, item x_1 cannot be placed in bag.

$x_1 = 0$.

$(x_1, x_2, x_3) = (0, 1, 1/2)$

$\Sigma P_i x_i = P_1x_1 + P_2x_2 + P_3x_3$

$= 25\times0 + 24\times1 + 15\times1/2$

$= 0 + 24 + 7.5$

profit $= 31.5$

∴ Optimal solution is $31.5 = \Sigma P_i x_i$ for the input values of $(x_1, x_2, x_3) = (0, 1, 1/2)$

2) construct the following instance of 0/1 knapsack problem $n=4$, $m=15$

$(P_1, P_2, P_3, P_4) = (18, 16, 4, 9)$

$(w_1, w_2, w_3, w_4) = (6, 4, 5, 10)$. Find maximum profit.

Sol:- case(i): Maximum profit.

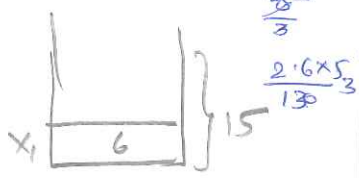
~~$x_1 = 10/10 = 1$~~ → place an item which has maximum profit in the bag (knapsack)

~~$x_2 = 5/6$~~

~~$x_3 = 0$~~

~~$x_4 = 0$~~ → place first item x_1

$x_1 = 6/6 = 1$



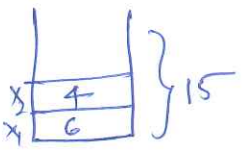
~~$\Sigma P_i x_i = 18\times1 + 16\times5/6 + 0 + 0$~~

~~$= 18 + 13 = 31$~~

x_1 is placed completely.

→ place x_2 item $x_2 = 4/4 = 1$

x_2 is placed completely.



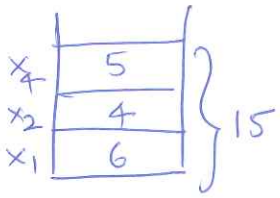
www.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

x_2 is placed completely

→ Now bag consists of 5 units of space x_4 item is placed partially.

$x_4 = 5/10 = 1/2$



→ Now knapsack is full so x_3 item cannot be inserted.

$$\text{Profit} = \sum_{i \le 1} P_i x_i = P_1x_1 + P_2x_2 + P_3x_3 + P_4x_4$$

$$= 18 \times 1 + 16 \times 1 + 4 \times 0 + 9 \times 1/2$$

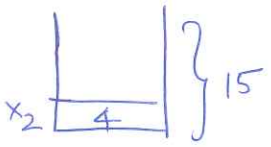
$$= 38.5$$

case(ii): Minimum weight.

→ place an item which has minimum weight into bag.

→ place x_2 item

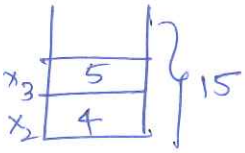
$x_2 = \frac{4}{4} = 1$



x_2 is placed completely

→ place x_3 item

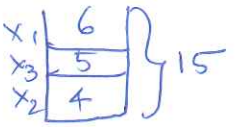
$x_3 = \frac{5}{5} = 1$



x_3 is placed completely.

→ Now bag consits of 6 units of space x_1 item is placed completely

$x_1 = \frac{6}{6} = 1$



www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ Now bag is full so x_4 item cannot be inserted

$$\text{profit} = \sum_{i \le 1} p_i x_i = 18 \times 1 + 16 \times 1 + 4 \times 1 + 9 \times 0$$
$$= 38.$$

Case(iii): Maximum profit per unit weight.

→ place an item which has maximum profit per unit weight.

→ ~~place~~ $x_1 = \frac{P_1}{\omega_1} = \frac{18}{6} = 3$

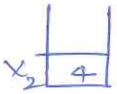
$$x_2 = \frac{P_2}{\omega_2} = \frac{16}{4} = 4$$

$$x_3 = \frac{P_3}{\omega_3} = \frac{4}{5} = 0.8$$

$$x_4 = \frac{P_4}{\omega_4} = \frac{9}{10} = 0.9$$

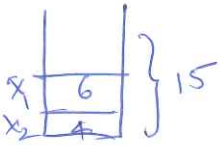
→ place an item x_2. x_2 is placed completely.

$$x_2 = \frac{4}{4} = 1$$



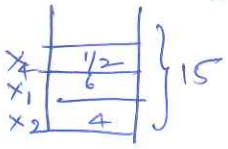
→ place an item x_1. x_1 is placed completely

$$x_1 = \frac{6}{6} = 1$$



→ place an x_4 item is placed partially completely

$$x_4 = \frac{5}{10} = 1/2$$



→ Now bag is full so x_3 cannot be inserted.

$$x_3 = 0.$$

$$\sum p_i x_i = 18 + 16 + 0 + 4.5$$
$$= 38.5$$

www.android.previousquestionpapers.com | www.universityupdates.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Optimal solution = 38.5 for input values

$(x_1\ x_2\ x_3\ x_4) = (1, 1, 0, 1/2) = 38.5$.

Feasible solution

S/No	Input $(x_1\ x_2\ x_3\ x_4)$	feasible solution
1	$(1, 1, 0, 1/2)$	38.5
2	$(1, 1, 1, 0)$	38
3	$(1, 1, 0, 1/2)$	38.5

→ optimal solution

* Pseudo code for 0/1 knapsack problem:

```
Greedy-knapsack (M,n)
// w[1:n] - Array of weights
// p[1:n] - Array of profits
// M - capacity of knapsack
// x[1:n] is solution
{
for i:=1 to n do
 x[i]=0
 cu:=m;
for i:=1 to n do
if (w[i] >cu) then
break;
x[i]:=1;
cu= cu - w[i];
}
```

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

```
if(i≤n) then
x[i] = cu/w[i];
}
```

→ Time complexity for knapsack problem is O(n).

* Job sequencing with deadlines:

→ Job sequencing deals with a set of 'n' jobs, associated with each job 'i' is an integer deadline $d_i \geq 0$ and a profit $P_i \geq 0$. For any job 'i' the profit 'P_i' is earned if it's job is completed by its deadline. To complete a job one has to process the job on a machine for one unit of time. Only one machine is available for processing jobs. A feasible solution for this problem is a subset 'J' of jobs such that each job in this subset can be completed by its deadline.

→ The value of a feasible solution 'J' is the sum of profits of the job in 'J'. An optimal solution is the feasible solution with maximum value.

Ex: Let N=4, $(P_1, P_2, P_3, P_4) = (100, 10, 15, 27)$ and $(d_1, d_2, d_3, d_4) = (2, 1, 2, 1)$. Solve the problem of job sequencing with deadlines using greedy method.

Sol:- Since the maximum deadline is 2 units, maximum of two jobs will form the feasible solution.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/intuh

S.No	feasible solution	processing sequence	profit	
1.	{1}	(1)	100	
2.	{1,2}	(2,1)	100+10=110	
3.	{1,3}	(1,3),(3,1)	100+15=115	
4.	{1,4}	(4,1)	100+27=127	optimal solution
5.	{2,3}	(2,3)	10+15=25	
6.	{2,4}	(2,4),(4,2)	10+27=37	
7.	{3,4}	(4,3)	15+27=42	
8.	{2}	(2)	10	
9.	{3}	(3)	15	
10.	{4}	(4)	27	

→ Serial no - 4 or solution 4 is optimal solution. In this solution only jobs 1 and 4 are processed and the profit is 127. These jobs must be processed in the order, job 4 followed by job 1. Thus, the processing of job 4 begins at time zero and that of job 1 is completed at time two.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/intuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Ex: 2) $N = 5$, $(P_1, P_2, P_3, P_4, P_5) = (20, 15, 10, 5, 1)$,

$(d_1, d_2, d_3, d_4, d_5) = (2, 2, 1, 3, 3)$.

Sol:-

S.No	feasible solution	processing sequence	profit	
1.	{1}	(1)	20	
2.	{1,2}	(1,2),(2,1)	35	
3.	{1,3}	(3,1)	30	
4	{1,4}	(1,4)	25	
5	{1,5}	(1,5)	21	
6	{2,3}	(3,2)	25	
7.	{2,4}	(2,4)	20	
8.	{2,5}	(2,5)	16	
9.	{3,4}	(3,4)	15	
10	{3,5}	(3,5)	11	
11.	{4,5}	(4,5)(5,4)	6	
12	{2}	(2)	15	
13	{3}	(3)	10	
14	{4}	(4)	5	
15	{5}	(5)	1	
16	{1,2,3}	(3,2,1)	45	optimal solution
17	{1,2,4}	(1,2,4)	40	

www.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

18	{1,2,5}	(1,2,5)	36
19	{1,3,4}	(3,1,4)	35
20	{1,3,5}	(3,1,5)	31
21	{1,4,5}	(1,4,5)	26
22	{2,3,4}	(3,2,4)	30
23	{2,3,5}	(3,2,5)	26
24	{2,4,5}	(2,4,5)	21
25	{3,4,5}	(5,4,3)	16.

3) $N = 4$, $(P_1, P_2, P_3, P_4) = (3, 5, 20, 18)$, $(d_1, d_2, d_3, d_4) = (1, 3, 2, 3)$

Sol:- According to greedy method, arrange the jobs in decreasing order of their profits

$(P_3, P_4, P_2, P_1) = (20, 18, 5, 3)$

$(d_3, d_4, d_2, d_1) = (2, 3, 3, 1)$

S.No	feasible solution	processing sequence	profit	
1.	{3}	(3)	20	
2.	{3,4}	(3,4)	38	
3.	{3,4,2}	(3,4,2),(3,2,4)	43	maximum profit is optimal solution
4.	{3,4,1}	(1,3,4)	41	
5.	{4,2,1}			

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

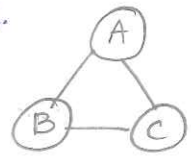
www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* Spanning tree:

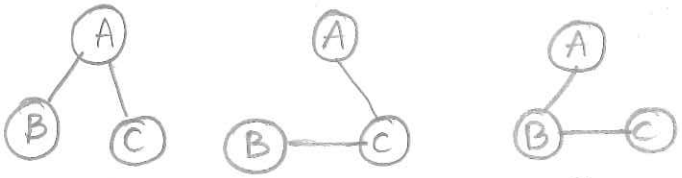
Spanning tree is a sub graph of a given graph G. It satisfies the following properties.

1. It contains all vertices of a graph.
2. It does not contains cycles or loops.
3. It contains (n-1) edges where n is number of vertices.

Ex: 1.



Spanning trees of given graph



(a) (b) (c)

→ we ~~formed~~ found three spanning trees of given complete graph. A complete undirected graph can have maximum "n^{n-2}" number of spanning trees where n is number of vertices.

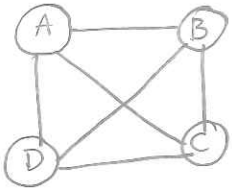
→ In above example,

$n^{n-2} = 3^{3-2} = 3$ ~~are~~ spanning trees are possible.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

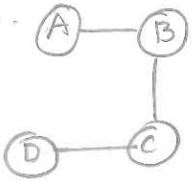
Ex: 2.

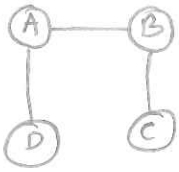


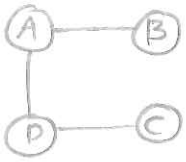
$n = 4$

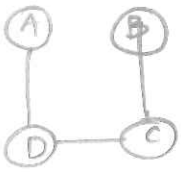
$n^{n-2} = 4^{4-2} = 4^2 = 16$

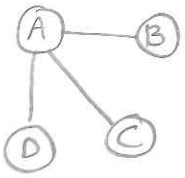
Maximum no. of spanning trees are 16.

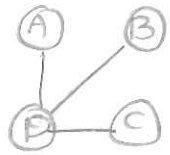
1. 

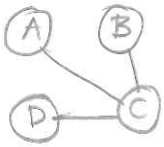
2. 

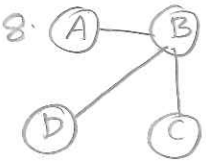
3. 

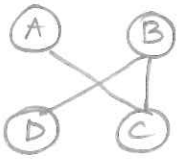
4. 

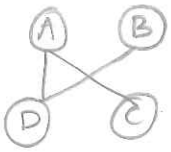
5. 

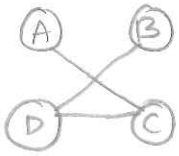
6. 

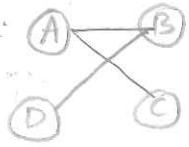
7. 

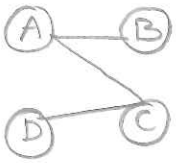
8. 

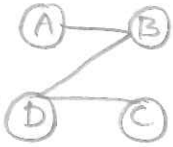
9. 

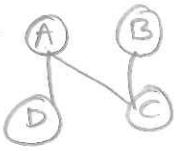
10. 

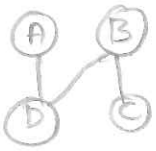
11. 

12. 

13. 

14. 

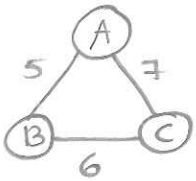
15. 

16. 

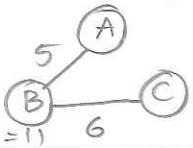
* Minimum cost spanning tree:

→ Spanning tree of graph with minimum cost is known as minimum cost spanning tree.

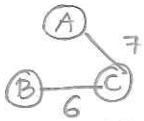
Ex:



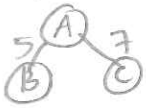
Spanning trees of the given graph



cost = 5+6 = 11



cost = 6+7 = 13



cost = 5+7 = 12

www.android.previousquestionpapers.com | www.universityupdates.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

minimum cost spanning tree is 5+6 = 11

* Minimum cost spanning tree applications:

→ Minimum cost spanning tree is used in design of networks. including computer networks, telecommunication networks, transportation networks, water supply network and electrical grides.

→ Constructing trees for broadcasting in computer networks.

→ Handwriting recognition of mathematical expression.

→ Image registration and segmentation.

→ Minimum cost spanning tree can be used to solve the travelling salesperson problem.

→ There are two algorithms to find minimum cost spanning tree.

1. Prims algorithm
2. krushkal's algorithm.

1. Prims algorithm:

→ The prims algorithm starts with a tree that includes only a minimum cost edge of graph 'G'.

→ Then edges are added to this tree one by one.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ Add an edge (i,j) with minimum cost to the tree.

→ The next edge (i,j) to be added in such a way that 'i' is a vertex already included in tree, 'j' is a vertex not yet included and the cost of (i,j) is minimum among all edges (k,l) such that vertex 'k' is in the tree and vertex 'l' is not in the tree.

→ To determine this edge (i,j), we associate with each vertex 'j' not yet included in the tree. A value near[j] is a vertex in the tree such that cost[j, near[j]] is minimum among all choices for near[j].

→ We define near[j] = 0 for all vertices 'j' that are already in the tree.

→ The next edge to include is defined by vertex 'j' such that near[j] ≠ 0 and cost[j, near[j]] is minimum.

* Prims algorithm

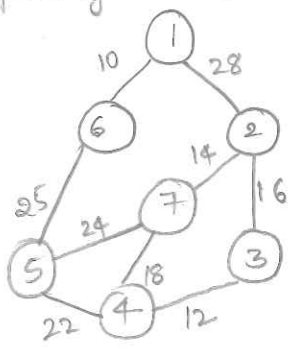
```
Algorithm prim (E, cost, n, t)
// E is set of edges in G
// cost [1:n, 1:n] is the cost adjacency matrix
//  ∞ - if no edge (i,j) exists
// t [1:n; 1:2] - min cost spanning tree
```

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

```
{
mincost = 0;
for i = 2 to n do
  near[i] := 1;
near[1] = 0;
for i = 1 to n-1 do
{
let j be an index such that near[j] ≠ 0 and
cost[j, near[j]) is minimum;
t[i,1] = j;
t[i,2] = near[j];
mincost = mincost + cost[j, near[j]);
near[j] = 0;
for k := 1 to n do
if ((near[k] ≠ 0) and (cost[k, near[k]] > cost[k,j]))
then
near[k] = j;
}
return mincost;
}
```

Spanning tree ex:



www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

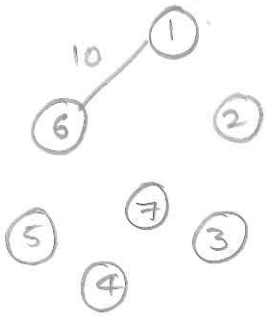
www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Cost Adjacent matrix

	1	2	3	4	5	6	7
1	∞	28	∞	∞	∞	10	∞
2	28	∞	16	∞	∞	∞	14
3	∞	16	∞	12	∞	∞	∞
4	∞	∞	12	∞	22	∞	18
5	∞	∞	∞	22	∞	25	24
6	10	∞	∞	∞	25	∞	∞
7	∞	14	∞	18	24	∞	∞

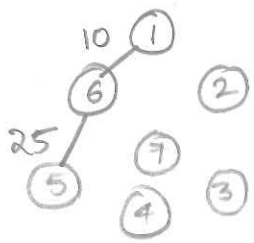
Step-1:

→ Start with a source vertex '1' find nearest vertex of '1' (i.e) vertex '6' because cost of (1,6) is 10 (i.e) minimum. Now add an edge 1,6 to the minimum cost spanning tree



Step-2:

→ Find out nearest vertex of vertices 1,6. Nearest vertex of 1 is 2 and cost is 28. Nearest vertex of 6 is 5 and cost is 25. Select vertex 5 and add to minimum cost spanning tree.

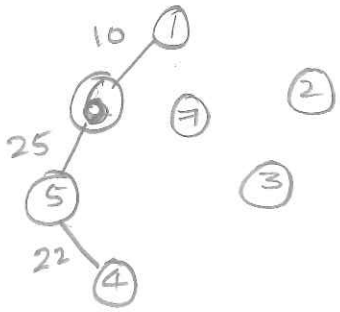


www.android.previousquestionpapers.com | www.universityupdates.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

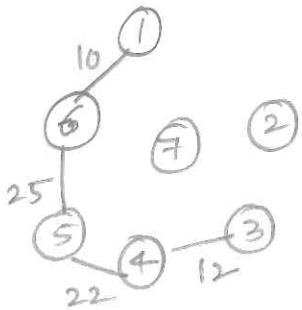
Step-3:

Find nearest vertex of vertices 1, 6, 5. Nearest vertex of 5 is 4 and cost is 22. Add this vertex to minimum cost spanning tree.



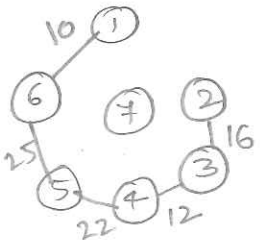
Step-4:

Find nearest vertex of vertices 1, 6, 5, 4. Nearest vertex of 4 is 3 and cost is 12. Add this vertex to minimum cost spanning tree.



Step-5:

Find nearest vertex of vertices 1, 6, 5, 4, 3. Nearest vertex of 3 is 2 and cost is 16. Add this vertex to minimum cost spanning tree.

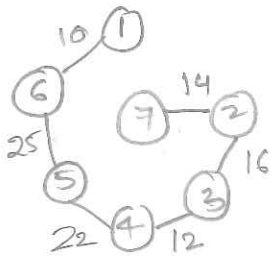


www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Step-6:

Find nearest vertex of vertices is 1, 6, 5, 4, 3, 2. Nearest vertex of 2 is 7 and cost is 14. Add this vertex to minimum cost spanning tree.



Minimum cost spanning tree is $10+25+22+12+16+14 = 99$

2. Krushkal's algorithm:

→ Kruskal's algorithm is another method to find minimum cost spanning tree. It follows the following steps to find minimum cost spanning tree

Step-1:

Arrange the edges in increasing order of cost.

Step-2:

Select an edge with minimum cost and add to the minimum cost spanning tree

Step-3:

If an edge forms a cycle then eliminate that edge from the spanning tree of minimum cost.

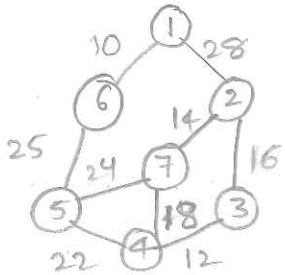
Step-4:

Repeat step 2 and step 3 until all vertices are placed in minimum cost spanning tree.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Ex:



Step-1:

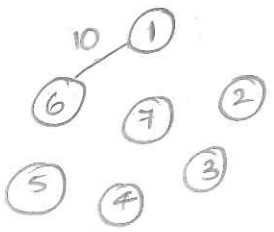
Arrange the edges in increasing order of cost

edges	cost
(1,6)	10
(3,4)	12
(2,7)	14
(2,3)	16
(4,7)	18
(4,5)	22
(5,7)	24
(5,6)	25
(1,2)	28

Step-2:

Select an edge with minimum cost (i.e) (1,6) edge.

add this edge to minimum cost spanning-tree

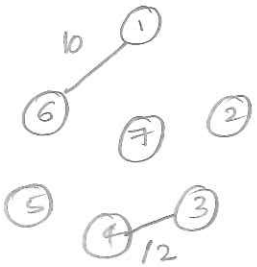


www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

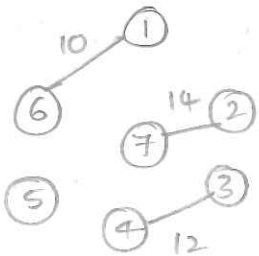
Step-3:

→ Select next minimum cost edge (i.e) (3,4) edge. and insert into MCSP



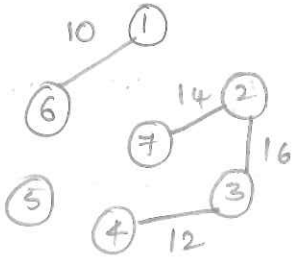
Step-4:

→ Select next minimum cost edge (2,7) add to MCSP



Step-5:

→ Select next minimum cost edge (2,3) add to MCSP



Step-6:

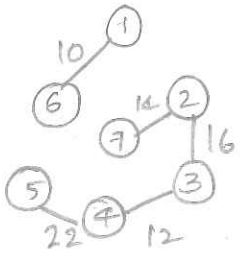
→ Select next minimum cost edge (4,7). This edge will form a cycle when it is placed in MC spanning tree so eliminate this edge.

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Step-7:

Select next minimum cost edge (4,5) add to MCST.

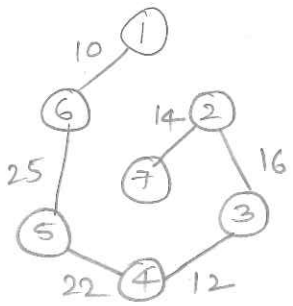


Step-8:

Select next minimum cost edge (5,7). This edge will form a cycle so it is eliminated.

Step-9:

Select next minimum cost edge (5,6) add to MCST.



Step-10:

Select next minimum cost edge (1,2). This edge will form a cycle so it is eliminated.

⇒ The cost of minimum spanning tree is

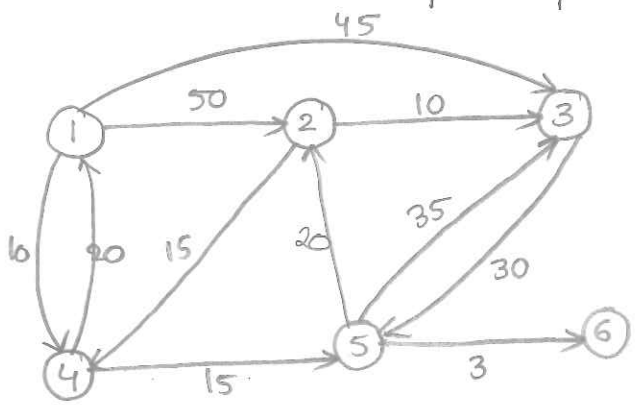
$10+14+16+12+22+25=99$.

DAA

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* Single source shortest paths problem:



→ Single source shortest paths problem is determined the shortest path from source vertex to all remaining vertices of graph 'G'.

S.No	Paths	Length	shortest path	length.
1.	1-2	50	1-4-5-2	45
	1-4-5-2	10+15+20=45		
	1-3-5-2	45+30+20=95		
2.	1-3	45	1-3	45
	1-2-3	60		
	1-4-5-3	60		
	1-4-5-2-3	55		
3.	1-4	10	1-4	10
	1-2-4	65		
	1-2-3-5-2-4	125		
4.	1-4-5	25	1-4-5	25
	1-2-4-5	80		
	1-2-3-5	90		
	1-3-5	75		
5.	1-4-5-6	28	1-4-5-6	28.
	1-2-3-5-6	93		
	1-3-5-6	78		
	1-2-4-5-6	83		

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ Shortest path from source vertex 1 to vertices 2, 3, 4, 5, 6.

Shortest path	Length
1-4-5-2	45
1-3	45
1-4	10
1-4-5	25
1-4-5-6	28

→ we can find solution to single source shortest path problem using "Dijkstra's algorithm".

```
Algorithm shortest paths (v, cost, dist, n)
//cost [1:n, 1:n] cost adjacency matrix
//dist [j], 1≤j≤n – length of shortest path from
// vertex v to vertex j.
{
for i=1 to n do
 {
  s[i]=0;
  dist[i] = cost [v,i];
 }
 s[v]=1;
 dist [i]: cost [v,i];
 for num = 2 to n-1 do
```

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

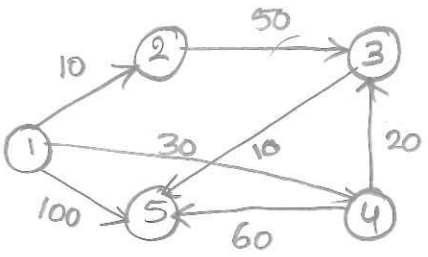
```
{
//determine n-1 paths from v
choose u from among those vertices not in
's' such that dist[u] is minimum.
for(each w adjacent to u with s[w]=0) do
if(dist[w]>dist[u]+cost([u,w])) then
dist[w]=dist[u]+cost[u,w];
}
}
```

Algorithm:

```
{
s={1};
for(i=2;i<=n;i++)
D[i]=c[1,i];
for(i=1; i<=n-1;i++)
{
choose a vertex w∈V-S such that
D[w] is minimum
S=S U{w};
for each vertex v∈V-S
} D[v]=min(D[v], D[w]+c[w,v])
}
```

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh



Initially:

$S = \{1\}$

$D[2] = C[1,2] = 10$

$D[3] = C[1,3] = \infty$

$D[4] = C[1,4] = 30$

$D[5] = C[1,5] = 100$

Iteration 1: $i = 1$

$w \in V - S$

$w \in \{1,2,3,4,5\} - \{1\}$

$w \in \{2,3,4,5\}$

choose $w = 2$ because $D[2] = 10$ is minimum

$S = S \cup \{w\}$

$= \{1\} \cup \{2\}$

$S = \{1,2\}$

for each vertex $v \in V - S$

$v \in \{1,2,3,4,5\} - \{1,2\}$

$v \in \{3,4,5\}$

$V = 3$

$D[v] = \min(D[v], D[w] + C[w,v])$

$D[3] = \min(D[3], D[2] + C[2,3])$

$= \min(\infty, 10 + 50)$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$= \min(\infty, 60)$

$D[3] = 60$.

V=4

$D[4] = \min(D[4], D[2] + c[2,4])$

$= \min(30, 10+\infty)$

$= 30$

$D[5] = \min(D[5], D[2] + c[2,5])$

V=5

$= \min(100, 10+\infty)$

$= 100$.

Iteration 2: i=2

$w \in V-S$

$w \in \{1,2,3,4,5\} - \{1,2\}$

$w \in \{3,4,5\}$

choose $w = 4$

$S = S \cup \{w\}$

$= \{1,2\} \cup \{4\} = \{1,2,4\}$

for each vertex $w \in V-S$

$w \in \{1,2,3,4,5\} - \{1,2,4\}$

$w \in \{3,5\}$

$D[3] = \min(D[3], D[4] + c[4,3])$

$= \min(60, 30+20)$

$= 50$

$D[5] = \min(D[5], D[4] + c[4,5])$

$= \min(100, 30+60)$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$= \min(100, 90)$

$= 90$.

Iteration 3: $i = 3$

$w \in V - S$

$w \in \{1,2,3,4,5\} - \{1,2,4\}$

$w \in \{3,5\}$

choose $w = 3$ because in iteration $D[3] = 50$ is min

$S = S \cup \{w\}$

$= \{1,2,4\} \cup \{3\}$

$S = \{1,2,4,3\}$

for each vertex $w \in V - S$

$w \in \{1,2,3,4,5\} - \{1,2,3,4\}$

$w \in \{5\}$

$D[5] = \min(D[5], D[3] + c[3,4])$

$= \min(90, 50+10)$

$= \min(90, 60)$

$= 60$

$\boxed{v=5}$

Iteration 4: $i = 4$:

Select $w = 5$ so that $S = \{1,2,3,4,5\}$

$D[2] = 10$

$D[3] = 50 \Rightarrow \boxed{i=2}$ min

$D[4] = 30$

$D[5] = 60$

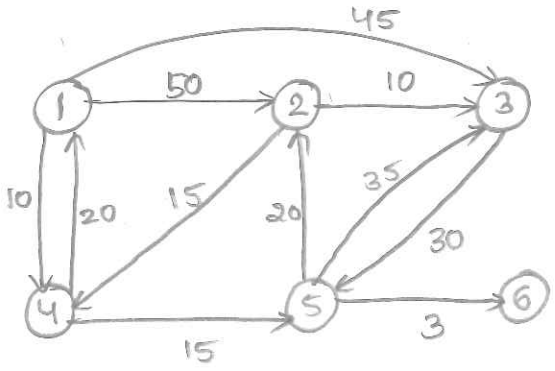
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

The time complexity of single source shortest path is $O(n^2)$.

* Alternate method:

i) Determine the shortest path from source vertex to remaining vertices.



Sol:-

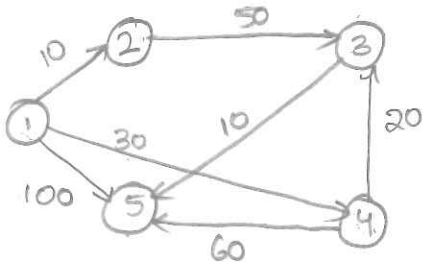
Iteration	S	Vertex Selected	Distance					
			1	2	3	4	5	6
Initial	{1}	-	∞	50	45	10	∞	∞
1	{1}	4	∞	50	45	10	25	∞
2	{1,4}	5	∞	45	45	10	25	28
3	{1,4,5}	6	∞	45	45	10	25	28
4	{1,4,5,6}	2	∞	45	45	10	25	28
5	{1,4,5,6,2}	3	∞	45	45	10	25	28

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Path	length
1-4	10
1-4-5	25
1-4-5-6	28
1-4-5-2	45
1-3	45

2) Determine shortest path:



Iteration	S	Vertex Selected	Distance				
			1	2	3	4	5
Initial	{1}	-	∞	10	∞	30	100
1	{1,}	2	∞	10	60	30	100
2	{1,2}	4	∞	10	50	30	90
3	{1,2,4}	3	∞	10	50	30	60
4	{1,2,4,3}	5	∞	10	50	30	60

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

path	length
1-2	10
1-4	30
1-2-3	50
1-5	60

* Dynamic programming:

→ Dynamic programming is an algorithm design method that can be used when there is a solution for problem can be viewed as the result of sequence of decisions.

Ex: Knapsack problem

→ The solution to the knapsack problem can be viewed as the result of sequence of decisions.

→ We have to decide the values of x_i, $1 \le i \le n$. First we make a decision on x_1, then on x_2, then on x_3..... So on. A optimal sequence of decisions maximizes the objective function $\sum P_i x_i$.

→ For some of the problems that may be viewed in this way. an optimal sequence of decisions can be formed by making the decisions one at a time and never making wrong decision. This is true for all problems that can be solved by greedy method

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

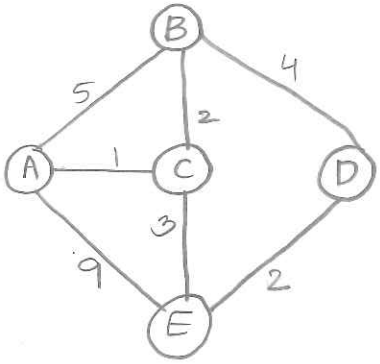
→ For many other problems, it is not possible to make step-wise decisions in such a manner that the sequence of decisions made is optimal.

**

* Principle of optimality:

→ It states that an optimal sequence of decision has the property that whatever the initial state and decision are, the remaining decision must contain an optimal decision sequence with regard to the state resulting from the first decision.

→ For example,



Find the shortest path from vertex A to vertex D

A - C - E - D = 1 + 3 + 2 = 6.

* Comparison between dynamic programming and greedy method:

Dynamic programming	Greedy method.
1. In this method, the solution to a problem can be viewed as result of sequence of	1. Greedy method is the most forward technique for constructing solution to

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

decisions	an optimal problem through a sequence of steps.
2. In this method more than one decision is made at a time.	2. Only one decision is made at a time.
3. Dynamic programming considers all possible sequences in order to obtain the optimal solution	3. Greedy method considers an optimal solution without revising the previous solution.
4. It is more expensive than greedy method.	4. Greedy method is comparitively less expensive.

* Comparison between dynamic programming and divide and conquer:

Dynamic programming	Divide and Conquer
→ It is a method in which a solution to a problem can be viewed as the result of sequence of decisions.	→ It is a method in which solution to a problem can be obtained by dividing it into several subproblems.
→ Dynamic programming algorithm solves every sub problem just once and saves the result in a table. From this, solution to the problem is obtained.	→ In divide and conquer, every subproblem is solved and the results are combined to get the original solution.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ Dynamic programming works best when all sub problems are dependent.	→ Divide and conquer works best when all sub problems are independent.

* Applications of dynamic programming:

1. Matrix chain multiplication
2. Optimal binary search trees.
3. 0/1 knapsack problem
4. All pairs shortest path problems
5. Travelling sales person problem
6. Reliability design problem.
7. Multi stage graph.

* Multi stage Graph:

→ A multi stage graph $G = (V, E)$ which is a directed graph. In this graph all the vertices are partitioned into K-stages where $K \geq 2$.

→ In multi stage graph problem, we have to find the shortest path from source vertex to destination vertex.

→ The cost of each path is calculated by using the weight given along that edge. The cost of a path from source vertex to destination vertex is the sum of the costs of edges on the path.

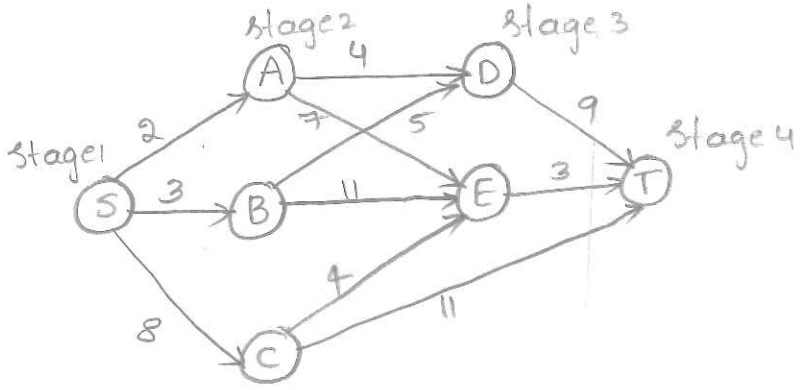
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ The multi stage graph can be solved using backward approach and forward approach

Backward approach:

→ Consider the following graph



→ There is a single vertex in stage 1, then 3 vertices in stage 2, then two vertices in stage 3 and only one vertex in stage 4 (This is a target stage)

→ c(s,T) where s is source vertex, T is target vertex.

$cost(s,T) = \min\{2 + cost(A,T), 3 + cost(B,T), 8 + cost(C,T)\}$ → eqn ①

→ Now compute cost(A,T), cost(B,T) and cost(C,T)

$cos(A,T) = \min\{4 + cost(D,T), 7 + cost(E,T)\}$ → eqn ②

$cost(B,T) = \min\{5 + cost(D,T), 11 + cost(E,T)\}$ → eqn ③

$cost(C,T) = \min\{4 + cost(E,T), 11\}$

→ Compute cost(D,T) and cost(E,T)

$Cost(D,T) = 9$

$Cost(E,T) = 3$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ The Backward vertex is E

→ Substitute these values in eqn ②, ③, ④

$$cost(A,T) = \min\{4+9, 7+3\}$$
$$= \min\{13, 10\}$$
$$= 10$$

$$cost(B,T) = \min\{5+9, 11+3\}$$
$$= 14$$

$$cost(C,T) = \min\{4+3, 11\}$$
$$= 7.$$

∴ Therefore, cost(S,T) is

$$cost(S,T) = \min\{2+cost(A,T), 3+cost(B,T), 8+cost(C,T)\}$$
$$= \min\{2+10, 3+14, 8+7\}$$
$$= \min\{12, 17, 15\}$$

$cost(S,T) = 12$

→ The path with minimum cost is S – A – E – T with the cost 12.

→ This method is known as backward approach.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

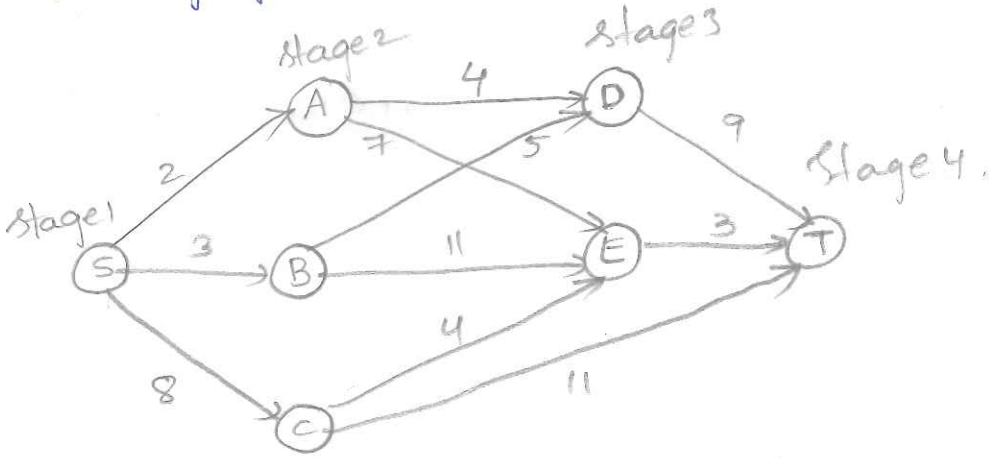
www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Algorithm for backward approach:

```
Algorithm BGraph (G, k, n, p)
{
b cost[i] := 0
for j := 2 to n do
{
let r be such that <r, j> is an edge of
  G and b cost[r] + c[r, j] is minimum
  b cost[j] := b cost[r] + c[r, j];
  d[j] := r;
  }
  P[1] := 1 ; P[k] := n;
  for j := k-1 to 2 do P[j] := d[P[j+1]];
  }
```

* Forward approach:

→ Find the shortest path between source vertex and destination vertex using forward approach for given graph.



cost(S, A) = 2

cost(S, B) = 3

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$cost(S,C) = 8$

$Cost(S,D) = \min\{2 + cost(A,D), 3 + cost(B,D)\}$

$= \min\{2+4, 3+5\}$

$= 6.$

$cost(S,E) = \min\{2 + cost(A,E), 3 + cost(B,E), 8 + cost(C,E)\}$

$= \min\{2+7, 3+11, 8+4\}$

$= 9.$

$Cost(S,T) = \min\{cost(S,D) + cost(D,T), cost(S,E) + cost(E,T),$

$cost(S,C) + cost(C,T)\}$

$= \min\{6+9, 9+3, 8+11\}$

$Cost(S,T) = 12$

→ The minimum cost is 12 with path S-A-D-T. This method is called forward approach. 2)

→ Time complexity for forward approach is

$\Theta(|V|+|E|)$

→ Time Complexity for backward approach is

$\Theta(|V|+|E|)$.

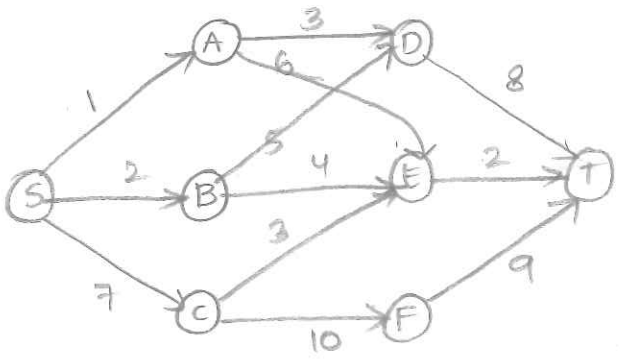
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Algorithm for forward approach:

```
Algorithm fGraph (G, k, n, p)
{
cost[n] := 0.0;
for j = n-1 to 1 step do
{
let r be a vertex such that <j, r> is an edge
of G and C[j, r] + cost[r] is minimum
    cost[j] := c[j, r] + cost[r];
    d[j] := r;
    }
    P[1] := 1; P[k] := n;
    for j := 2 to k-1 do p[j] := d[p(j-1)];
    }
```

2)



. Find using backward approach.

$$cost(S,T) = \min\{1+cost(A,T),\ 2+cost(B,T),\ 7+cost(C,T)\}$$

$$cost(A,T) = \min\{3+cost(D,T),\ 6+cost(E,T)\}$$

$$= \min\{3+8,\ 6+2\}$$

$$= 8$$

$$cost(B,T) = \min\{5+cost(D,T),\ 4+cost(E,T)\}$$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$= \min\{5+8, 4+2\}$

$= 6$

$cost(C,T) = \{\min\{3+cost(E,T), 10+cost(E,T)\}$

$= \min\{3+2, 10+9\}$

$= 5$

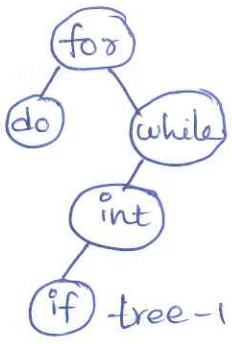
$cost(S,T) = \{1+8, 2+6, 7+5\}$

$= 8$

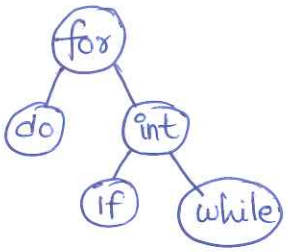
path is S-B-E-T.

*Optimal binary search tree:

→ Optimal binary search tree is a binary search tree for which the nodes are arranged such that the tree cost is minimum. Consider the 2 following possible BST.



tree-1



tree 2

→ The above tree consist of set of identifiers for, do, while, int, if.

→ we can create different BST for same identifier set.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ In the 1st tree, we require 4 comparisons to find the identifier in the worst case.

→ In the 2nd tree, we requires '3' comparisons to find an identifier in the worst case.

→ Average no of comparisons required for 1st tree is

$$\frac{1+2+2+3+4}{5} = \frac{12}{5} = 2.4$$

→ Average no of comparisons required for 2nd tree is

$$\frac{1+2+2+3+3}{5} = \frac{11}{5} = 2.2.$$

→ Second BST is considered as OBST where its cost is 2.2.

→ In a general situation, we can expect different identifiers to be searched for different probabilities. In addition, we can expect unsuccessful searches also to be made.

→ To obtain a cost function for binary search trees it is useful to add external nodes.

→ External nodes are denoted with square symbol.

→ Internal nodes are denoted with circular symbol.

→ If a binary search tree represents 'n' identifiers then there will be exactly 'n' internal nodes and

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

'n+1' external nodes.

→ Every internal node represents a point where a successful search may terminate.

→ Every external node represents a point where an unsuccessful search may terminate.

→ The following formula is used to find the cost of a binary search tree.

$$\sum_{1 \le i \le n} P(i) * level(a_i) + \sum_{0 \le i \le n} q(i) * (level(E_i) - 1)$$

where P(i) is probability of internal nodes

a_i $(a_1, a_2, \ldots a_n)$ represents internal nodes.

q(i) is probability of external nodes

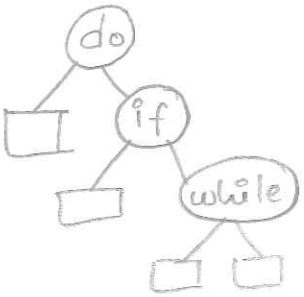
E_i $(E_1, E_2, \ldots E_n)$ represents external nodes

10) Consider the identifier set $(a_1, a_2, a_3) = (do, if, while)$ and probabilities of internal nodes are $P(1) = 0.5$, $P(2) = 0.1$, $P(3) = 0.05$ and probabilities of external nodes are $q(0) = 0.15$, $q(1) = 0.1$, $q(2) = 0.05$, $q(3) = 0.05$. Construct possible binary search trees and find OBST.

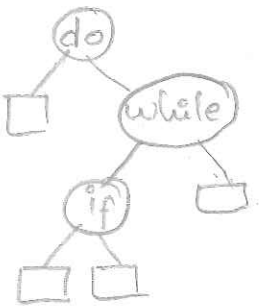
Sol:- Construct possible BST's for the given identifiers (do, if, while).

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

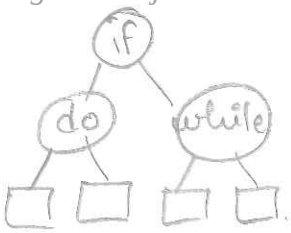
www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh



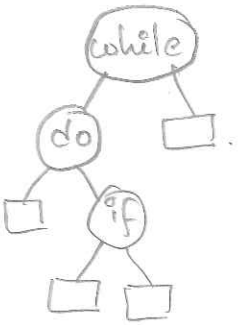
tree 1



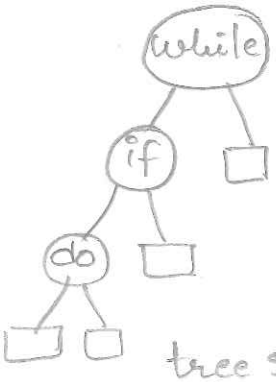
tree 2



tree 3



tree 4



tree 5

cost(tree 1) = $[P(1).\ level(a_1) + P(2).\ level(a_2) + P(3).\ level(a_3)]$

$+[q(0)\ level(\varepsilon_0 - 1) + q(1)\ level(\varepsilon_1 - 1) + q(2)\ level(\varepsilon_2 - 1)$

$+ q(3)\ level(\varepsilon_3 - 1)]$

$= [0.5 \times 1 + 0.1 \times 2 + 0.05 \times 3] + [0.15 \times 1 + 0.1 \times 2 + 0.05 \times 3 + 0.05 \times 3]$

$= 0.85 + 0.65$

$= 1.5.$

cost(tree2) $= (0.5 \times 1 + 0.1 \times 3 + 0.05 \times 2) + (0.15 \times 1 + 0.1 \times 2 + 0.05 \times 3 + 0.05 \times 3)$

$= 0.9 + 0.65$

$= 1.55$

cost(tree3) $= (0.5 \times 2 + 0.1 \times 2 + 0.05 \times 2) + (0.15 \times 2 + 0.1 \times 2 + 0.05 \times 2 + 0.05 \times 2)$

$= 1.9$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

cost (tree 4) = [0.5x2 + 0.1x3 + 0.05x1] + [0.15x1 + 0.1x2 + 0.05x3 + 0.05x3]

= 1.35 + 0.65.

= 2.

cost (trees) = [0.5x3 + 0.1x2 + 0.05x1] + [0.15x1 + 0.1x2 + 0.05x3 + 0.05x3]

= (1.5 + 0.2 + 0.05) + (0.65)

= 1.75 + 0.65

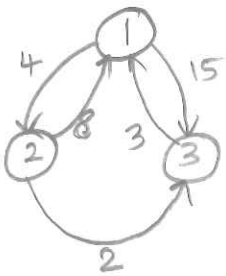
= 2.4

1.5
0.2
0.05
1.75
0.65
2.40

* All pairs shortest path problem:

→ The problem of determining the shortest path among all pairs of vertices of a given graph is

Consider the given graph



→ The above graph consisting of three vertices 1, 2, 3. So we have to find shortest paths b/w all types of vertices 1, 2, 3.

→ All pairs are A(1,1), A(1,2), A(1,3), A(2,1), A(2,2), A(2,3), A(3,1), A(3,2), A(3,3)

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Step-1 → ~~In adjacen~~ Represent the given matrix in adjacency matrix form.

$$A^0 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \end{array} \begin{array}{c} \begin{array}{ccc} 1 & 2 & 3 \end{array} \\ \begin{bmatrix} 0 & 4 & 15 \\ 8 & 0 & 2 \\ 3 & \infty & 0 \end{bmatrix} \end{array}$$

$$A^0 = w[i,j]$$

$$A^k_{(i,j)} = \min\left\{ A^{k-1}_{(i,j)},\ A^{k-1}_{(i,k)} + A^{k-1}_{(k,j)} \right\}$$

Step-2: Take vertex 1 as intermediate vertex ($k=1$) and find shortest path among all pairs of vertices

$i=1, j=1, k=1$

$$A^1_{(1,1)} = \min\left\{ A^{k-1}_{(1,1)},\ A^{k-1}_{(1,1)} + A^{k-1}_{(1,1)} \right\}$$

$$= \min\left\{ A^0_{(1,1)},\ A^0_{(1,1)} + A^0_{(1,1)} \right\}$$

$$= \min\{0, 0+0\}$$

$$= 0$$

$i=1, j=2, k=1$

$$A^1_{(1,2)} = \min\left\{ A^0_{(1,2)},\ A^0_{(1,1)} + A^0_{(1,2)} \right\}$$

$$= \min\{4, 0+4\}$$

$$= 4$$

$i=1, j=3, k=1$

$$A^1_{(1,3)} = \min\left\{ A^0_{(1,3)},\ A^0_{(1,1)} + A^0_{(1,3)} \right\}$$

$$= \min\{15, 0+15\}$$

$$= 15$$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Step-1

→ ~~In adjacen~~ Represent the given matrix in adjacency matrix form.

$$A^0 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \end{array} \begin{array}{c} \begin{array}{ccc} 1 & 2 & 3 \end{array} \\ \begin{bmatrix} 0 & 4 & 15 \\ 8 & 0 & 2 \\ 3 & \infty & 0 \end{bmatrix} \end{array}$$

$$A^0 = w[i,j]$$

$$A^k_{(i,j)} = \min\left\{ A^{k-1}_{(i,j)},\ A^{k-1}_{(i,k)} + A^{k-1}_{(k,j)} \right\}$$

Step-2: Take vertex 1 as intermediate vertex ($k=1$) and find shortest path among all pairs of vertices

$i=1, j=1, k=1$

$$A^1_{(1,1)} = \min\left\{ A^{k-1}_{(1,1)},\ A^{k-1}_{(1,1)} + A^{k-1}_{(1,1)} \right\}$$

$$= \min\left\{ A^0_{(1,1)},\ A^0_{(1,1)} + A^0_{(1,1)} \right\}$$

$$= \min\{0, 0+0\}$$

$$= 0.$$

$i=1, j=2, k=1$

$$A^1_{(1,2)} = \min\left\{ A^0_{(1,2)},\ A^0_{(1,1)} + A^0_{(1,2)} \right\}$$

$$= \min\{4, 0+4\}$$

$$= 4$$

$i=1, j=3, k=1$

$$A^1_{(1,3)} = \min\left\{ A^0_{(1,3)},\ A^0_{(1,1)} + A^0_{(1,3)} \right\}$$

$$= \min\{15, 0+15\}$$

$$= 15$$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$i=2, j=1, k=1$

$A^1_{(2,1)} = \min\{A^0_{(2,1)}, A^0_{(2,1)} + A^0_{(1,1)}\}$

$= \min\{8, 8+0\}$

$= 8$

$A^1_{(2,2)} =$

$i=2, j=2, k=1$

$= \min\{A^0_{(2,2)}, A^0_{(2,1)} + A^0_{(1,2)}\}$

$= \min\{0, 8+4\}$

$= 0.$

$A^1_{(2,3)} =$

$i=2, j=3, k=1$

$= \min\{A^0_{(2,3)}, A^0_{(2,1)} + A^0_{(1,3)}\}$

$= \min\{2, 8+15\}$

$= 2.$

$A^1_{(3,1)} =$

$i=3, j=1, k=1$

$= \min\{A^0_{(3,1)}, A^0_{(3,1)} + A^0_{(1,1)}\}$

$= \min\{3, 3+0\}$

$= 3$

$A^1_{(3,2)} =$

$i=3, j=2, k=1$

$= \min\{A^0_{(3,2)}, A^0_{(3,1)} + A^0_{(1,2)}\}$

$= \min\{\infty, 3+4\}$

$= 7$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$A^1_{(3,3)} \Rightarrow i=3, j=3, k=1$

$= \min(0, 3+15)$

$= 0.$

$$A^1 = \begin{bmatrix} 0 & 4 & 15 \\ 8 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$$

Step-3: Take vertex '2' as a intermediate and find shortest path between all pairs of vertices.

$A^2_{(1,1)} = \min\{0, 4+8\} = 0$

$A^2_{(1,2)} = \min\{4, 4+0\} = 4$

$A^2_{(1,3)} = \min\{15, 4+2\} = 6.$

$A^2_{(2,1)} = \min\{8, 0+8\} = 8$

$A^2_{(2,2)} = \min\{0, 0+0\} = 0.$

$A^2_{(2,3)} = \min\{2, 0+2\} = 2$

$A^2_{(3,1)} = \min\{3, 7+8\} = 3$

$A^2_{(3,2)} = \min\{7, 7+0\} = 7$

$A^2_{(3,3)} = \min\{0, 7+2\} = 0.$

$$A^2 = \begin{bmatrix} 0 & 4 & 6 \\ 8 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Step-4: Take vertex '3' as intermediate and find shortest path between all pairs of vertices.

$A^3_{(1,1)} = \min\{0, 6+3\} = 0$

$A^3_{(1,2)} = \min\{4, 6+7\} = 4$

$A^3_{(1,3)} = \min\{6, 6+0\} = 6$

$A^3_{(2,1)} = \min\{8, 2+3\} = 5$

$A^3_{(2,2)} = \min\{0, 2+7\} = 0$

$A^3_{(2,3)} = \min\{2, 2+0\} = 2$

$A^3_{(3,1)} = \min\{3, 0+3\} = 3$

$A^3_{(3,2)} = \min\{7, 0+7\} = 7$

$A^3_{(3,3)} = \min\{0, 0+0\} = 0$

$$A^3 = \begin{bmatrix} 0 & 4 & 6 \\ 5 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$$

* 0/1 knapsack problem:

→ It contains n items with profits $(P_1, P_2, \ldots P_n)$ and weights $(w_1, w_2, \ldots w_n)$. knapsack means empty bag it has a capacity of m units. and the objective

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

function of this problem is to get maximum profit by filling empty back with given items. Subject to constraints $0 \le x_i \le 1$ and $m \le \Sigma w_i$, $i \le n$.

→ we can solve this problem by using dynamic programming method

→ It uses the following steps.

Step-1:

Computational formula

$$S_1^i = S^{i-1} + (P_i, w_i)$$

→ It is used to add tuples

$$S^i = S^{i-1} + S_1^i$$

→ This formula is used for merging operation.

Step-2:

purging rule

→ It decides which tuple is to be discard and which tuple is to be accept

→ If one of S^{i-1} and S_1^i has a pair (P_j, w_j) and other (P_k, w_k) and $P_j \le P_k$, $w_j > w_k$ then discard (P_j, w_j)

Step-3:

Selection of items:

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

In this step, we can decide which item is to be placed in the bag based on following constraints

if $(P_i, w_i) \in s^n$ & $(P_i, w_i) \notin s^{n-1}$ then $x_n = 1$ else $x_n = 0$.

Problems:

1) Consider the instance $n=3$, $m=6$, $(P_1, P_2, P_3) = (1,2,5)$ $(w_1, w_2, w_3) = (2,3,4)$. Find the maximum profit by using dynamic programming method.

Sol:- Step-1: Initially

$$s_i^0 = (0,0), \quad s^0 = (0,0)$$
$$= (P_i, w_i)$$

$i=1 \Rightarrow s_1^i = s^{i-1} + (P_i, w_i)$

$$= s^0 + (P_1, w_1)$$
$$= (0,0) + (1,2)$$
$$= (1,2)$$

merging $i=1$

$$s^i = s^{i-1} + s_1^i$$
$$s^1 = s^0 + s_1^1$$
$$= (0,0) + (1,2)$$
$$s^1 = \{(0,0), (1,2)\}$$

$i=2 \Rightarrow s_2^i = s^{i-1} + (P_i, w_i)$

$$= s^1 + (P_2, w_2)$$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$= \{(0,0),(1,2)\} + \{(2,3)\}$

$= \{(2,3),(3,5)\}$

$S^i = S^{i-1} + S_1^i$

$S^2 = S^1 + S_1^2$

$= \{(0,0),(1,2)\} + \{(2,3),(3,5)\}$

$S^2 = \{(0,0),(1,2),(2,3),(3,5)\}$

i = 3

$S_1^i = S^{i-1} + (P_i, W_i)$

$S_1^3 = S^2 + (P_3, W_3)$

$\{(0,0),(1,2),(2,3),(3,5)\} + \{(5,4)\}$

$= \{(5,4),(6,6),(7,7),(8,9)\}$

$S^i = S^{i-1} + S_1^i$

$S^3 = S^2 + S_1^3$

$= \{(0,0),(1,2),(2,3),(3,5)\} + \{(5,4),(6,6),(7,7),(8,9)\}$

$= \{(0,0),(1,2),(2,3),(3,5),(5,4),(6,6),(7,7),(8,9)\}$

Step-2: Apply the purging rule on tuples of

$S^3 = \{(0,0),(1,2),(2,3),(3,5),(5,4),(6,6),(7,7),(8,9)\}$

discard tuples (7,7), (8,9) because tuple values exceeded the size of the knapsack size m=6.

→ Apply the purging rule on (3,5) and (5,4)

$(3,5) \xi (5,4)$

$(P_j, W_j) \xi (P_k, W_k)$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$(P_j < P_k)$ and $(w_j > w_k)$

$(3 < 5)$ and $(5 > 4)$ so discard $(3,5)$ tuple.

$S^3 = \{(0,0), (1,2), (2,3), (5,4), (6,6)\}$

Step-3:

Decide which items are placed in the bag

$(P_i w_i) = S^n$ & $(P_i w_i) \notin S^{n-1}$

$n = 3$

$(6,6) \in S^3$ and $(6,6) \notin S^2$

T & T = T

$x_n = 1 = x_3 = 1$

$(P_3, w_3) = (5,4)$

$(6,6) - (5,4) = (1,2)$

$(1,2) \in S^2$ & $(1,2) \notin S^1$

T & F

$x_2 = 0$.

$(1,2) \in S^1$ & $(1,2) \notin S^0$

T & T = T

$(x_1, x_2, x_3) = (1,0,1)$

$\sum P_i x_i = P_1 x_1 + P_2 x_2 + P_3 x_3$

$= 1 \cdot 1 + 2 \cdot 0 + 5 \cdot 1$

$= 1 + 0 + 5$

$= 6$.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* Travelling salesperson problem:

→ It is to find a tour of minimum cost. A tour of graph 'G' is a directed simple cycle that includes every vertex in 'V'.

→ The cost of the tour is the sum of the cost of the edges on the tour.

→ Suppose we have to route a postal van to pick up mail from mail boxes located at 'i' different sites.

→ One vertex represents the post office from which the postal van starts and to which it must return.

→ Edge (i, j) is assigned a cost equal to the distance from site 'i' to site 'j'.

→ The route taken by postal van is a tour and we are interested to find a tour of minimum length.

→ We can solve travelling salesperson problem by using dynamic programming method.

→ It uses the following computational formulae..

$$g(i,s) = \min_{j \in s} \{ c_{ij} + g(j, s-\{j\}) \}$$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

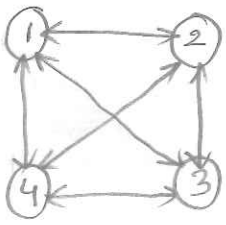
$g(i, \phi) = c_{i1}, \quad 1 \le i \le n$

where $g(i, S)$ be the length of shortest path starting at vertex i going to all vertices in S and terminating at vertex 1.

Problem:

1) consider the directed graph and its adjacent matrix and find tour of minimum cost of travelling sales person problem.

Sol:-



$$\begin{bmatrix} 0 & 10 & 5 & 20 \\ 5 & 0 & 9 & 10 \\ 6 & 13 & 0 & 12 \\ 8 & 8 & 9 & 0 \end{bmatrix}$$

Initially

$g(i, \phi) = c_{i1} \quad 1 \le i \le 4$

$g(1, \phi) = c_{11} = 0$

$g(2, \phi) = c_{21} = 5$

$g(3, \phi) = c_{31} = 6$

$g(4, \phi) = c_{41} = 8$

→ Take $|s| = 1$ that means set S contains only one vertex. From one vertex we can visit all the vertices 2, 3, 4

$$g(i, S) = \min_{j \in S} \{c_{ij} + g(j, S - \{j\})\}$$

$$g(2, \{3\}) = \min_{j \in S} \{c_{23} + g(3, \{3\} - \{3\})\}$$

$$= \min_{j \in S} \{9 + g(3, \phi)\}$$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$= \min\{9+6\}$

$= 15$

$g(2,\{4\}) = \min_{j\in\{4\}} \{c_{24} + g(4, \{4\}-\{4\})\}$

$= \min\{10+g(4,\phi)\}$

$= \min\{10+8\}$

$= 18$

$g(3,\{2\}) = \min_{j\in\{2\}} \{c_{32} + g(2,\{2\}-\{2\})\}$

$= \min_{j\in\{2\}} (13+5)$

$= 18$

$g(3,\{4\}) = \min_{j\in\{4\}} \{c_{34} + g(4,\{4\}-\{4\})\}$

$= \min\{12+g(4,\phi)\}$

$= 12+8$

$= 20.$

$g(4,\{2\}) = \min_{j\in\{2\}} \{c_{42} + g(2,\{2\}-\{2\})\}$

$= \min\{8+g(2,\phi)\}$

$= 13$

From all the

$g(4,\{3\}) = \min\{c_{43} + g(3,\{3\}-\{3\})\}$

$= \min\{9+g(3,\phi)\}$

$= 15.$

→ Take $|s| = 2$ that means set s contains only two vertices.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$$g(2,\{3,4\}) = \min_{j\in\{3,4\}} \{c_{23}+g(3,\{3,4\}-\{3\}),\ c_{24}+g(4,\{3,4\}-\{4\})\}$$

$$= \min\{9+g(3,\{4\}),\ 10+g(4,\{3\})\}$$

$$= \min\{9+20,\ 10+15\}$$

$$= 25.$$

$$g(3,\{2,4\}) = \min_{j\in\{2,4\}} \{c_{32}+g(2,\{2,4\}-\{2\}),\ c_{34}+g(4,\{2,4\}-\{4\})\}$$

$$= \min\{13+g(2,\{4\}),\ 12+g(4,\{2\})\}$$

$$= \min\{13+18,\ 12+13\}$$

$$= 25$$

$$g(4,\{2,3\}) = \min_{j\in\{2,3\}} \{c_{42}+g(2,\{2,3\}-\{2\}),\ c_{43}+g(3,\{2,3\}-\{3\})\}$$

$$= \min\{8+g(2,\{3\}),\ 9+g(3,\{2\})\}$$

$$= \min\{8+15,\ 9+18\}$$

$$= 23$$

$$g(1,\{2,3,4\}) = \min_{j\in\{2,3,4\}} \{c_{12}+g(2,\{2,3,4\}-\{2\}),\ c_{13}+g(3,\{2,3,4\}-\{3\}),$$
$$c_{14}+g(4,\{2,3,4\}-\{4\})\}$$

$$= \min\{10+g(2,\{3,4\}),\ 15+g(3,\{2,4\}),\ 20+g(4,\{2,3\})\}$$

$$= \min\{10+25,\ 15+25,\ 20+23\}$$

$$= \min\{35, 40, 43\}$$

$$= 35$$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

path of the four with minimum cost. 35

$c_{12} + g(2, \{3,4\})$

$= c_{12} + c_{24} + g(4, \{3\})$

$= c_{12} + c_{24} + c_{43} + g(3, \phi)$

$= c_{12} + c_{24} + c_{43} + c_{31}$

$= 10 + 10 + 9 + 6$

$= 35$

path is $1 \to 2 \to 4 \to 3 \to 1$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

28/3/17

UNIT-IV

BACK TRACKING

→ Many problems which deals with searching for a set of solutions ~~are~~ which ask for an optimal solution satisfying some constraints can be solved using back-tracking method.

→ Dynamic programming design method uses ~~root~~ drute force approach to find the optimal solution. Here drute force approach will use ~~always~~ all possible ways to find an optimal solution.

→ we can use backtracking method to find the optimal solution with less no.of steps. In this method, if we are trying to find the Soln in one path and unable to get the solution then stop there, come back to previous step and choose alternate path to find the optimal solution.

→ many of the problems we solve using back-tracking method require that all the solutions satisfies

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

a complex set of constraints.

→ For any problem these constraints can be divided into two catogories.

1. Explicit constraint
2. Implicit constraint.

1. Explicit constraint:

→ Explicit constraints are the rules that restricts each 'x_i' to take on values only from a given set.

→ common examples of Explicit constraints are

$x_i \geq 0$, $x_i = 0$ or 1

2. Implicit constraint:

→ Implicit constraints are the rules that determine which of the tuples in the solution space of 'I' satisfies the criterian function.

*Applications of backtracking:

1. n-queens problem
2. Sum of Subsets
3. Graph colouring
4. Hamiltonean cycle.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

1. n-queens problems:

→ In this problem, n-queens are to be placed on an n×n chess board in such a way that no two queens are placed on same row, same column and same diagonal.

Ex: 4-queens problem
8-queens problem

*4-queens problem:

→ In this problem, 4-queens are to be placed on 4×4 chess board so that no two queens are placed on same row, column and diagonal.

Explicit constraint:

→ Let us consider the number the rows and columns of chess board as 1,2,3,4. Queens can also be numbered 1,2,3,4. So the values of set $s_i = \{1,2,3,4\}$. Therefore solution space contains '4' ways.

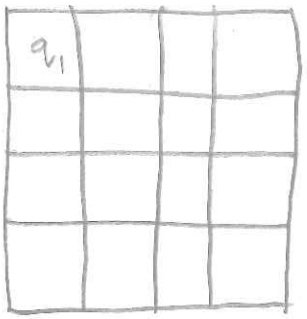
Implicit constraint:

→ No two queens are placed on same row, same column and same diagonal.

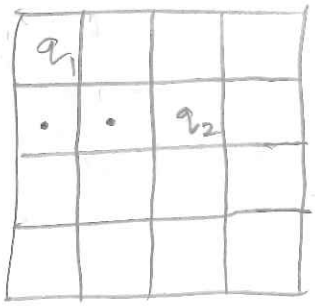
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

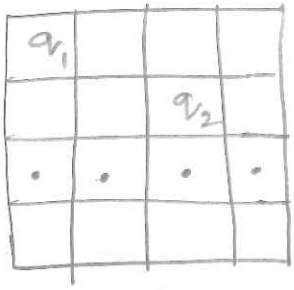
Step-1: Place queen 1 (q_1) on first row, first column.



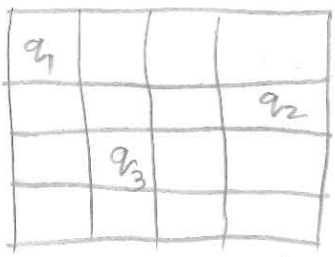
Step-2: Queen 2 (q_2) cannot be placed on second row first column and also second column. So place on third column.



Step-3: Queen 3(q_3) cannot be placed on any one of the columns



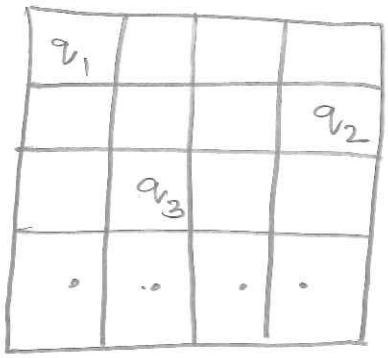
Backtrack to the previous step and place q_2 in another possible column i.e 4th column. and place q_3 in 2nd Column

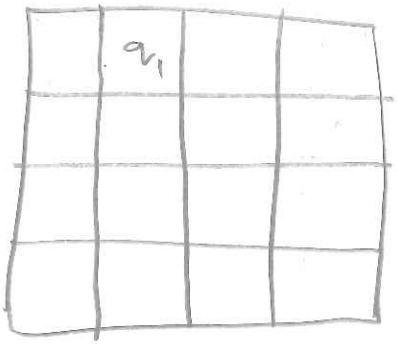


www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

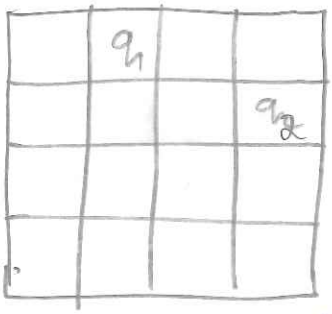
www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Step-4: Queen 4 (q_4) cannot be placed on any of the columns. So backtrack to previous step q_3. All possible ways are completed for q_3 so backtrack to q_2. All possible ways are completed for q_2 also so backtrack to q_1. and change the position of q_1 from 1st column to second column.

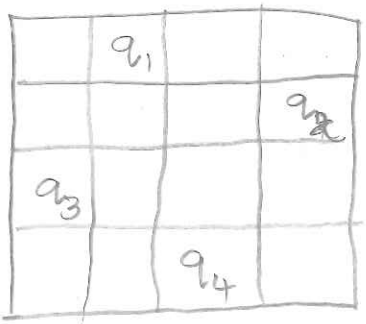




Step-5: Place q_2 on fourth column



Step-6: Place q_3 on 1st column & q_4 on 3rd column.



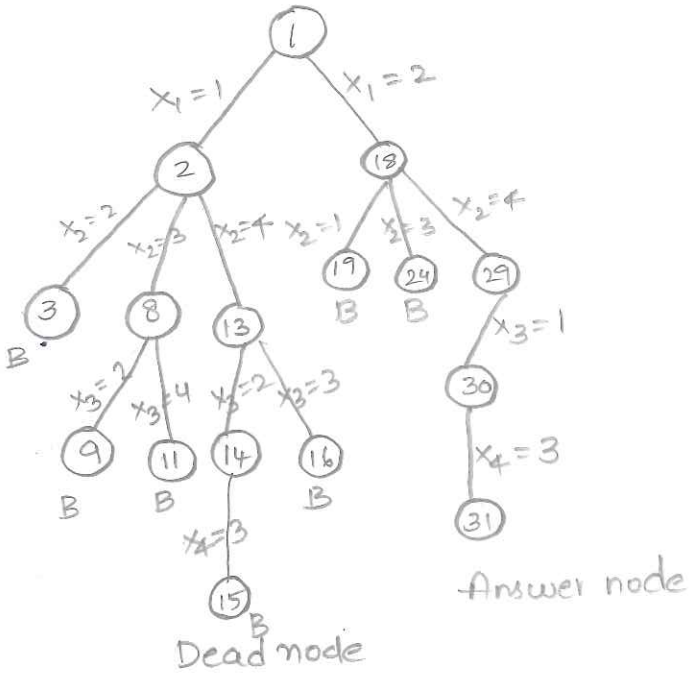
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Optimal solution:

$\{x_1, x_2, x_3, x_4\} = \{2, 4, 1, 3\}$

State space tree for 4-queens problem:



* Problem state:

Each node in state space tree defines a problem state

* State space:

All the paths from root node to other nodes define the state space of the problem.

* Solution state:

→ Solution state are problem states for which the path from the root to the other nodes defines a tuples in a solution space.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* Answer State:

→ Answer states are the solution states for which the path from root to other nodes defines a tuples that is a member of the set of the solutions of the problem.

* State Space tree:

→ A tree organisation of solution space is represented as state space tree.

* Live node:

→ A node which has been generated and whose children have not yet been generated is called a live node

* E-node:

→ The live node whose children are currently been generated is called E-node.

* Dead node:

→ A dead node is a generated node which is not to be expanded further

* Bounding function:

→ These are used to kill live nodes without

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

generating all their children.

* 8-queen's problem:

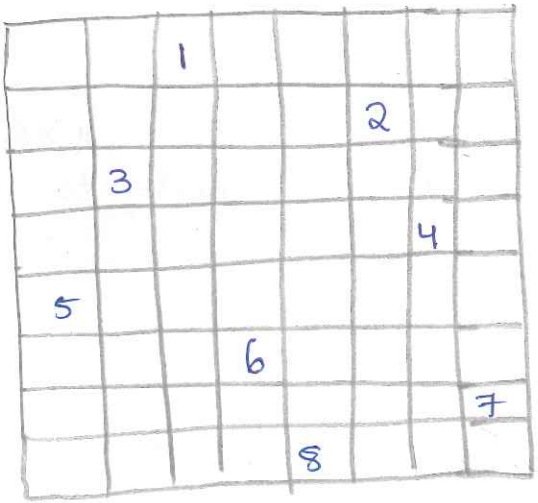
8-queen's prob is an objective function or criterian function. In this problem 8 queens are to be placed on 8x8 chess board so that no 2 queens placed on same row, same column and same diagonal.

Explicit constraint:

8x8 chess board contains 8 rows & 8 columns. The queens can also be numbered 1 to 8. So $S_i = \{1, 2, \ldots 8\}$ Therefore the solution space contains 8^8 tuples.

Implicit constraints:

No two queens are placed on same row, same column, same diagonal. One of solution's of 8 queen's problem.



$(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8) = (3, 6, 2, 7, 1, 4, 8, 5)$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Algorithm:

```
Algorithm place(k, i)
{
for j=1 to k-1 do
if ((x[j]= i) or (Abs(x[j]-i)= Abs(j-k)))
then return false;
return true
}
```

```
Algorithm NQueen(k, A)
{
for i:=1 to n do
{
if place(k, i) then
{
x[k]= i;
if (k==n) then write(x[1:n]);
else NQueens(k+1, n);
}
}
}
```

* Sum of subsets problem:

→ Suppose we are given n-distinct positive numbers and we need to find all combinations of this numbers whose sums are 'm'. This is called the sum of subsets problem.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Bounding function:

$$\sum_{i=1}^{k} w_i x_i + \sum_{i=k+1}^{n} w_i \geq m$$

Example: $n=4 \quad (w_1, w_2, w_3, w_4) = (11, 13, 24, 7)$

$m = 31$

Find all combination of the number whose sum is m.

Sol:-

Subset	Sum	Remarks
{ }	0	
{11}	11	
{11,13}	11+13=24	
{11,13,24}	11+13+24=48	Exceeds m value backtrack
{11,13}	-	-
{11,13,7}	11+13+7=31	Optimal soln

→ The optimal solution

$(x_1, x_2, x_3, x_4) = (1, 1, 0, 1)$

$11+13+7 = 31$ [whose sum = m]

→ One more optimal solution is

$(x_1, x_2, x_3, x_4) = (0, 0, 1, 1)$

$24+7 = 31$ [whose sum = m]

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* Recursive backtracking algorithm for sum of subset problem:

```
Algorithm sumofsub(s, i, sum)
{
x[i]:= 1;
if (s+w[i]= m) then write(x[1:n])
else if (s+w[i]+w[i+1]≤m)
then sumofsub(s+w[i], i+1, sum-w[i]);
if((s+sum+w[i]≥m)and(s+w[i+1]≤m))then
{
x[i]:=0;
sumofsub(s, i+1, sum-w[i]);
}
}
```

www.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ This algorithm takes 3 parameters (i.e) s, i, sum

where s is values of subsets

i is position of values or elements

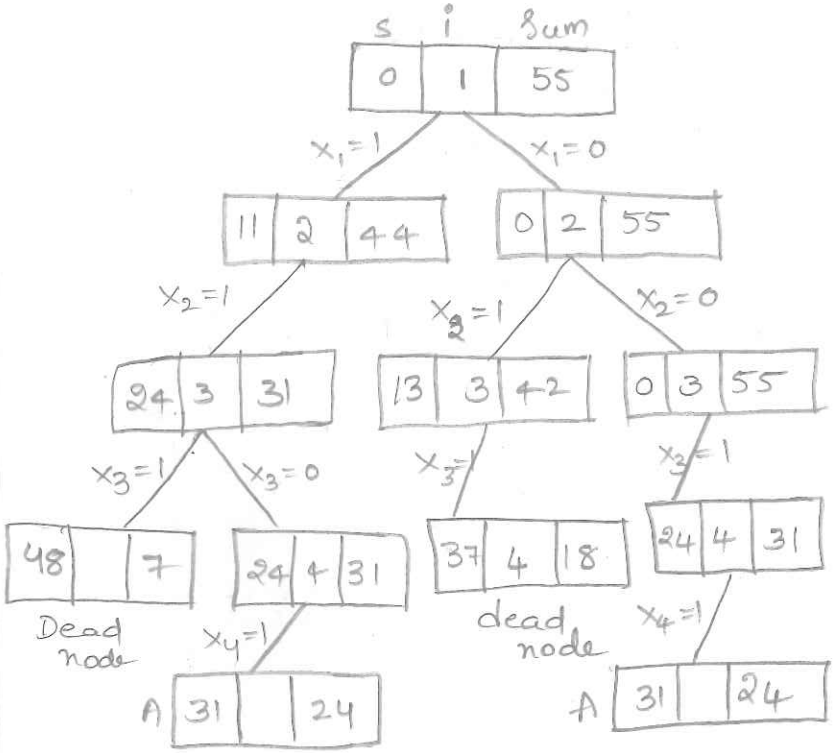
sum is sum of all elements of the given set.

for example: 1) $n = 4$

$(w_1, w_2, w_3, w_4) = (11, 13, 24, 7)$

$m = 31$

Draw the state space tree using recursive backtracking algorithm.



we got 2 optimal solutions subset (x_1, x_2, x_3, x_4)

$= (1, 1, 0, 1)$

$11 + 13 + 7 = 31$ [whose sum = m]

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

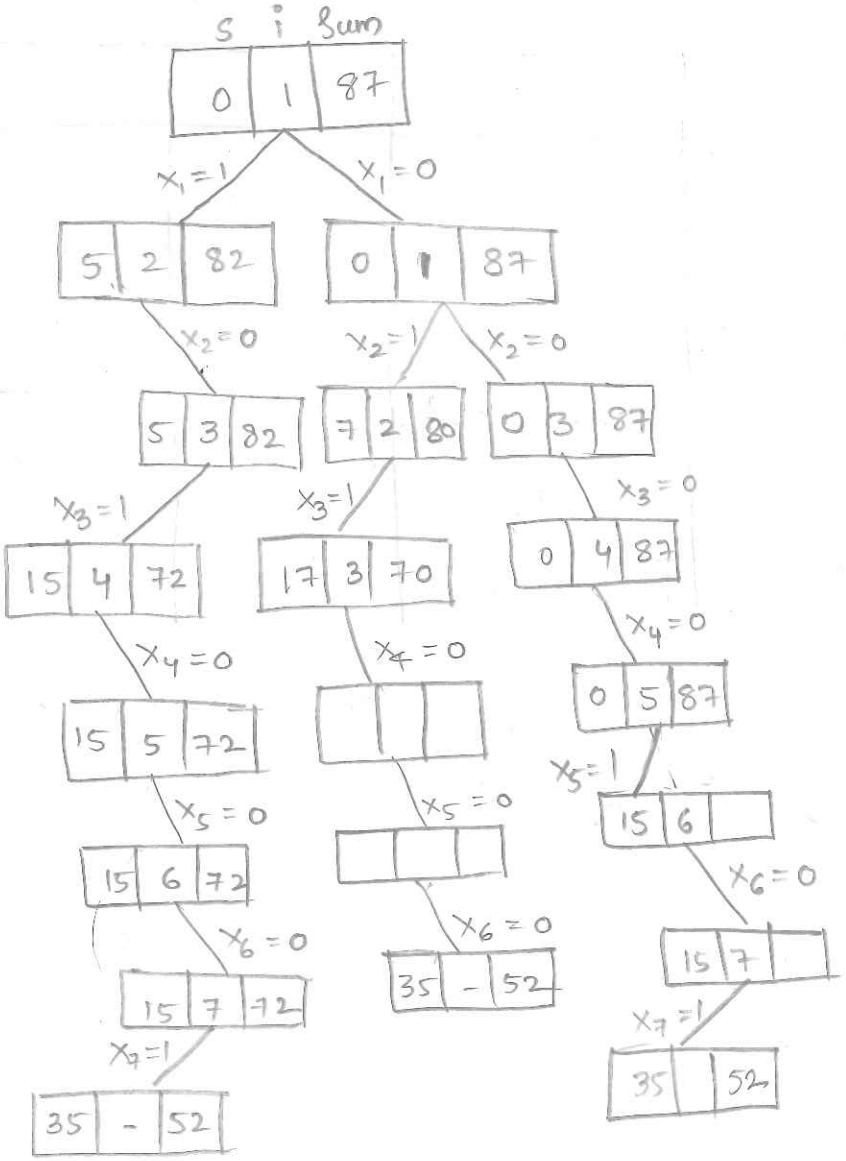
www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Subset 2 = $(x_1, x_2, x_3, x_4) = (0, 0, 1, 1)$

$2_4 + 7 = 31$

2) Let $w = (5, 7, 10, 12, 15, 18, 20)$ and $m = 35$ find all possible subsets of w that sum is m. Draw the portion of state space tree

Sol:-



www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* Graph coloring:

→ The process of coloring all vertices of a given graph in such a way that no two adjacent vertices have same colour yet only 'm' colours are used.

→ Note that if 'd' is the degree of the given graph, then it can be coloured with 'd+1' colours.

→ The m-colorability optimization problem asks for the smallest integer 'm' for which the graph 'G' can be coloured. This integer is referred to as the chromatic number of the graph.

→ A graph is said to be planar graph iff it can be drawn in a plane in such a way that no two edges cross each other.

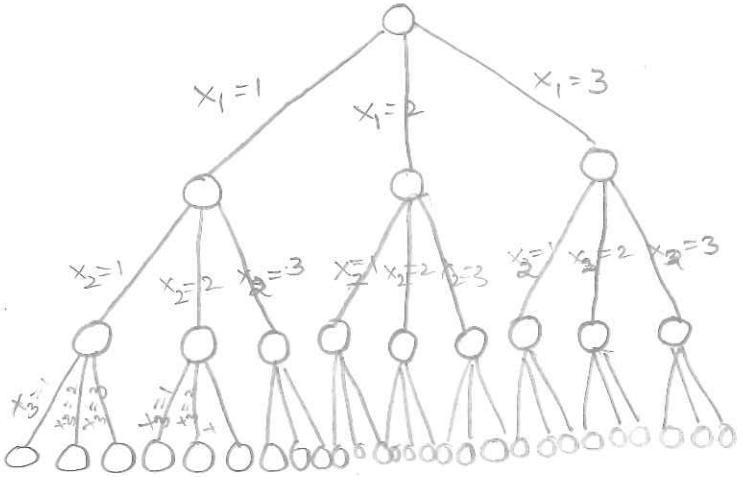
Ex:1) Draw a state space tree for m colouring. n=3,

m=3. n = no. of vertices
m = no. of colours.

Sol:- n=3
m=3

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh



Algorithm:

finding all m-colorings of graph

```
Algorithm mcoloring(k)
{
 repeat
 {
 Nextvalue(k);
 if(x[k]=0)then return;
if (k=n) then
write (x[1:n]);
else mcoloring(k+1);
}until(false);
}
```

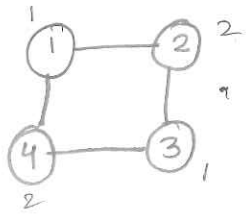
```
Algorithm Nextvalue(k)
{
 repeat
 {
  x[k]:=x[k]+1 mod(m+1);
if(x[k]=0) then return;
```

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

```
for j:=1 to n do
{
if((G[k,j]≠0) and (x[k]= x[j]))
then break;
}
if(j=n+1) then return
}until (false);
}
```

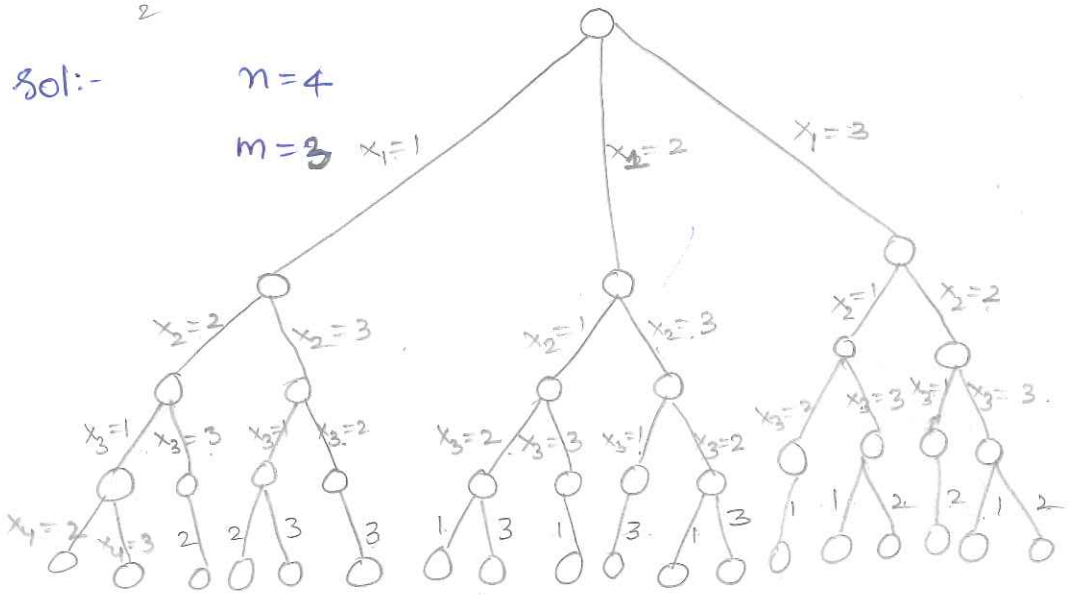
2) Color the given graph with m=3 colours



. Draw the state space tree.

Sol:- n=4

m=3



www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

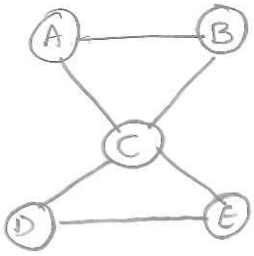
*Hamiltanion cycle:

→Let G = (V, E) be a connected graph with m-vertices.

*Hamiltonian path:

→ It is a path in a connected graph that visits each vertex exactly once.

Ex:



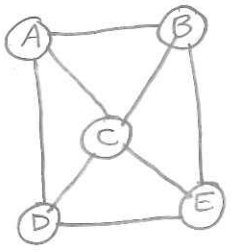
→ Hamiltanion path is A B C E D (or)
A B C D E.

→ Non-hamiltonion path is A B C E D C (or)
A B C D.

* Hamiltanion cycle:

→ Hamiltanion cycle is a cycle in a connected graph that visits each vertex exactly once and starting vertex, ending vertex must be same.

Ex:



→Hamiltanian cycle is A B C E D A (or)
A B E C D A

www.previousquestionpapers.com | https://telegram.me/jntuh

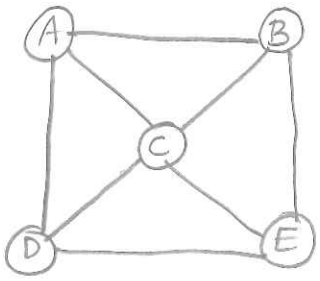
www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→Not Hamiltanian cycle is ABCEBA

* Hamiltanian graph:

→Hamiltonian graph is a graph that contains hamiltonian cycle.

Ex:

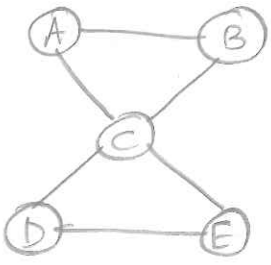


It consists of hamiltonian cycle. So, it is a hamiltonian graph.

*Traceable graph:

→Traceable graph is a graph that contains hamiltonian path is called traceable graph.

Ex:



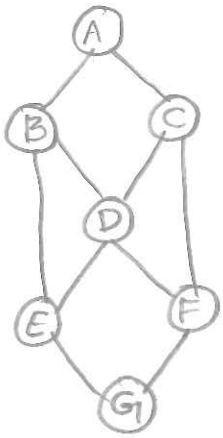
It consists of hamiltonian path ~~graph~~ so it is a traceable graph.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Problem-1:

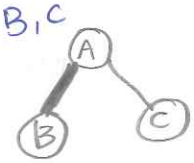
Check whether this graph consists of hamiltonian cycle or not?



Sol:- Step-1: Visit the source vertex A.

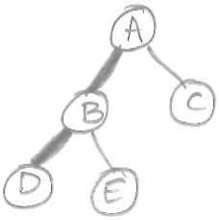
Step-2: Find all adjacent vertices of A and visit the nearest node.

A - B, C



Step-3: Find adjacent vertices of B and visit the nearest node. D

B - D, E.

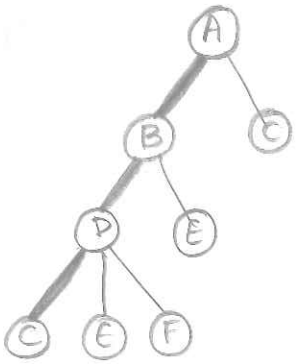


Step-4: Find all adjacent vertices of D (i.e) D - B, C, E, F but B is already visited so visit next nearest

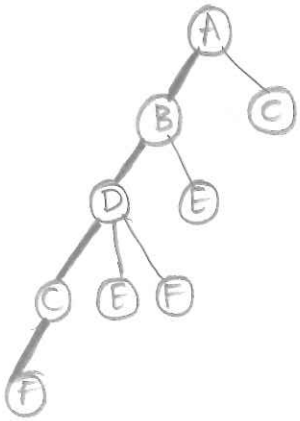
www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

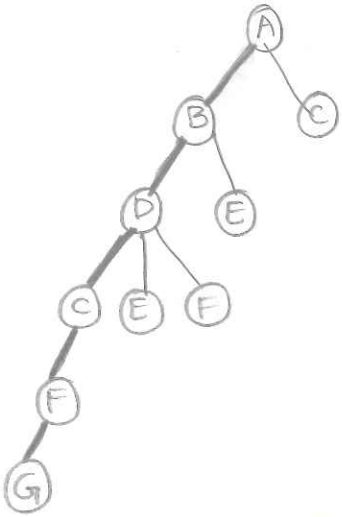
Vertex C.



Step-5: Find all adjacent vertices of C i.e C-A, D, F. Since A and D are already visited visit F.



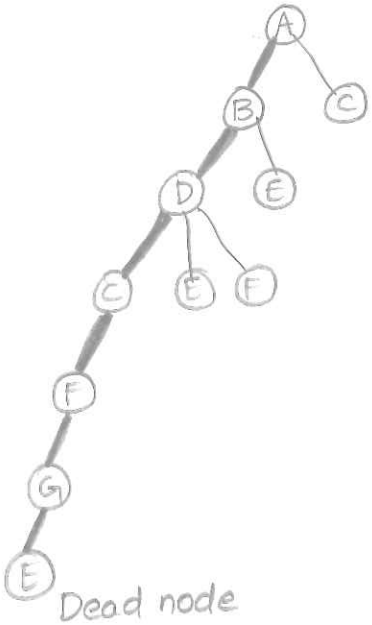
Step-6: Find all adjacent vertices of F i.e F-C, D, G. Since C and D are already visited visit 'G'.



Step-7: Find all adjacent vertices of G (i.e) G-E, F. F is already visited. visit E.

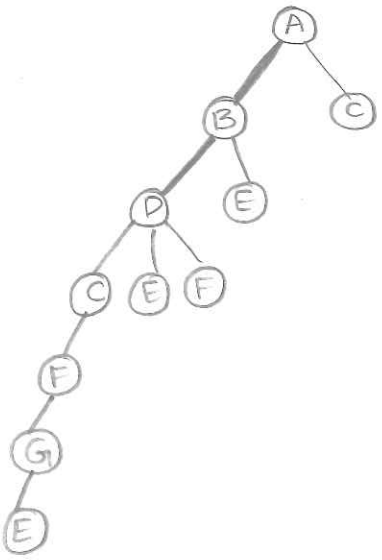
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh



Dead node

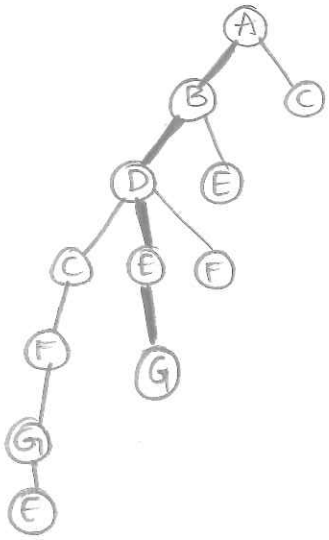
Step-8: Find all adjacent vertices of E i.e E - B, D, G. These vertices already visited and unable to generate child node. This node is known as dead node. Backtrack to vertex G there is no alternate path for G. So back track to F. no alternate path for vertex F also so backtrack to C. Since there is no alternate path for C backtrack to D. Now, choose alternate path and visit vertex E discard previous paths.



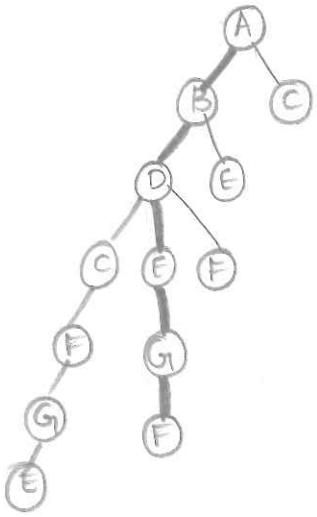
www.android.previousquestionpapers.com | www.universityupdates.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

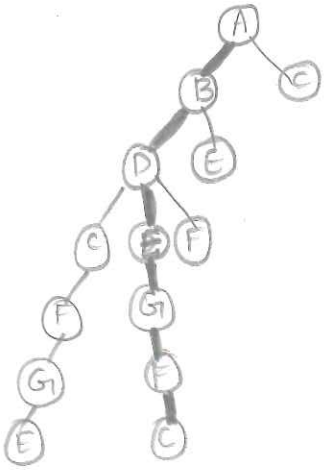
Step-9: Find all adjacent vertices of E (i.e) E - B, D, G visit G.



Step-10: Find all adjacent vertices of G (i-e) E, F. E is already visited so visit F.



Step-11: Find all adjacent vertices of F - G, C then visit C.

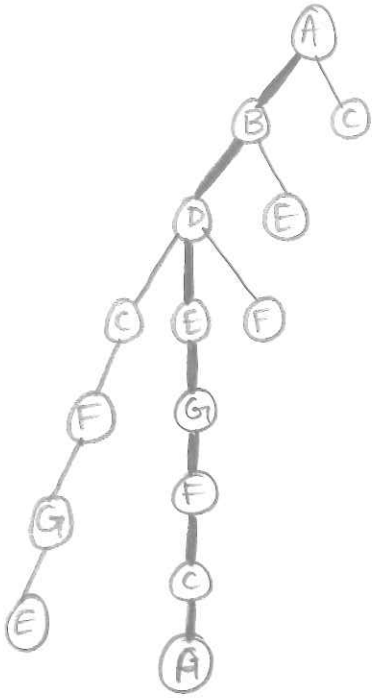


www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Step-12: Similarly find all adjacent vertices of C - A, D, F

Since D, F are already visited visit A.



Hamiltonian cycle: A B D E G F C A.

BRANCH AND BOUND

* Branch and bound:

→ The term branch and bound refers to all state space search methods in which all children of the E-node are generated before any other live node can become the E-node.

→ There are two graph search strategies.

1. BFS
2. DFS

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ In this methods, the expansion of a new node cannot begin until the node currently being the exploded is fully explored.

→ Both of this methods generalise to branch and bound strategies.

→ In Branch and bound terminology, BFS called as FIFO Search and DFS called as LIFO Search.

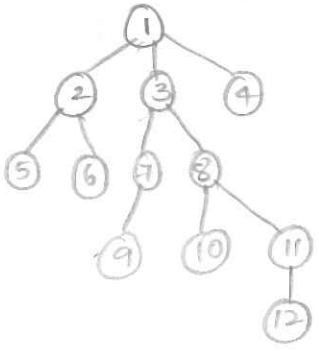
→ A branch and bound method searches a state space tree using any search mechanism in which all the children of the E-node are generated before another node becomes the E-node.

→ There are three common search strategies.

1. First in first out (FIFO)
2. Last in first out (LIFO)
3. Least cost search (LC)

1. First in first out search (FIFO):

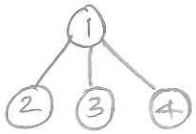
→ First in first out search method uses queues data structure. consider the following example.

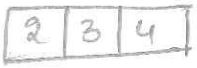


www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

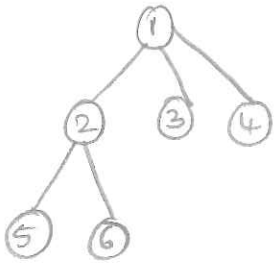
www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

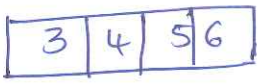
→ Assume that there is only one node. This node becomes Enode. Expand and ~~extent~~ generate all its children 2,3,4 nodes. Insert this nodes into the queue.



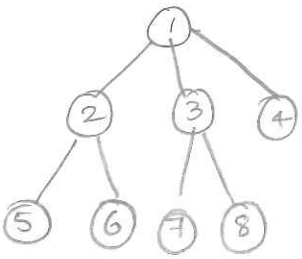


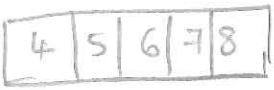
→ Queue follows FIFO. So remove node 2 from the queue and make it as E-node and generate its child nodes. 5,6 and insert 5,6 into queue





→ Make node 3 as E-node and generate it child nodes and insert into queue.





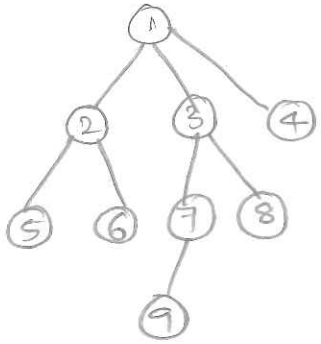
~~→ Make node 4 as E-node~~

→ Nodes 4,5,6 killed by bounding function because they are not possible to generate child nodes. Now 4,5,6 are removed from queue. Now, queue contains 7,8

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

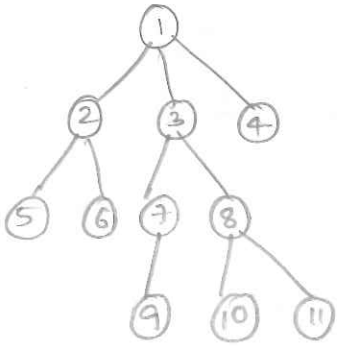
www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ Make 7 as E-node and generate its children and insert into queue.



8	9

→ Make 8 as E-node and generate its children and insert into queue

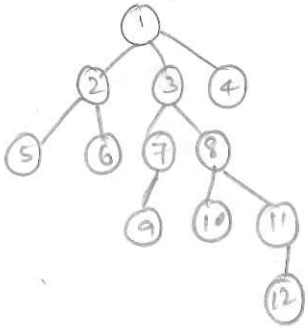


9	10	11

→ Make 9,10 as E-node since it does not generate children. i.e They are dead nodes. Remove 9,10 from queue.

11

→ Make 11 as the E-node and generate its child nodes and insert it into queue. 12 is the answer node and stop the process.



www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

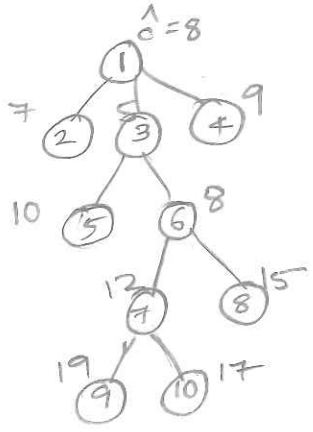
3. Least cost (LC):

→ In both FIFO and LIFO branch and bound the selection rule for next E-node is not efficient. The selection rule for next E-node does not give any preference to any node that has a very good chance of getting the search to an answer node quickly.

→ In least cost search, the next E-node is selected on the basis of ranking function ($\hat{c}()$).

→ Ranking function is used to find cost of each and every node. The node which has least cost is selected as E-node to generate child nodes. This process continues until the ~~defined~~ to find. answer node.

→ Consider the following example.



→ The cost of node 1 is 8 and make this node as E-node to generate child nodes 2, 3, 4. Find

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

the cost of this node using ranking function. The cost of nodes 2, 3, 4 are 7, 5, 9 respectively. The cost of node 3 is least and is selected as E-node to generate child nodes 5 and 6.

→ Find cost of 5 and 6 using ranking function. The node 6 cost is least(8) and select this node as E-node to generate child nodes 7 and 8.

→ Find cost of 7 and 8 using ranking function The costs are 12 and 15 respectively. The cost of node 7 is least and select this node as E-node to generate child nodes 9 & 10. The costs are 19 and 17 respectively. The cost of node 10 is least and it is answer node. Stop the process.

* Least cost algorithm:

→ Control abstraction for branch and bound using least cost search method:

```
Algorithm LCsearch (t)
{
if *t is an answer node then
    output *t and return;
  E := t, //E-node
```

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

```
Initialize the list of live nodes to be empty
repeat
for each child x of E do
{
if x is an answer node then output the path
from x to t and return
Add(x);
(x→parent) := E;
}
if there are no more live nodes then write
("No answer node"); return;
E = least ( );
}until (false);
}
```

Algorithm

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* 0/1 knapsack problem using least cost search method:

→ Branch and bound technique is used to solve minimization problems where as knapsack problem is a maximization problem. So convert maximization problem into minimization problem by replacing objective function $\Sigma P_i x_i$ by the function $-\Sigma P_i x_i$. The modified knapsack problem is stated as

$$\text{minimize} \; -\sum_{i=1}^{n} P_i x_i$$

$$\text{Subject to} \; \sum_{i=1}^{n} w_i x_i \leq m$$

$$x_i = 0 \text{ or } 1, \; 1 \leq i \leq n.$$

* LC branch and bound solution for knapsack problem:

1) Consider the knapsack instance $n=4$, $(P_1, P_2, P_3, P_4) = (10, 10, 12, 18)$, $(w_1, w_2, w_3, w_4) = (2, 4, 6, 9)$ and $m = 15$. Solve this problem using LC branch and bound.

Sol:- The search begins with root node as E-node

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Initially root node is node '1'. Find lower bound cost and upper bound cost for node 1 using ranking function.

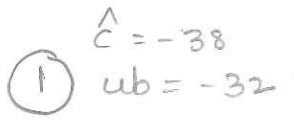
→ lower bound will accept the fractional values and upper bound does not accept fraction values.

Lower bound $(\hat{c}) = \langle P_i x_i \rangle = -(P_1x_1 + P_2x_2 + P_3x_3 + P_4x_4)$

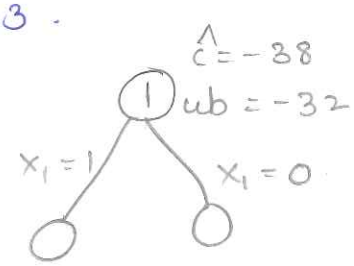
$\hat{c}(lb) = (10 \cdot 1 + 10 \cdot 1 + 12 \cdot 1 + 18 \cdot 3/9) = 38 = -38$

$ub = (10 \cdot 1 + 10 \cdot 1 + 12 \cdot 1 + 18 \cdot 0) = 32 \cdot = -32$

upper bound = -32.



→ Make node 1 as E-node to generate child nodes 2 and 3.



$x_1 = 1$ means first item must be placed in the knapsack.

$x_1 = 0$ means first item should not be placed in the knapsack.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ for node 2 $[x_1 = 1]$: First item must be placed in bag

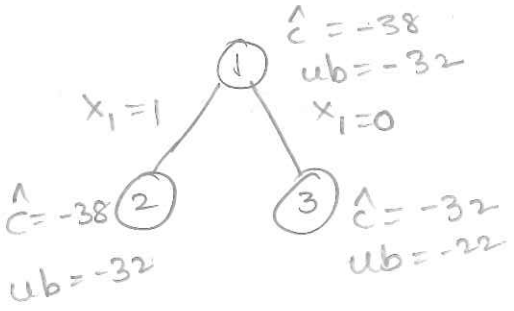
$\hat{c} = 10 \cdot 1 + 10 \cdot 1 + 12 \cdot 1 + 18 \cdot 3/9 = 38 = -38$

$ub = 10 \cdot 0 + 10 \cdot 1 + 12 \cdot 1 + 18 \cdot 0 = 32 = -32$

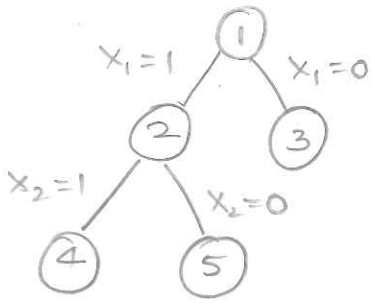
→ for node 3 $[x_1 = 0]$: first item should not be placed in bag.

$\hat{c} = 10 \cdot 0 + 10 \cdot 1 + 12 \cdot 1 + 18 \cdot 5/9 = -32$

$ub = 10 \cdot 0 + 10 \cdot 1 + 12 \cdot 1 + 18 \cdot 0 = -22$



→ upper bound cost of node 2 is -32 which is least compared to node 3. So select node 2 as E-node to generate child nodes. 4 and 5



→ node 4 $[x_2 = 1]$: Second item must be placed in bag

$\hat{c} = lb = 10 \cdot 1 + 10 \cdot 1 + 12 \cdot 1 + 18 \cdot 3/9 = -38$

$ub = 10 \cdot 0 + 10 \cdot 1 + 12 \cdot 1 + 18 \cdot 0 = 32 = -32$

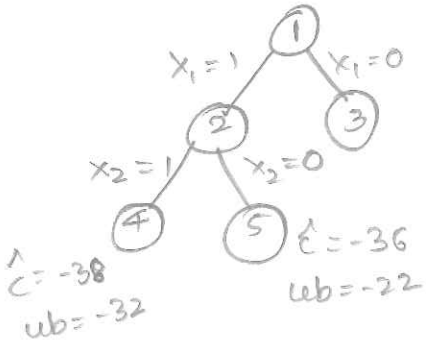
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

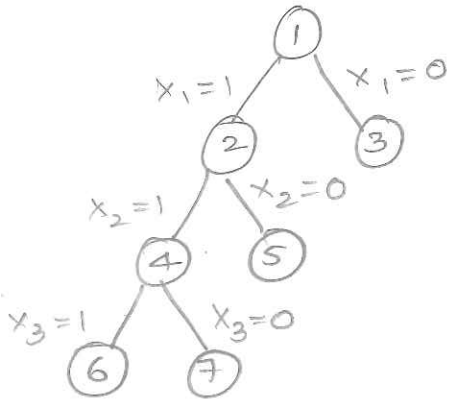
→ node 5 $[x_2 = 0]$: Second item should not be placed in bag

$$\hat{c} = lb = -(10 \cdot 1 + 10 \cdot 0 + 12 \cdot 1 + 18 \cdot 7/9) = -36$$

$$ub = -(10 \cdot 1 + 10 \cdot 0 + 12 \cdot 1 + 18 \cdot 0) = -22$$



→ upper bound cost of node 4 is -32 which is least compared to node 5. So select node 4 as E-node to generate its child nodes 6 & 7.



node 6 $[x_3 = 1]$:

$$\hat{c} = lb = 10 \cdot 1 + 10 \cdot 1 + 12 \cdot 1 + 18 \cdot 3/9 = -38$$

$$ub = 10 \cdot 1 + 10 \cdot 1 + 12 \cdot 1 + 18 \cdot 0 = -32$$

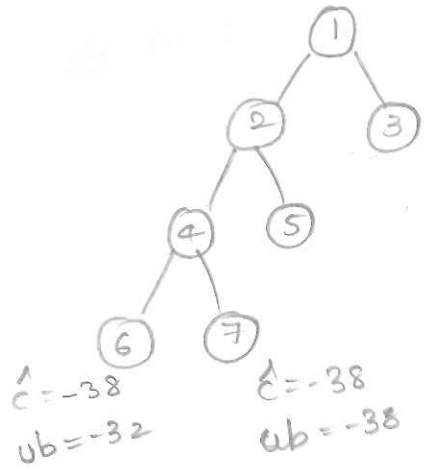
node 7 $[x_3 = 0]$:

$$\hat{c} = lb = -(10 \cdot 1 + 10 \cdot 1 + 18 \cdot 1) = -38$$

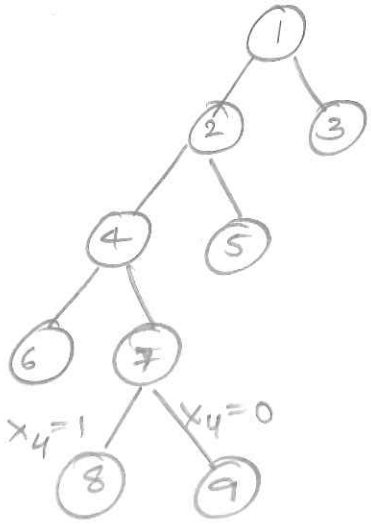
$$ub = -(10 \cdot 1 + 10 \cdot 1 + 18 \cdot 1) = -38$$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh



→ upper bound cost of node 7 is -38 which is least compared to node 6. So select node 7 as E-node to generate its child nodes 8 & 9



node 8 $[x_4 = 1]$:

$$\hat{c} = 10 \cdot 1 + 10 \cdot 1 + 12 \cdot 0 + 18 \cdot 9/9 = -38$$

$$ub = 10 \cdot 1 + 10 \cdot 1 + 12 \cdot 0 + 18 \cdot 9/9 = -38$$

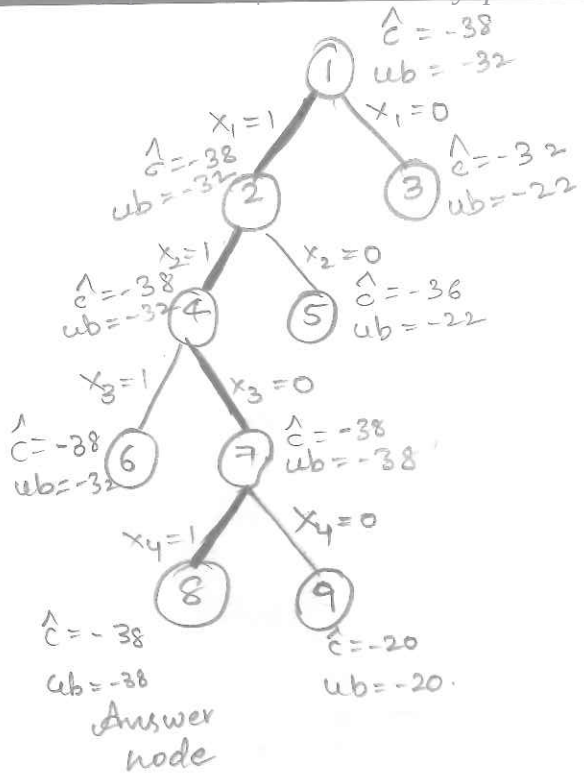
node 9 $[x_4 = 0]$:

$$\hat{c} = 10 \cdot 1 + 10 \cdot 1 + 12 \cdot 0 + 18 \cdot 0 = -20$$

$$ub = 10 \cdot 1 + 10 \cdot 1 + 12 \cdot 0 + 18 \cdot 0 = -20$$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh



→ubcost of node 8 is -38 which is least then compared to node 9 and select this node as answer node.

→ So optimal solution is $x_1 = 1, x_2 = 1, x_3 = 0, x_4 = 1$.

Maximum profit

$$\sum p_i x_i = p_1 x_1 + p_2 x_2 + p_3 x_3 + p_4 x_4$$

$$= 10 \cdot 1 + 10 \cdot 1 + 12 \cdot 0 + 18 \cdot 1$$

$$\sum p_i x_i = 38$$

*Travelling salesperson problem:

→Travelling salesperson problem consists of salesman and set of cities. A salesman has to visit every city, from starting a certain city(home city) and

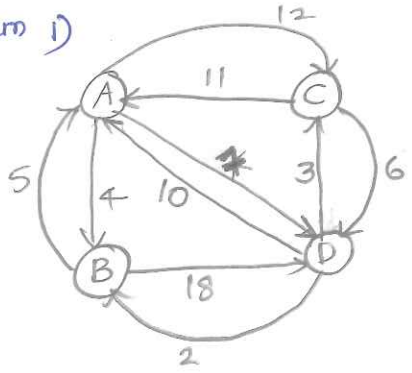
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Come back to same city. The objective of this problem is that salesman wants to reduce cost of the trip.

→ Consider the following graph.

Problem 1)



Sol:- LC branch and bound solution.

Step-1: Represent the given graph in the form of adjacent cost matrix.

	A	B	C	D
A	∞	4	12	7
B	5	∞	∞	18
C	11	∞	∞	6
D	10	2	3	∞

Now reduce the cost matrix by using row reduction and column reduction.

Row reduction:

→ Subtract least element from all elements of the selected row

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Initial

By row reduction

$$\begin{bmatrix} \infty & 4 & 12 & 7 \\ 5 & \infty & \infty & 18 \\ 11 & \infty & \infty & 6 \\ 10 & 2 & 3 & \infty \end{bmatrix} \begin{matrix} 4 \\ 5 \\ 6 \\ 2 \end{matrix} \quad 17$$

$$\Rightarrow \begin{bmatrix} \infty & 0 & 8 & 3 \\ 0 & \infty & \infty & 13 \\ 5 & \infty & \infty & 0 \\ 8 & 0 & 1 & \infty \end{bmatrix}$$

Column reduction:

→ Subtract least element from all elements of selected row

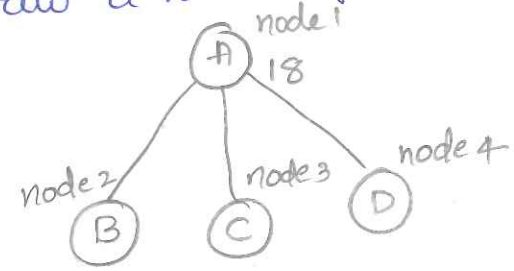
$$\begin{bmatrix} \infty & 0 & 8 & 3 \\ 0 & \infty & \infty & 13 \\ 5 & \infty & \infty & 0 \\ 8 & 0 & 1 & \infty \\ 0 & 0 & 1 & 0 \end{bmatrix} \Rightarrow \begin{bmatrix} \infty & 0 & 7 & 3 \\ 0 & \infty & \infty & 13 \\ 5 & \infty & \infty & 0 \\ 8 & 0 & 0 & \infty \end{bmatrix}$$

→ So cost of node 1 is cost of row reduction + cost of column reduction

node1 cost = 17 + 1

= 18

→ Draw a state space tree



Step-2:

→ Choose vertex 'B' [node 2] to visit (path A → B)

1. From the reduced matrix of step-1 cost of [A, B] = 0

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

2. Set elements of row A and column B to infinite

3. Set cost of $[B, A] = \infty$.

4. The resulting cost matrix is

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 13 \\ 5 & \infty & \infty & 0 \\ 8 & \infty & 0 & \infty \end{bmatrix}$$

Find the reduced cost matrix for above matrix.

Row reduction

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 13 \\ 5 & \infty & \infty & 0 \\ 8 & \infty & \textcircled{\infty} & \infty \end{bmatrix} \begin{matrix} \\ 13 \\ 0 \\ 0 \end{matrix} \Rightarrow \begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 0 \\ 5 & \infty & \infty & 0 \\ 8 & \infty & \textcircled{\infty} & \infty \end{bmatrix}$$

13

Column reduction

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 0 \\ 5 & \infty & \infty & 0 \\ 8 & \infty & 0 & \infty \end{bmatrix} \Rightarrow \begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 0 \\ 0 & \infty & \infty & 0 \\ 3 & \infty & 0 & \infty \end{bmatrix}$$

5 0 0

reduced cost matrix

→ cost of node 2 = cost of node 1 + reduction cost + cost of [A, B]

= 18 + (13 + 5) + 0 = 36.

→ Now determine cost of node 3. choose vertex 3 (node 3) path A → C

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

1. From reduced matrix of step 1, cost $[A, C] = 7$

2. Set all elements of row A and column C to infinite

3. Set cost of $[C, A] = \infty$

4. Now resulting cost matrix is

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & 13 \\ \infty & \infty & \infty & 0 \\ 8 & 0 & \infty & \infty \end{bmatrix}$$

→ This matrix is already reduced the cost of nodes row reduction and column reduction is already done in above matrix.

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & 13 \\ \infty & \infty & \infty & 0 \\ 8 & 0 & \infty & \infty \end{bmatrix}$$

reduced matrix

cost of node 3 = cost of node 1 + reduction cost + cost $[A, C]$

$= 18 + 0 + 7 = 25.$

~~Step 3:~~

→ choose vertex D [node 4] path $[A \to D]$ 1. From the reduced matrix of step 'i' cost of $[A, D]$ is 3

2. Set all elements of row A and column D to ∞

3. Set cost $[D, A] = \infty$

4. ~~Cos~~ Resulting cost matrix is

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty \\ 5 & \infty & \infty & \infty \\ \infty & 0 & \infty & \infty \end{bmatrix}$$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

row reduction

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty \\ 5 & \infty & \infty & \infty \\ \infty & 0 & 0 & \infty \end{bmatrix} \begin{matrix} \infty \\ 0 \\ 5 \\ 0 \end{matrix} \Rightarrow \begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty \\ \infty & 0 & 0 & \infty \end{bmatrix}$$

This matrix is already column reduced.

Cost of node 4 = cost of node 1 + reduction cost + Cost of [A, D]

$= 18 + 5 + 3 = 26.$

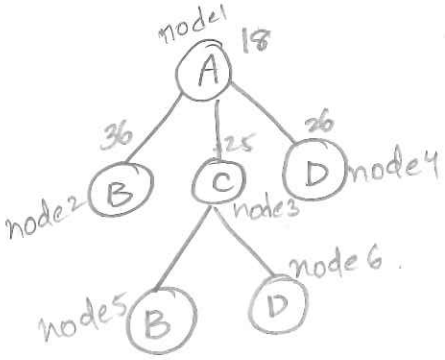
Cost of node 2 = 36 [A → B]

cost of node 3 = 25 [A → C]

cost of node 4 = 26 [A → D]

→ we have to choose the node with least cost

Therefore we choose node 3.



Step-3:

Note that we have to use cost matrix of node 3

(i.e) $\begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & 13 \\ \infty & \infty & \infty & 0 \\ 8 & 0 & \infty & \infty \end{bmatrix}$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ choose vertex B to visit from vertex C [A→C→B] is path.

1. From reduced matrix of step 2 cost of [C, B] = ∞
2. Set all elements of row C and column B to infinite.
3. Set cost of [B, A] to infinite.
4. Now resulting cost matrix is:

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 13 \\ \infty & \infty & \infty & \infty \\ 8 & \infty & \infty & \infty \end{bmatrix}$$

row reduction

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 13 \\ \infty & \infty & \infty & \infty \\ 8 & \infty & \infty & \infty \end{bmatrix} \begin{matrix} \bullet \\ 13 \\ \\ 8 \\ 21 \end{matrix} \Rightarrow \begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 0 \\ \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty \end{bmatrix}$$

This matrix is already column reduced.

cost of node 5 = cost of node 3 + reduction cost + cost(C, B)

$= 25 + 21 + \infty$

$= \infty$.

→ choose vertex D (node 6) to visit (A → C → D)

1. From reduced matrix of step 2 cost of [C, D] = 0
2. Set all elements of row C and column D to infinite
3. Set cost of [D, A] = ∞
4. The resulting cost matrix is

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \\ \infty & 0 & \infty & \infty \end{bmatrix}$$

This matrix is already reduced so that

cost of node 6 = cost of node 3 + reduction cost + cost of [C, D]

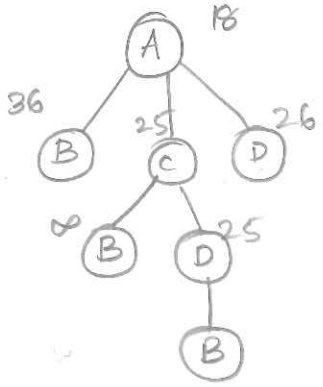
= 25 + 0 + 0

= 25.

→ Thus we have cost(5) = ∞, cost(6) = 25

path [A → C → D]

→ Now LC node is 6 select this node as E-node to generate child nodes.



→ choose vertex B [node 7] path [A → C → D → B]

1. From reduced matrix of step-3 cost of [D, B] = 0
2. Set all elements of row D and column B to infinite
3. Set cost of [B, A] = ∞
4. The resultant matrix is

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \end{bmatrix}$$

→ This matrix is already reduced. So

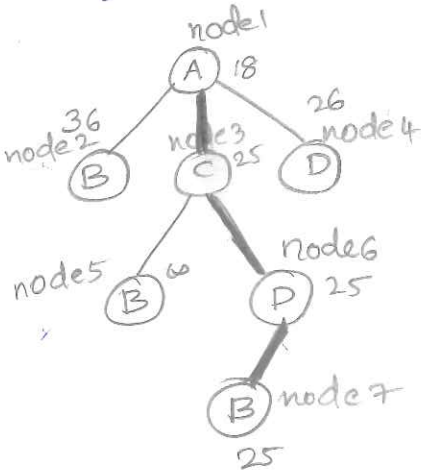
cost of node 7 = cost of node 6 + reduction cost + cost [D, B]

= 25 + 0 + 0

= 25

∴ cost of node 7 = 25

→ Thus optimal path is A → C → D → B → A with the cost of 25.



Solve travelling salesperson problem using FIFO.

www.android.previousquestionpapers.com | www.previous... /telegram.me/jntuh

android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

6/4/18

UNIT-V

NP HARD AND NP-COMPLETE PROBLEMS.

→ Various problems have different time complexities such as

Time complexity	Description
$O(1)$	Constant
$O(n)$	Linear
$O(n^2)$	Quadratic
$O(n^3)$	Cubic
$O(\log n)$	logarithm
$O(n \log n)$	n log n
$O(2^n)$	exponent

Order of time complexity:

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n)$

* P-problems:

→ The problems that can be solvable in polynomial time using deterministic algorithm is called as P-problems.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ Polynomial time can be expressed in big 'Oh' notation that is $O(n^k)$ where $k=0, 1, 2, \ldots$

Ex: Searching of elements, sorting of elements.

* NP-problems:

→ NP stands for non-deterministic polynomial time.

→ NP is set of all decision problems solvable by non-deterministic algorithms in polynomial time.

→ Note that deterministic algorithms are just a special case of non-deterministic algorithms so we conclude that $P \subseteq NP$

→ NP problems catagorised into two classes.

1. NP complete
2. NP Hard.

1. NP complete:

→ A problem that is NP complete has the property that it can be solved in polynomial time if and only if all other NP-complete problems can also be solved in polynomial time.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/intuh

Ex: travelling salesperson problem ($O(n^2 2^n)$), Knapsack problem ($O(2^{n/2})$), Graph colouring, Sums of subsets, Hamiltanion cycle

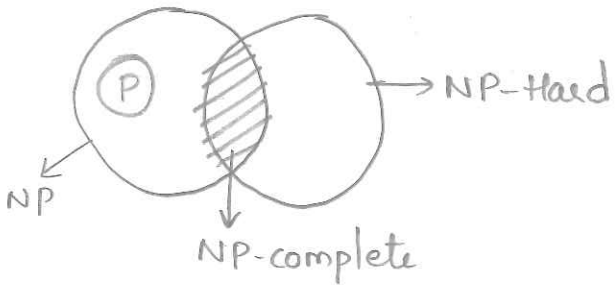
2. NP-Hard problem:

→ If any NP-Hard problem can be solved in polynomial time then all NP complete problems can be solved in polynomial time

→ Note that all NP-complete problems are NP-hard problems but not vice versa.

Ex: Halting problem.

* Relationship among P, NP, NP-complete and NP-Hard:



* Decision problem:

→ Any problem for which the answer is either yes or no is called a decision problem.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/intuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

*Decision Algorithm:

→ The algorithm used for a decision problem is known as decision algorithm.

*Optimization problem:

→ Any problem for which the solution is optimal [minimum or maximum] value is called optimization problem

*Optimization algorithm:

→ The algorithm used to solve a optimization problem is known as optimization algorithm.

*Deterministic algorithm:

→ Deterministic algorithm has the property that the result of every operation is uniquely defined.

*Non-deterministic algorithm:

→ The algorithm has the property that the result of every operation is not uniquely defined but are limited to specified sets of possibilities.

→ Non-deterministic algorithm consists of 3 functions

1. Choice(s):

. This function randomly chooses one of element of set 's'

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

2. Failure():

Signals an unsuccessful completion of algorithm.

3. Success();

Signals a successful completion of algorithm

Ex:1. Searching of element using non-deterministic algorithm.

```
Algorithm search [A[], n, x)
{
j:= choice [i, n];
if (A[j]=x) then
{
write(j);
Success();
}
else
{
write(0);
failwe();
}
}
```

2. Sorting of elements using non-deterministic algorithm.

```
Algorithm Nsort (A, n)
{
for i:=1 to n do B[i]=0;
for i:=1 to n do
```

www.android.previousquestionpapers.com | www.universityupdates.in | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

```
{
j:= choice (i,n);
if (B[j] ≠ 0) then failure(),
B[j] = A[i];
}
for i:=1 to n-1 do
if B[i] > B[i+1] then failure();
write (B[1:n]);
success();
}
```

3. knapsack problem algorithm using NDA:

```
Algorithm DKP (P,w,n,m,r, x)
{
w=0; p=0;
for i:=1 to n do
{
x[i]:= choice(0,1);
w = w+x[i]·w[i];
P = P+x[i]·P[i];
}
if ((w>m) or (p<r)) then
failure ();
else
  success();
  }
```

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

*cliqueproblem:

Consider undirected graph G=(V, E).

Clique:

A subset of vertices in V all connected to each other by edges in E.[i.e forming a complete graph]

Size of clique:

No. of vertices it contains

Max clique:

A maximal complete subgraph of graph 'G' is called a clique. whose size is no. of vertices in it.

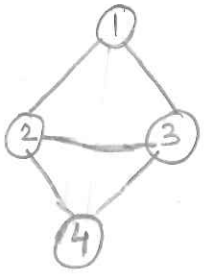
Optimization problem:

Find the maximal clique in a given graph 'G'.

Decision problem:

Find whether graph 'G' has a clique of size atleast k for some given 'k'.

Ex:



G = (V, E)

V = {1, 2, 3, 4}

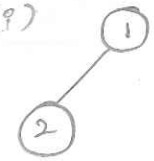
E = {(1,2), (1,3), (2,3), (2,4), (3,4)}

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

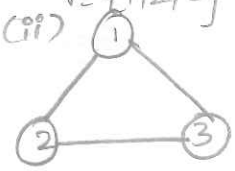
→ The above graph consisting of two cliques

(i)



clique(G,2) = Yes

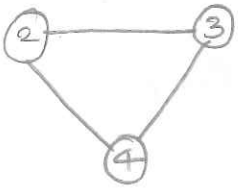
(ii) V={1,2,3}



clique(G,3) = Yes

It is complete graph and also max clique.

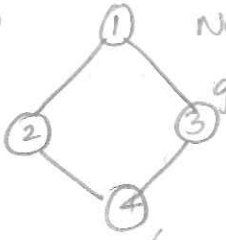
(iii)



V={2,3,4}

clique (G,3) = Yes

(iv)



Not a complete graph, not clique.

clique (G,4) = NO

→ Complete graph is a graph in which every vertex is connected to remaining vertices.

* Satisfiability:

→ The satisfiability problem is to determine whether the formula is true for some assignment of truth values to the variables

→ Let $x_1, x_2, \ldots x_n$ denote boolean variables (there value is either true or false)

Ex: Conjunctive normal form (CNF), Disjunctive normal form (DNF)

* Conjunctive normal formal form:

→ A formula is in CNF iff it is represented as $\bigwedge_{i=1}^{k} C_i$ where C_i are clauses each represented as

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$\vee l_{ij}$ where l_{ij} are literals.

Ex: 2-CNF $= (x_3 \vee \bar{x}_4) \wedge (x_1 \vee \bar{x}_2)$

3-CNF $= \underbrace{(x_1 \vee x_2 \vee x_4)}_{\text{clause-1}} \wedge \underbrace{(x_3 \vee \bar{x}_4 \vee x_5)}_{\text{clause-2}}$

Here each clause consisting of three literals. So this formula is said to be 3CNF.

* Disjunctive normal form (DNF):

A formula is in DNF iff it is represented as $\vee_{i=1}^{k} c_i$ and clause c_i is represented as $\wedge l_{ij}$ are literals.

Ex: 2-DNF $= (x_3 \wedge \bar{x}_4) \vee (x_1 \wedge \bar{x}_2)$

3-DNF $= (x_1 \wedge x_2 \wedge x_4) \vee (x_3 \wedge \bar{x}_4 \wedge x_5)$

Ex: 1. This formula is said to be ~~for~~ satisfiability

$p \wedge q$ where $p = T$

$T \wedge T$ $q = T$

T

2. $p \wedge \neg p$

$T \wedge F$ $p = T$

F

This formula is said to be not satisfiability.

www.android.previousquestionpapers.com | www.universityupdates.in | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* Halting problem:

→ Best example of NP- hard decision problem is halting problem.

→ The halting problem is to determine for an arbitary deterministic algorithm A and an input i whether algorithm A with input 'i' ever terminates or enters an infinite loop.

→ This problem is undecidable. Hence there exists no algorithm to solve this problem. So it clearly cannot be NP.

→ To show satisfiability of halting problem simply construct an algorithm 'A' whose input is a proportional formula 'x'. If x has n variables then algorithm 'A' tries out all 2^n possible truth assignments and verifies whether x is satisfiable.

→ If it is satisfiable then algorithm 'A' terminates successfully if it is not then algorithm 'A' enters infinite loop. Hence algorithm 'A' halts on input 'x' iff x is satisfiable.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* Cook's theorem:

→ Cook's theorem states that satisfiability is in P. iff P=NP.

→ What is satisfiability:

The satisfiability problem is to determine whether a formula is true for some assignment of truth values to the variables.

for ex: 1. $P \wedge q$ is satisfiable if $P=T, q=T$

2. $P \wedge \neg P$ is not satisfiable if $P=T$ or $P=F$

Theorem:

Satisfiability problem is NP-complete. The proof consists of two steps.

1. Convert the execution of polynomial time non-deterministic machine to a bunch of well-formed formula such that formula satisfies iff the machine accept input.
2. Show the sum of lengths of formula is polynomial in the size of problem

Step-1: NP-Hard (L) - can polynomially reduce any NP problem to L.

Step-2: NP-complete - $L \in NP$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Step-3: $L \in NP \rightarrow$ Non deterministic machine for L that runs in polynomial time.

→ A problem that is NP-complete has the property that it can be solved in polynomial time iff all other NP-complete problems can also be solved in polynomial time.

→ If an NP-Hard problem can be solved in polynomial time then all NP-complete problems can be solved in polynomial time.

→ Satisfiability problem (SAT) SAT $\in$ NP.

Therefore if we can polynomially reduce and an arbitary polynomial non-deterministic machine to satisfiability problem [SAT]. It means we have proved that satisfiability problem is NP-complete.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

7/4/17

UNIT-III

* Reliability design problem:

→ The problem is to design a system that is composed of several devices connected in series.

→ Let 'r_i' be the reliability of the device 'D_i' [i.e r_i is the probability ~~of the entire system~~ that device 'i' will function properly].

→ Then the reliability of entire system is 'Πr_i'.

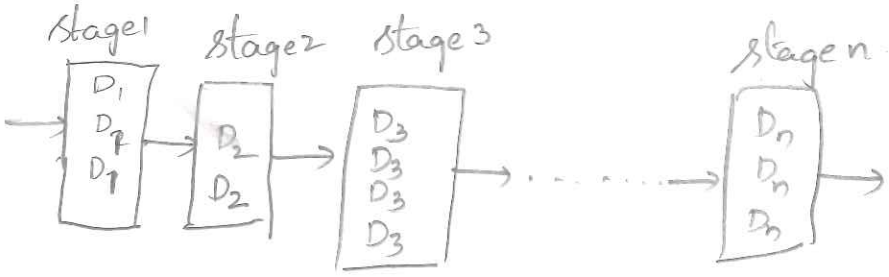
→ Even if the individual devices are very reliable then the reliability of system may not be very good.

for example: If $n=10$, $r_i = 0.99$, $1 \le i \le 10$. then

$\Pi r_i = 0.99 \times 3.14 = 0.904$

→ Hence it is required to use duplicate devices.

→ Multiple copies of same device type are connected in parallel through the use of switching circuit.



www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Step-3: $L \in NP \rightarrow$ Non deterministic machine for L that runs in polynomial time.

→ A problem that is NP-complete has the property that it can be solved in polynomial time iff all other NP-complete problems can also be solved in polynomial time.

→ If an NP-Hard problem can be solved in polynomial time then all NP-complete problems can be solved in polynomial time.

→ Satisfiability problem (SAT) SAT $\in$ NP.

Therefore if we can polynomially reduce and an arbitary polynomial non-deterministic machine to satisfiability problem [SAT]. It means we have proved that satisfiability problem is NP-complete.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

7/4/17

UNIT-III

* Reliability design problem:

→ The problem is to design a system that is composed of several devices connected in series.

→ Let 'r_i' be the reliability of the device 'D_i' [i.e r_i is the probability ~~of the entire system~~ that device 'i' will function properly].

→ Then the reliability of entire system is 'Πr_i'.

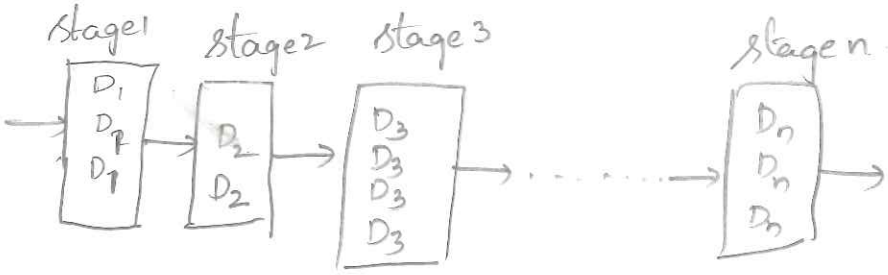
→ Even if the individual devices are very reliable then the reliability of system may not be very good.

for example: If $n=10$, $r_i = 0.99$, $1 \le i \le 10$. then

$\Pi r_i = 0.99 \times 3.14 = 0.904$

→ Hence it is required to use duplicate devices.

→ Multiple copies of same device type are connected in parallel through the use of switching circuit.



www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Problem: Design a 3 stage system with device types D_1, D_2, D_3. The cost are 30, 15, 20 $ respectively. The cost of system should no more than 105 $. The reliability of each device type is 0.9, 0.8, 0.5 respectively.

Sol:- Our problem is to use device duplication to maximize reliability. This maximization is to be carried out under a cost constraint.

→ Let c_i be the cost of each device 'i' and c be the maximum allowable cost of the system being designed.

→ Maximize $\prod_{1 \le i \le n} \phi_i(m_i)$

Subject to $\sum_{1 \le i \le n} c_i m_i \le c$

$m_i \ge 1$ and integer, $1 \le i \le n$

$$u_i = \left\lfloor \left(c + c_i - \sum_{1}^{n} c_j\right) / c_i \right\rfloor$$

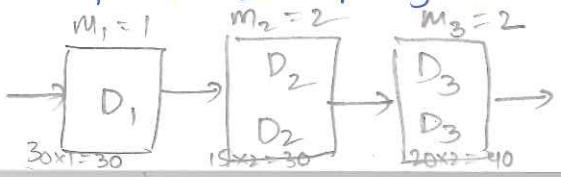
$$\phi(m_i) = 1 - (1 - r_i)^{m_i}$$

→ $c_1 = 30, \; c_2 = 15, \; c_3 = 20$

System cost = 105

$r_1 = 0.9, \; r_2 = 0.8, \; r_3 = 0.5$

$m_1 = 1, \; m_2 = 2, \; m_3 = 2$



www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$= 30+30+40$

$= 100$

→ The best design has a reliability of 0.648 and cost of 100 dollars ($)

UNIT-I

Amortized analysis:

→ In an amortized analysis, the time required to perform a sequence of data structure operation is average over all operation performed. Amortised analysis can be used to show that average cost of an operation is small, if one average over a sequence of operations, even though a single operation might be expensive. Unlike the average probability distribution functions, the amortized analysis guarantees the 'average' performance of each operation in worst case.

→ CLR covers three most common techniques used in amortised analysis. The main difference is way the cost is assign.

www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

1. Aggregate method:

→ Computes an upper bound ($T(n)$) on total cost of a sequence of n operations.

2. Accounting method:

→ Overcharge some operations early in sequence. This 'overcharge' is used later in sequence for pay operation that charges less than they actually cost.

3. Potential method:

→ Maintain the credit as the potential energy to pay for future operations.

*Aggregate method characteristics:

→ It computes the worst case time $T(n)$ for a sequence of n operations.

→ The amortised cost is $T(n)/n$ per operation.

→ It gives the average performance of each Operation in worst case.

→ This method is less precise than other methods, as all operations are assigned the same cost.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

*Accounting method characteristics:

→In this method, we assign charges to different operations charged more or less than they actually cost. In other words, we assign artificial charges to different operations.

1. Any overcharge for an operation on an item is stored (in an bank account) reserved for that item.

2. Later, a different ~~question~~ operation on that item can pay for its cost with the credit for that item.

3. The balanced in (bank) account is not allowed to become negative.

4. The sum of amortized cost ~~of each operation must~~ for any sequence of operations is an upper bound for the actual total cost of these operations.

5. The amortized cost of each operation must be choosen wisely in order to pay for each operation at or before the cost is incurred.

*Potential method characteristics:

→This method stores pre-payments as potential or potential energy that can be released to pay for

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

future operations. The stored potential is associated with the entire data structure rather than specific objects within the data structure.

Notation:

→ D_0 is initial data structure (ex: stack)

→ D_i is data structure after i^{th} operation.

→ C_i is actual cost of i^{th} operation.

→ The potential function ψ maps each D_i to its potential value $\psi(D_i)$

→ Live node: A node which has been generated and all of whose children have not yet generated.

→ E-node: The live node whose children are currently being generated.

→ Dead-node: which is not expanded further or all of whose children have been generated.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh